

# Teoria di Java

Alessandro Stella

July 5, 2026

## Contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                     | <b>2</b>  |
| 1.1      | Compilazione del codice                                 | 2         |
| 1.2      | Java Platform                                           | 2         |
| 1.3      | Hello World!                                            | 2         |
| <b>2</b> | <b>Concetti fondamentali dell'OOP</b>                   | <b>3</b>  |
| 2.1      | Oggetti                                                 | 3         |
| 2.2      | Classi                                                  | 3         |
| 2.3      | Ereditarietà                                            | 3         |
| 2.4      | Interfacce                                              | 4         |
| 2.5      | Package                                                 | 5         |
| <b>3</b> | <b>Basi del linguaggio Java</b>                         | <b>6</b>  |
| 3.1      | Variabili                                               | 6         |
| 3.1.1    | Tipi di dati primitivi                                  | 7         |
| 3.1.2    | Tipi riferimento                                        | 7         |
| 3.2      | String                                                  | 8         |
| 3.2.1    | Concetto generale                                       | 8         |
| 3.2.2    | Creazione di stringhe                                   | 8         |
| 3.2.3    | Lunghezza e accesso ai caratteri                        | 8         |
| 3.2.4    | Conversione tra stringhe e array di caratteri           | 9         |
| 3.2.5    | Concatenazione                                          | 10        |
| 3.2.6    | Stringhe formattate                                     | 10        |
| 3.2.7    | Conversione da stringhe a numeri                        | 11        |
| 3.2.8    | Conversione da numeri a stringhe                        | 11        |
| 3.2.9    | Ricerca e manipolazione                                 | 12        |
| 3.2.10   | Confronto tra stringhe                                  | 13        |
| 3.2.11   | StringBuilder                                           | 14        |
| 3.3      | Array                                                   | 16        |
| 3.4      | Enum                                                    | 16        |
| 3.5      | Annotation                                              | 17        |
| 3.6      | Operatori                                               | 17        |
| <b>4</b> | <b>Classi e Oggetti</b>                                 | <b>18</b> |
| 4.1      | Dichiarazione di attributi                              | 18        |
| 4.2      | Modificatori d'accesso                                  | 18        |
| 4.3      | Metodi e Overloading                                    | 18        |
| 4.4      | Costruttori                                             | 19        |
| 4.5      | Passaggio di parametri ad un metodo o ad un costruttore | 19        |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 4.5.1    | Passaggio per valore . . . . .                                 | 20        |
| 4.5.2    | Passaggio per riferimento . . . . .                            | 20        |
| 4.6      | Membri statici . . . . .                                       | 21        |
| 4.6.1    | Variabili statiche . . . . .                                   | 21        |
| 4.6.2    | Metodi statici . . . . .                                       | 21        |
| 4.6.3    | Accesso tra membri statici e membri di istanza . . . . .       | 22        |
| 4.6.4    | Costanti . . . . .                                             | 22        |
| 4.7      | Inizializzazione dei campi . . . . .                           | 23        |
| 4.7.1    | Blocchi di inizializzazione statici . . . . .                  | 23        |
| 4.7.2    | Inizializzazione delle istanze . . . . .                       | 23        |
| 4.7.3    | Metodi <code>final</code> per l'inizializzazione . . . . .     | 24        |
| <b>5</b> | <b>Oggetti . . . . .</b>                                       | <b>25</b> |
| 5.1      | Creazione di oggetti . . . . .                                 | 25        |
| 5.1.1    | Dichiarazione . . . . .                                        | 25        |
| 5.1.2    | Istanziamento . . . . .                                        | 25        |
| 5.1.3    | Inizializzazione . . . . .                                     | 25        |
| 5.2      | Oggetti temporanei . . . . .                                   | 25        |
| 5.3      | Garbage Collector . . . . .                                    | 25        |
| 5.4      | Keyword <code>this</code> . . . . .                            | 26        |
| <b>6</b> | <b>Classi annidate . . . . .</b>                               | <b>27</b> |
| 6.1      | Perchè usare le classi annidate . . . . .                      | 27        |
| 6.2      | Inner classes . . . . .                                        | 27        |
| 6.3      | Local classes . . . . .                                        | 27        |
| 6.4      | Anonymous classes . . . . .                                    | 28        |
| 6.5      | Lambda Expressions . . . . .                                   | 28        |
| 6.5.1    | Method references . . . . .                                    | 29        |
| <b>7</b> | <b>Annotations . . . . .</b>                                   | <b>30</b> |
| 7.1      | Concetti fondamentali . . . . .                                | 30        |
| 7.2      | Dichiarazione di un tipo di annotazione . . . . .              | 30        |
| 7.3      | Tipi di annotazione predefiniti . . . . .                      | 31        |
| 7.4      | Annotazioni che si applicano ad altre annotazioni . . . . .    | 31        |
| 7.5      | Annotazioni di tipo e sistemi di tipi estendibili . . . . .    | 32        |
| 7.6      | Annotazioni ripetibili . . . . .                               | 32        |
| 7.7      | Recupero delle annotazioni . . . . .                           | 32        |
| 7.8      | Scelte progettuali . . . . .                                   | 33        |
| <b>8</b> | <b>Interfacce . . . . .</b>                                    | <b>34</b> |
| 8.1      | Usare un'interfaccia come un tipo . . . . .                    | 34        |
| 8.2      | Evoluzione delle interfacce . . . . .                          | 35        |
| 8.3      | Default methods . . . . .                                      | 35        |
| <b>9</b> | <b>Ereditarietà . . . . .</b>                                  | <b>36</b> |
| 9.1      | Casting . . . . .                                              | 36        |
| 9.2      | Ereditarietà multipla: stato, implementazione e tipo . . . . . | 36        |
| 9.3      | Sovrascrivere e nascondere metodi . . . . .                    | 36        |
| 9.3.1    | Metodi d'istanza . . . . .                                     | 36        |
| 9.3.2    | Metodi statici . . . . .                                       | 37        |
| 9.3.3    | Metodi d'interfaccia . . . . .                                 | 37        |
| 9.3.4    | Override di metodi . . . . .                                   | 38        |

|           |                                                         |           |
|-----------|---------------------------------------------------------|-----------|
| 9.4       | Polimorfismo . . . . .                                  | 39        |
| 9.5       | Keyword <b>super</b> . . . . .                          | 40        |
| 9.5.1     | Accedere ai membri della superclasse . . . . .          | 40        |
| 9.5.2     | Costruttori della sottoclasse . . . . .                 | 40        |
| 9.6       | Object come Superclasse . . . . .                       | 41        |
| 9.6.1     | Metodo <b>clone</b> . . . . .                           | 41        |
| 9.6.2     | Metodo <b>equals</b> . . . . .                          | 41        |
| 9.6.3     | Metodo <b>finalize</b> . . . . .                        | 41        |
| 9.6.4     | Metodo <b>getClass</b> . . . . .                        | 41        |
| 9.6.5     | Metodo <b>hashCode</b> . . . . .                        | 42        |
| 9.6.6     | Metodo <b>toString</b> . . . . .                        | 42        |
| 9.7       | Classi e metodi <b>final</b> . . . . .                  | 43        |
| 9.8       | Classi e metodi astratti . . . . .                      | 43        |
| 9.8.1     | Classi astratte vs Interfacce . . . . .                 | 43        |
| 9.8.2     | Classe astratta che implementa un'interfaccia . . . . . | 44        |
| <b>10</b> | <b>Strutture dati e Collections . . . . .</b>           | <b>45</b> |
| 10.1      | Introduzione alle Collections . . . . .                 | 45        |
| 10.2      | Collections Framework . . . . .                         | 45        |
| 10.3      | Componenti principali del framework . . . . .           | 45        |
| 10.4      | Vantaggi del Java Collections Framework . . . . .       | 46        |
| 10.5      | Uso delle interfacce come tipo . . . . .                | 46        |
| 10.6      | Interfaccia Collection . . . . .                        | 48        |
| 10.6.1    | Concetto generale . . . . .                             | 48        |
| 10.6.2    | Metodi principali . . . . .                             | 48        |
| 10.6.3    | Attraversamento di una Collection . . . . .             | 49        |
| 10.6.4    | Operazioni bulk . . . . .                               | 49        |
| 10.6.5    | Conversione in array . . . . .                          | 50        |
| 10.6.6    | Stream . . . . .                                        | 51        |
| 10.7      | Set . . . . .                                           | 52        |
| 10.7.1    | Concetto generale . . . . .                             | 52        |
| 10.7.2    | Implementazioni principali . . . . .                    | 52        |
| 10.7.3    | Rimozione dei duplicati . . . . .                       | 53        |
| 10.7.4    | Operazioni insiemistiche . . . . .                      | 54        |
| 10.8      | List . . . . .                                          | 55        |
| 10.8.1    | Concetto generale . . . . .                             | 55        |
| 10.8.2    | Implementazioni principali . . . . .                    | 55        |
| 10.8.3    | Operazioni principali . . . . .                         | 55        |
| 10.8.4    | Concatenazione di liste . . . . .                       | 56        |
| 10.8.5    | Arrays.asList . . . . .                                 | 56        |
| 10.8.6    | ListIterator . . . . .                                  | 57        |
| 10.8.7    | subList . . . . .                                       | 58        |
| 10.8.8    | Algoritmi sulle liste . . . . .                         | 58        |
| 10.9      | Queue . . . . .                                         | 60        |
| 10.9.1    | Concetto generale . . . . .                             | 60        |
| 10.9.2    | Metodi principali . . . . .                             | 60        |
| 10.9.3    | Implementazioni principali . . . . .                    | 61        |
| 10.9.4    | Code limitate e valori null . . . . .                   | 61        |
| 10.10     | Deque . . . . .                                         | 62        |
| 10.10.1   | Concetto generale . . . . .                             | 62        |
| 10.10.2   | Metodi principali . . . . .                             | 62        |
| 10.10.3   | Uso come coda . . . . .                                 | 63        |

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| 10.10.4   | Usò come stack . . . . .                                 | 63        |
| 10.10.5   | Rimozione di occorrenze . . . . .                        | 63        |
| 10.11     | Map . . . . .                                            | 64        |
| 10.11.1   | Concetto generale . . . . .                              | 64        |
| 10.11.2   | Implementazioni principali . . . . .                     | 64        |
| 10.11.3   | Operazioni principali . . . . .                          | 65        |
| 10.11.4   | Esempio: tabella delle frequenze . . . . .               | 65        |
| 10.11.5   | Operazioni bulk . . . . .                                | 66        |
| 10.11.6   | Collection views . . . . .                               | 67        |
| 10.11.7   | Modifica durante l'iterazione . . . . .                  | 67        |
| 10.11.8   | Map come multimap . . . . .                              | 68        |
| 10.12     | Ordinamento, SortedSet e SortedMap . . . . .             | 70        |
| 10.12.1   | Ordinamento naturale e Comparable . . . . .              | 70        |
| 10.12.2   | Comparator . . . . .                                     | 71        |
| 10.12.3   | SortedSet . . . . .                                      | 72        |
| 10.12.4   | Operazioni principali di SortedSet . . . . .             | 72        |
| 10.12.5   | SortedMap . . . . .                                      | 73        |
| 10.12.6   | Operazioni principali di SortedMap . . . . .             | 73        |
| 10.12.7   | TreeSet e TreeMap con Comparator . . . . .               | 74        |
| <b>11</b> | <b>Exceptions . . . . .</b>                              | <b>76</b> |
| 11.1      | Catch or Specify . . . . .                               | 76        |
| 11.2      | Tipi di eccezioni . . . . .                              | 76        |
| 11.3      | Catturare e gestire le eccezioni . . . . .               | 77        |
| 11.3.1    | Costrutto <code>try-catch-finally</code> . . . . .       | 77        |
| 11.3.2    | Statement <code>try-with-resources</code> . . . . .      | 78        |
| 11.4      | Specificare le eccezioni lanciate da un metodo . . . . . | 78        |
| 11.5      | Lanciare eccezioni . . . . .                             | 79        |
| 11.6      | Eccezioni concatenate . . . . .                          | 79        |
| 11.7      | Stack trace . . . . .                                    | 79        |
| 11.8      | Quando e come creare una eccezione custom . . . . .      | 80        |
| 11.9      | Unchecked exceptions . . . . .                           | 80        |
| 11.10     | Vantaggi delle eccezioni . . . . .                       | 81        |
| <b>12</b> | <b>I/O . . . . .</b>                                     | <b>82</b> |
| 12.1      | Byte Streams . . . . .                                   | 82        |
| 12.2      | Character Streams . . . . .                              | 83        |
| 12.2.1    | Line-Oriented I/O . . . . .                              | 84        |
| 12.3      | Buffered Streams . . . . .                               | 85        |
| 12.4      | Scansionare i dati . . . . .                             | 85        |
| 12.5      | Formattare i dati . . . . .                              | 87        |
| 12.6      | I/O da Riga Comando . . . . .                            | 88        |
| 12.7      | Data Streams . . . . .                                   | 89        |
| 12.7.1    | Scrittura dei dati . . . . .                             | 89        |
| 12.7.2    | Lettura dei dati . . . . .                               | 89        |
| 12.7.3    | Fine del file . . . . .                                  | 90        |
| 12.7.4    | Nota sui valori monetari . . . . .                       | 90        |
| 12.8      | Object Streams . . . . .                                 | 91        |
| 12.8.1    | Serializzazione . . . . .                                | 91        |
| 12.8.2    | Oggetti complessi . . . . .                              | 91        |
| 12.8.3    | Riferimenti condivisi . . . . .                          | 91        |
| 12.8.4    | Uso con tipi primitivi . . . . .                         | 92        |

|           |                                           |            |
|-----------|-------------------------------------------|------------|
| <b>13</b> | <b>Reflection e oggetti Class</b>         | <b>93</b>  |
| 13.1      | Operazioni principali della Reflection    | 93         |
| 13.2      | Ottenere un oggetto Class                 | 93         |
| 13.3      | Metodi che restituiscono oggetti Class    | 94         |
| <b>14</b> | <b>Generics</b>                           | <b>95</b>  |
| 14.1      | Introduzione                              | 95         |
| 14.2      | Generic Types                             | 95         |
| 14.2.1    | Convenzioni sui nomi                      | 96         |
| 14.2.2    | Istanziare un tipo generico               | 96         |
| 14.2.3    | Parametri di tipo multipli                | 96         |
| 14.3      | Raw Types                                 | 97         |
| 14.4      | Generic Methods                           | 98         |
| 14.5      | Bounded Type Parameters                   | 99         |
| 14.5.1    | Bound multipli                            | 99         |
| 14.6      | Metodi generici e bounded type parameters | 100        |
| 14.7      | Generics, ereditarietà e sottotipi        | 101        |
| 14.7.1    | Classi generiche e sottotipi              | 101        |
| 14.8      | Type Inference                            | 102        |
| 14.8.1    | Metodi generici                           | 102        |
| 14.8.2    | Diamond operator                          | 102        |
| 14.8.3    | Target Type                               | 102        |
| 14.9      | Wildcards                                 | 103        |
| 14.9.1    | Upper bounded wildcard                    | 103        |
| 14.9.2    | Unbounded wildcard                        | 103        |
| 14.9.3    | Lower bounded wildcard                    | 104        |
| 14.10     | Wildcards e sottotipi                     | 105        |
| 14.11     | Type Erasure                              | 106        |
| 14.11.1   | Erasure dei tipi generici                 | 106        |
| 14.11.2   | Erasure dei metodi generici               | 107        |
| 14.11.3   | Bridge methods                            | 107        |
| 14.12     | Non-Reifiable Types                       | 108        |
| 14.13     | Restrizioni dei Generics                  | 109        |
| <b>15</b> | <b>ClassLoader</b>                        | <b>110</b> |
| 15.1      | Concetto generale                         | 110        |
| 15.2      | Classi array                              | 110        |
| 15.3      | ClassLoader e risorse                     | 110        |
| 15.4      | Modello di delega                         | 110        |
| 15.5      | ClassLoader predefiniti                   | 111        |
| 15.6      | Metodi principali                         | 112        |
| 15.7      | ClassLoader personalizzati                | 112        |
| 15.8      | Nome binario delle classi                 | 113        |
| 15.9      | defineClass e sicurezza                   | 114        |
| 15.10     | Package e moduli                          | 114        |
| 15.11     | Caricamento concorrente                   | 114        |
| 15.12     | ClassLoader e Reflection                  | 115        |
| <b>16</b> | <b>Maven e gestione delle dipendenze</b>  | <b>116</b> |
| 16.1      | Cos'è Maven                               | 116        |
| 16.2      | Project Object Model                      | 116        |
| 16.3      | Struttura standard di un progetto Maven   | 117        |

|      |                           |     |
|------|---------------------------|-----|
| 16.4 | Dipendenze . . . . .      | 117 |
| 16.5 | Repository . . . . .      | 118 |
| 16.6 | Build lifecycle . . . . . | 119 |
| 16.7 | Plugin . . . . .          | 120 |

# 1 Introduzione

## 1.1 Compilazione del codice

Tutti i programmi in Java vengono scritti in file con l'estensione `.java`; in seguito i file vengono compilati in file `.class` dal compilatore `javac`.

Il file compilato contiene bytecode interpretabile dalla Java Virtual Machine, che si occupa di eseguire il bytecode usando chiamate specifiche dell'architettura in cui è installata la JVM.

In pratica, Java viene compilato nel bytecode, che però può essere interpretato dalla JVM così com'è oppure potrebbe fare una compilazione aggiuntiva per ottimizzare alcune parti del codice per l'architettura specifica.

## 1.2 Java Platform

La piattaforma Java è composta dalla Java Virtual Machine e dalla Java Application Programming Interface (API).

L'API contiene una collezione di elementi già utilizzabili, raggruppati in pacchetti di interfacce e classi già utilizzabili, chiamati "packages".

La piattaforma Java rallenta leggermente l'esecuzione del codice, a scapito però di una portabilità assoluta del codice scritto.

## 1.3 Hello World!

```
1 public class Main {  
2     public static void main(String[] args) {  
3         System.out.println("Java e' compilato in bytecode");  
4     }  
5 }
```

Ogni applicazione in Java contiene un metodo `main`, il punto d'inizio dell'esecuzione del programma.

Dentro `String[] args` sono contenuti ulteriori parametri passati a riga comando, ad esempio lanciando con `java MyApp arg1 arg2` il vettore sarà `args = [arg1, arg2]`.

## 2 Concetti fondamentali dell'OOP

### 2.1 Oggetti

Un oggetto è una rappresentazione di un'entità attraverso uno stato e un comportamento. Come un cane può avere delle caratteristiche, per esempio il colore, e può eseguire delle azioni, come abbaiare o correre, anche un oggetto può avere al suo interno un insieme di dati e metodi per definire stati e comportamenti.

Questo comporta diversi vantaggi durante la scrittura del codice:

1. **Modularità:** il codice per un oggetto può essere mantenuto e modificato indipendentemente dal resto del programma;
2. **Occultamento:** interagendo solo con i metodi dell'oggetto non ci è necessario conoscere l'implementazione interna delle funzionalità;
3. **Riutilizzabilità:** se un oggetto già esiste non è necessario scrivere nuovamente l'intero codice, velocizzando di molto la scrittura del programma;
4. **Sostituibilità:** se un oggetto ha un problema/bug è semplice sostituirlo o fare debugging solo di quello.

### 2.2 Classi

Una classe può essere vista come uno schema riutilizzabile per creare oggetti. Nella programmazione ad oggetti, si può dire che l'oggetto `Obj` è un'istanza della classe di oggetti conosciuta come `Obj`.

All'interno di una classe vanno definiti attributi e metodi degli oggetti che saranno istanze di essa, definendo le caratteristiche degli attributi e il funzionamento dei metodi.

Non contengono un metodo `main` poiché una classe usata per rappresentare un oggetto deve fare solo quello; l'uso di suddetta classe viene lasciato ad altre classi del programma.

### 2.3 Ereditarietà

Nella programmazione ad oggetti un concetto fondamentale è la riusabilità e prevedibilità del codice; per conseguire questo obiettivo è stato introdotto il concetto di "ereditarietà": quando due oggetti hanno molto in comune, come ad esempio `Bicycle` e `MountainBike`, uno dei due oggetti può ereditare attributi e metodi definiti nella classe "padre".

Ogni classe padre, detta "superclasse", può avere un numero illimitato di "sottoclassi", mentre ogni classe può avere una sola superclasse.

```
1 class MountainBike extends Bicycle {  
2     // new fields and methods defining  
3     // a mountain bike would go here  
4 }
```

In questo modo `MountainBike` ha tutti gli attributi e metodi di `Bicycle`, ma è possibile dichiarare codice specifico solo per `MountainBike`.

## 2.4 Interfacce

Un'interfaccia può essere vista come un contratto che una classe si impegna a rispettare: è un gruppo di metodi con corpo vuoto che dovranno essere implementati dalla classe che implementa l'interfaccia.

```
1 interface Bicycle {
2     // wheel revolutions per minute
3     void changeCadence(int newValue);
4
5     void changeGear(int newValue);
6     void speedUp(int increment);
7     void applyBrakes(int decrement);
8 }
```

```
1 class ACMEBicycle implements Bicycle {
2     int cadence = 0;
3     int speed = 0;
4     int gear = 1;
5
6     // The compiler will now require that methods
7     // changeCadence, changeGear, speedUp, and applyBrakes
8     // all be implemented. Compilation will fail if those
9     // methods are missing from this class.
10
11     void changeCadence(int newValue) {
12         cadence = newValue;
13     }
14
15     void changeGear(int newValue) {
16         gear = newValue;
17     }
18
19     void speedUp(int increment) {
20         speed = speed + increment;
21     }
22
23     void applyBrakes(int decrement) {
24         speed = speed - decrement;
25     }
26
27     void printStates() {
28         System.out.println("cadence:" +
29             cadence + " speed:" +
30             speed + " gear:" + gear);
31     }
32 }
```

## 2.5 Package

Un pacchetto è un namespace che organizza un insieme di classi e interfacce legate concettualmente tra loro. Si può vedere come cartelle del computer, usate per organizzare un progetto che si sta creando. Diventa necessario organizzare un progetto dato che potrebbe essere composto da centinaia o migliaia di classi individuali.

La piattaforma Java offre un'enorme libreria di classi (un insieme di pacchetti) detta "Application Programming Interface", in cui vengono implementate tutte le generiche funzionalità di cui un programmatore potrebbe aver bisogno, permettendo al developer di concentrarsi supramente sul design dell'applicazione che sta sviluppando senza doversi preoccupare dell'implementazione a basso livello.

## 3 Basi del linguaggio Java

### 3.1 Variabili

```
1  public class Studente {
2
3      // 1. Variabile d'istanza
4      private String nome;
5
6      // 2. Variabile di classe
7      private static int numeroStudenti = 0;
8
9      // 2b. Costante di classe: static + final
10     private static final String SCUOLA = "Universita";
11
12     public Studente(String nome) { // 4. Parametro
13         this.nome = nome;
14         numeroStudenti++;
15     }
16
17     public void saluta(String messaggio) { // 4. Parametro
18         // 3. Variabile locale
19         String salutoCompleto = messaggio + ", sono " + nome;
20
21         System.out.println(salutoCompleto);
22     }
23 }
```

Nel linguaggio Java sono definite quattro tipi di variabili:

1. **Variabili d'istanza (attributi non statici):** attributi unici per ogni istanza della classe, dichiarati senza la keyword `static`;
2. **Variabili di classe (attributi statici):** attributi condivisi da ogni istanza della classe, dichiarati usando la keyword `static`. Sono valori condivisi da ogni istanza della classe, indipendentemente dal numero di istanze create. In aggiunta si può inserire la keyword `final` per evitare che il valore definito possa essere cambiato;
3. **Variabili locali:** variabili dichiarate all'interno di un metodo e visibili solo all'interno di esso, memorizzano informazioni temporanee utilizzate all'interno del metodo stesso. Non ci sono keyword particolari per dichiararle, basta la loro posizione;
4. **Parametri:** variabili passate ad un metodo, dichiarate all'interno delle parentesi tonde. Sono dei valori che il metodo si aspetta di ricevere in input quando viene chiamato. Importante ricordare che in questo caso si parla sempre di "valori" e non di "attributi".

Dato che i nomi delle variabili sono case-sensitive, come nomenclatura standard si usa `camelCase`.

### 3.1.1 Tipi di dati primitivi

| Nome tipo | Valore salvato    | Range valori                                                 | Valore di default |
|-----------|-------------------|--------------------------------------------------------------|-------------------|
| byte      | Intero a 8 bit    | -128 a 127                                                   | 0                 |
| short     | Intero a 16 bit   | -32768 a 32767                                               | 0                 |
| int       | Intero a 32 bit   | $-2^{31}$ a $2^{31} - 1$                                     | 0                 |
| long      | Intero a 64 bit   | $-2^{63}$ a $2^{63} - 1$                                     | 0L                |
| float     | Decimale a 32 bit | circa $\pm 1.4 \times 10^{-45}$ a $\pm 3.4 \times 10^{38}$   | 0.0f              |
| double    | Decimale a 64 bit | circa $\pm 4.9 \times 10^{-324}$ a $\pm 1.8 \times 10^{308}$ | 0.0d              |
| char      | Carattere Unicode | \u0000 a \uFFFF                                              | \u0000            |
| boolean   | Vero/falso        | true o false                                                 | false             |

La classe `java.lang.String` è usata per rappresentare stringhe di caratteri. Una variabile di tipo `String` è immutabile, le uniche modifiche possono avvenire tramite riassegnazione o usando metodi specifici del tipo.

### 3.1.2 Tipi riferimento

| Tipo riferimento | Cosa rappresenta                                                                     | Esempio                                                             |
|------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Classi           | Oggetti creati a partire da una classe. Possono contenere attributi e metodi.        | <code>String</code> , <code>Scanner</code> , <code>Studente</code>  |
| Interfacce       | Tipi che definiscono un comportamento che una classe può implementare.               | <code>List</code> , <code>Runnable</code> , <code>Comparable</code> |
| Array            | Strutture che contengono più valori dello stesso tipo. Anche gli array sono oggetti. | <code>int[]</code> , <code>String[]</code>                          |
| Enum             | Tipi che rappresentano un insieme fisso di costanti.                                 | <code>Giorno.LUNEDI</code>                                          |
| Annotazioni      | Tipi usati per aggiungere metadati al codice.                                        | <code>@Override</code> , <code>@Deprecated</code>                   |

In Java sono tutti i tipi che non salvano direttamente il valore, ma un riferimento a un oggetto in memoria

## 3.2 String

### 3.2.1 Concetto generale

Le stringhe sono sequenze di caratteri molto usate nella programmazione Java. Nel linguaggio Java una stringa non è un tipo primitivo, ma un oggetto della classe **String**, contenuta nel package `java.lang`.

La classe **String** permette di creare, leggere, confrontare e manipolare sequenze di caratteri. Anche se molte operazioni sembrano modificare una stringa esistente, in realtà gli oggetti **String** sono immutabili: una volta creato, il contenuto di un oggetto **String** non può essere cambiato.

Quando si applica un metodo che sembra modificare una stringa, viene creato e restituito un nuovo oggetto **String** contenente il risultato dell'operazione.

```
1 String s = "ciao";
2 String maiuscola = s.toUpperCase();
3
4 System.out.println(s);           // ciao
5 System.out.println(maiuscola);  // CIAO
```

La variabile `s` continua a riferirsi alla stringa originale, mentre `maiuscola` contiene un nuovo oggetto.

### 3.2.2 Creazione di stringhe

Il modo più diretto per creare una stringa è usare un letterale stringa, cioè una sequenza di caratteri racchiusa tra doppi apici.

```
1 String greeting = "Hello world!";
```

Quando il compilatore incontra un letterale stringa, crea un oggetto **String** con quel valore. I letterali stringa sono il modo più comune per creare stringhe.

Come per gli altri oggetti, è possibile creare una stringa anche usando la keyword **new** e un costruttore. Ad esempio, si può creare una stringa a partire da un array di caratteri.

```
1 char[] helloArray = { 'h', 'e', 'l', 'l', 'o', ' ', '.' };
2
3 String helloString = new String(helloArray);
4
5 System.out.println(helloString);
```

L'output sarà *"hello."*

Nella pratica, quando si conosce già il contenuto della stringa, si preferisce usare il letterale stringa. L'uso esplicito di `new String(...)` è meno comune, perché crea direttamente un nuovo oggetto invece di riutilizzare eventualmente una stringa già presente.

### 3.2.3 Lunghezza e accesso ai caratteri

I metodi che permettono di ottenere informazioni da un oggetto sono detti metodi accessori. Nel caso delle stringhe, uno dei metodi più usati è `length()`, che restituisce il numero di caratteri contenuti nella stringa.

```

1 String palindrome = "Dot saw I was Tod";
2
3 int len = palindrome.length();
4
5 System.out.println(len);

```

Gli indici dei caratteri partono da 0. Il primo carattere si trova quindi all'indice 0, mentre l'ultimo si trova all'indice `length() - 1`. Per ottenere il carattere in una determinata posizione si usa il metodo `charAt()`.

```

1 String anotherPalindrome = "Niagara. O roar again!";
2
3 char aChar = anotherPalindrome.charAt(9);
4
5 System.out.println(aChar);

```

In questo esempio il carattere in posizione 9 è 'O'.

Se si vuole ottenere una parte della stringa, si usa il metodo `substring`. La versione con due parametri riceve l'indice iniziale incluso e l'indice finale escluso.

```

1 String text = "Niagara. O roar again!";
2
3 String roar = text.substring(11, 15);
4
5 System.out.println(roar);

```

L'output sarà *"roar"*. Il metodo `substring(11, 15)` restituisce i caratteri dalla posizione 11 fino alla posizione 14. L'indice 15 non è incluso.

### 3.2.4 Conversione tra stringhe e array di caratteri

Una stringa può essere convertita, interamente o parzialmente, in un array di caratteri. Un modo semplice è leggere i caratteri uno alla volta tramite `charAt()`.

```

1 String palindrome = "Dot saw I was Tod";
2 int len = palindrome.length();
3
4 char[] tempCharArray = new char[len];
5
6 for (int i = 0; i < len; i++) {
7     tempCharArray[i] = palindrome.charAt(i);
8 }

```

La classe `String` fornisce anche il metodo `getChars()`, che permette di copiare una porzione della stringa dentro un array di caratteri.

```

1 String palindrome = "Dot saw I was Tod";
2 int len = palindrome.length();
3
4 char[] tempCharArray = new char[len];
5
6 palindrome.getChars(0, len, tempCharArray, 0);

```

I primi due parametri indicano l'intervallo della stringa da copiare. Il terzo parametro è l'array di destinazione, mentre il quarto indica da quale posizione dell'array iniziare a scrivere.

### 3.2.5 Concatenazione

La concatenazione consiste nell'unire due o più stringhe. La classe `String` fornisce il metodo `concat()`, che restituisce una nuova stringa ottenuta aggiungendo una stringa alla fine di un'altra.

```
1 String s1 = "Hello, ";
2 String s2 = "world!";
3
4 String result = s1.concat(s2);
5
6 System.out.println(result);
```

L'output sarà *"Hello, world!"*. Nella pratica, la concatenazione viene fatta più spesso con l'operatore `+`.

```
1 String result = "Hello, " + "world" + "!";
2
3 System.out.println(result);
```

L'operatore `+` può concatenare stringhe anche con valori di altri tipi. Se un elemento non è una stringa, Java lo converte automaticamente chiamando il metodo `toString()`.

```
1 int eta = 20;
2
3 String frase = "Eta: " + eta;
4
5 System.out.println(frase);
```

L'output sarà *"Eta: 20"*. Quando una stringa è molto lunga, può essere divisa su più righe nel codice sorgente usando l'operatore `+`.

```
1 String quote =
2     "Now is the time for all good " +
3     "men to come to the aid of their country.";
```

### 3.2.6 Stringhe formattate

La classe `String` fornisce il metodo statico `format()`, che permette di costruire una stringa formattata. A differenza di `System.out.printf()`, che stampa direttamente il risultato, `String.format()` restituisce un oggetto `String` riutilizzabile.

```
1 float floatVar = 3.14f;
2 int intVar = 10;
3 String stringVar = "test";
4
5 String fs = String.format(
6     "Float: %f, intero: %d, stringa: %s",
7     floatVar,
8     intVar,
9     stringVar
10 );
11
12 System.out.println(fs);
```

Questo è utile quando il risultato formattato deve essere salvato, passato a un metodo o stampato

in un secondo momento.

### 3.2.7 Conversione da stringhe a numeri

Spesso un programma riceve dati numerici sotto forma di stringhe, ad esempio da input utente, file o argomenti da riga di comando. Per usare questi valori in operazioni aritmetiche, è necessario convertirli in numeri.

Le classi wrapper dei tipi primitivi numerici, come `Integer`, `Float`, `Double`, `Long` e `Short`, forniscono metodi per convertire stringhe in numeri.

```
1 String s1 = "4.5";
2 String s2 = "87.2";
3
4 float a = Float.parseFloat(s1);
5 float b = Float.parseFloat(s2);
6
7 System.out.println("a + b = " + (a + b));
8 System.out.println("a - b = " + (a - b));
9 System.out.println("a * b = " + (a * b));
10 System.out.println("a / b = " + (a / b));
```

Il metodo `parseFloat()` restituisce direttamente un valore primitivo di tipo `float`. In modo analogo esistono metodi come `Integer.parseInt()`, `Double.parseDouble()` e `Long.parseLong()`.

È possibile usare anche `valueOf()`, che restituisce invece un oggetto wrapper.

```
1 Integer x = Integer.valueOf("42");
2 Double y = Double.valueOf("3.14");
```

Se la stringa non contiene un valore numerico valido, viene lanciata una `NumberFormatException`.

### 3.2.8 Conversione da numeri a stringhe

Un numero può essere convertito in stringa in diversi modi. Il più semplice è concatenarlo con una stringa vuota.

```
1 int i = 10;
2
3 String s1 = "" + i;
```

Un modo più esplicito è usare `String.valueOf()`.

```
1 int i = 10;
2
3 String s2 = String.valueOf(i);
```

Le classi wrapper forniscono anche metodi `toString()` statici.

```
1 int i = 10;
2 double d = 858.48;
3
4 String s3 = Integer.toString(i);
5 String s4 = Double.toString(d);
```

Una volta convertito un numero in stringa, si possono usare i metodi di `String` per analizzarne la rappresentazione testuale.

```

1 double d = 858.48;
2 String s = Double.toString(d);
3
4 int dot = s.indexOf('.');
5
6 System.out.println(dot + " digits before decimal point.");
7 System.out.println((s.length() - dot - 1)
8     + " digits after decimal point.");

```

### 3.2.9 Ricerca e manipolazione

La classe `String` offre vari metodi per cercare caratteri o sottostringhe. I più importanti sono `indexOf()`, `lastIndexOf()` e `contains()`.

```

1 String text = "Niagara. O roar again!";
2
3 int first = text.indexOf('a');
4 int last = text.lastIndexOf('a');
5
6 boolean containsRoar = text.contains("roar");
7
8 System.out.println(first);
9 System.out.println(last);
10 System.out.println(containsRoar);

```

Il metodo `indexOf()` cerca dall'inizio della stringa, mentre `lastIndexOf()` cerca dalla fine. Se il carattere o la sottostringa non vengono trovati, questi metodi restituiscono `-1`.

Il metodo `contains()` restituisce invece un valore booleano e va usato quando interessa solo sapere se una certa sequenza di caratteri è presente.

Altri metodi comuni per manipolare stringhe sono `trim()`, `toLowerCase()`, `toUpperCase()`, `split()` e `replace()`.

```

1 String text = "  Java Programming  ";
2
3 String trimmed = text.trim();
4 String lower = text.toLowerCase();
5 String upper = text.toUpperCase();
6
7 System.out.println(trimmed);
8 System.out.println(lower);
9 System.out.println(upper);

```

Il metodo `replace()` restituisce una nuova stringa in cui alcune occorrenze sono state sostituite.

```

1 String text = "Java e' difficile";
2
3 String result = text.replace("difficile", "potente");
4
5 System.out.println(result);

```

L'output sarà *"Java e' potente"*. Le versioni `replaceAll()` e `replaceFirst()` usano espressioni regolari: la prima sostituisce tutte le occorrenze corrispondenti, la seconda solo la prima.

### 3.2.10 Confronto tra stringhe

Le stringhe non vanno confrontate con l'operatore `==` quando si vuole verificare l'uguaglianza del contenuto. L'operatore `==` confronta i riferimenti, cioè verifica se due variabili puntano allo stesso oggetto. Per confrontare il contenuto si usa `equals()`.

```
1 String a = new String("Java");
2 String b = new String("Java");
3
4 System.out.println(a == b);           // false
5 System.out.println(a.equals(b));      // true
```

Il metodo `equalsIgnoreCase()` confronta due stringhe ignorando la differenza tra maiuscole e minuscole.

```
1 String a = "Java";
2 String b = "java";
3
4 System.out.println(a.equalsIgnoreCase(b));
```

Altri metodi utili per il confronto sono `startsWith()`, `endsWith()`, `compareTo()` e `compareToIgnoreCase()`.

```
1 String text = "Java Programming";
2
3 System.out.println(text.startsWith("Java"));
4 System.out.println(text.endsWith("ing"));
5 System.out.println(text.compareTo("Kotlin"));
```

Il metodo `compareTo()` confronta due stringhe lessicograficamente e restituisce un intero. Il risultato è 0 se le stringhe sono uguali, minore di 0 se la prima viene prima della seconda, maggiore di 0 se viene dopo.

Il metodo `regionMatches()` permette invece di confrontare solo una porzione di una stringa con una porzione di un'altra.

```
1 String searchMe = "Green Eggs and Ham";
2 String findMe = "Eggs";
3
4 int searchMeLength = searchMe.length();
5 int findMeLength = findMe.length();
6
7 boolean foundIt = false;
8
9 for (int i = 0; i <= searchMeLength - findMeLength; i++) {
10     if (searchMe.regionMatches(i, findMe, 0, findMeLength)) {
11         foundIt = true;
12         System.out.println(searchMe.substring(i,
13             i + findMeLength));
14         break;
15     }
16 }
17
18 if (!foundIt) {
19     System.out.println("No match found.");
20 }
```

### 3.2.11 StringBuilder

La classe `StringBuilder` rappresenta una sequenza modificabile di caratteri. A differenza di `String`, un oggetto `StringBuilder` può essere modificato senza creare ogni volta un nuovo oggetto.

Questo è utile quando bisogna effettuare molte concatenazioni o modifiche successive su una sequenza di caratteri. In questi casi, usare `StringBuilder` può essere più efficiente rispetto a concatenare molte stringhe.

```
1  StringBuilder sb = new StringBuilder();
2
3  sb.append("Hello");
4  sb.append(", ");
5  sb.append("world!");
6
7  System.out.println(sb.toString());
```

La classe `StringBuilder` ha un concetto di lunghezza e capacità. La lunghezza, ottenuta con `length()`, indica il numero di caratteri effettivamente contenuti. La capacità, ottenuta con `capacity()`, indica lo spazio allocato internamente.

```
1  StringBuilder sb = new StringBuilder();
2
3  sb.append("Greetings");
4
5  System.out.println(sb.length());
6  System.out.println(sb.capacity());
```

La capacità è sempre maggiore o uguale alla lunghezza e viene aumentata automaticamente quando necessario.

I metodi principali di `StringBuilder` sono `append()`, `insert()`, `delete()`, `replace()`, `setCharAt()`, `reverse()` e `toString()`.

```
1  StringBuilder sb = new StringBuilder("Java");
2
3  sb.insert(4, " language");
4  sb.replace(0, 4, "The Java");
5  sb.delete(8, 17);
6  sb.reverse();
7
8  System.out.println(sb);
```

Un esempio classico è l'inversione di una stringa. Con `String` sarebbe necessario lavorare manualmente con array di caratteri, mentre con `StringBuilder` basta usare `reverse()`.

```

1 public class StringBuilderDemo {
2     public static void main(String[] args) {
3         String palindrome = "Dot saw I was Tod";
4
5         StringBuilder sb = new StringBuilder(palindrome);
6
7         sb.reverse();
8
9         System.out.println(sb);
10    }
11 }

```

L'output sarà *"doT saw I was toD"*.

Quando un oggetto `StringBuilder` viene passato a `System.out.println()`, il metodo `toString()` viene chiamato implicitamente, come avviene per gli altri oggetti.

Esiste anche la classe `StringBuffer`, molto simile a `StringBuilder`. La differenza principale è che `StringBuffer` è thread-safe perché i suoi metodi sono sincronizzati. In assenza di esigenze legate alla concorrenza, si preferisce normalmente `StringBuilder`.

### 3.3 Array

```
1 int[] a = new int[3];           // dimensione fissa, valori default
2
3 int[] b = {1, 2, 3};           // inizializzazione diretta
4
5 int[] c = new int[] {1, 2, 3};  // new + valori
6
7 int[] d;
8 d = new int[] {1, 2, 3};       // assegnazione successiva
9
10 int[][] e = {{1, 2}, {3, 4}};  // array bidimensionale
```

Invece di implementare uno a uno metodi utili per lavorare sugli array, come `copy`, `sort` o `find`, si può utilizzare il pacchetto `java.util.Arrays`.

### 3.4 Enum

Un `enum` si usa quando una variabile può assumere solo un insieme limitato e predefinito di valori. Utile per stabilire stati e variabili ben definiti, evitando valori non validi.

```
1 enum Giorno {
2     LUNEDI, MARTEDI, MERCOLEDI, GIOVEDI, VENERDI, SABATO, DOMENICA
3 }
```

Un `enum` può avere anche attributi e metodi:

```
1 enum StatoOrdine {
2     IN_ATTESA("Ordine in attesa"),
3     SPEDITO("Ordine spedito"),
4     CONSEGNATO("Ordine consegnato");
5
6     private String descrizione;
7
8     StatoOrdine(String descrizione) {
9         this.descrizione = descrizione;
10    }
11
12    public String getDescrizione() {
13        return descrizione;
14    }
15 }
```

### 3.5 Annotation

Le **annotazioni** servono per aggiungere informazioni al codice, senza modificare direttamente la logica del programma. Possono essere intese come metadati aggiuntivi, utili per il compilatore, per strumenti esterni o per framework.

Per leggerle bisogna usare un concetto detto "reflection": permette di ispezionare e manipolare classi, metodi, campi, costruttori e annotazioni durante l'esecuzione del programma.

```
1 import java.lang.annotation.Retention;
2 import java.lang.annotation.RetentionPolicy;
3
4 @Retention(RetentionPolicy.RUNTIME)
5 @interface Author {
6     String nome();
7 }
8
9 @Author(nome = "Alessandro")
10 class AppuntiJava {
11 }
12
13 public class Main {
14     public static void main(String[] args) {
15         Class<AppuntiJava> classe = AppuntiJava.class;
16         Author author = classe.getAnnotation(Author.class);
17         System.out.println(author.nome());
18     }
19 }
```

### 3.6 Operatori

Gli operatori di Java operano in modo analogo agli operatori di altri linguaggi di programmazione. Confronti, operazioni numeriche, bitwise e logiche sono assolutamente identiche ad ogni altro linguaggio.

Degno di nota è l'operatore **instanceof**, usato per verificare se un oggetto è istanza di una certa classe, sottoclasse, oppure implementa una certa interfaccia.

## 4 Classi e Oggetti

### 4.1 Dichiarazione di attributi

Per dichiarare gli attributi di una classe (generalmente si fa come prima cosa dopo aver dichiarato la classe stessa all'inizio del file) c'è bisogno di specificare tre componenti: uno o più modificatori (es. `public` o `private`), il tipo dell'attributo e il nome dell'attributo.

### 4.2 Modificatori d'accesso

Esistono quattro tipi di modificatori d'accesso:

1. **public:** livello più aperto di tutti, un attributo `public` può essere usato da qualunque altra classe, anche in package diversi;
2. **protected:** l'attributo è visibile alle classi nello stesso package e alle sottoclassi (anche in package diversi). Si usa quando si vuole permettere l'accesso alle classi figlie senza rendere l'attributo totalmente pubblico;
3. **private:** livello più restrittivo, l'attributo può essere usato solo dentro la stessa classe. Di solito è il modificatore più corretto per gli attributi, necessitando però l'implementazione di metodi pubblici, detti `getter` e `setter`, per ottenere e impostare il valore dell'attributo;
4. **package-private:** se non si scrive nulla prima del tipo dell'attributo, allora l'attributo sarà visibile solo dalle classi dello stesso package.

### 4.3 Metodi e Overloading

Per la dichiarazione del metodo di una classe bastano sei componenti:

1. **Modificatore d'accesso:** detti in precedenza;
2. **Tipo di ritorno:** tipo del valore restituito dal metodo;
3. **Nome:** il nome del metodo. Dovrebbe seguire la convenzione `camelCase` ed essere riguardante la logica che esegue;
4. **Lista dei parametri:** una lista dei valori di input della funzione, inclusi il loro tipo;
5. **Lista di eccezioni:** le eccezioni che potrebbero essere lanciate (in inglese si dice `throw exception`) dal metodo;
6. **Corpo:** il testo vero e proprio del metodo.

Java supporta l'overloading di metodi, ovvero la scrittura di metodi aventi lo stesso nome ma firma diversa. Ciò significa che all'interno della stessa classe si possono avere metodi con nomi uguali ma parametri differenti. Questa feature non andrebbe abusata dato che rende il codice meno leggibile e prevedibile.

Da notare che un metodo avente una classe come tipo di ritorno può restituire istanze della classe stessa, istanze di sottoclassi di essa o istanze di classi che implementano l'interfaccia della classe.

## 4.4 Costruttori

Una classe può (di solito sarebbe meglio) contenere uno o più costruttori, usati per inizializzare un'istanza della classe.

La dichiarazione è simile a quella di un metodo, eccetto che non si ha un valore di ritorno e si usa lo stesso nome della classe.

```
1 // Esempio di costruttore
2 public Bicycle(int startCadence, int startSpeed, int startGear) {
3     gear = startGear;
4     cadence = startCadence;
5     speed = startSpeed;
6 }
7
8 // Creazione di un'istanza
9 Bicycle myBike = new Bicycle(30, 0, 8);
```

Bisogna stare attenti a non fornire un costruttore ad una classe: di default, se non esplicitamente dichiarato viene fornito un costruttore senza parametri in input; questo costruttore chiamerà il costruttore della superclasse, che di default sarà la classe **Object** se non diversamente specificato. **Object** ha un suo costruttore senza argomenti, cosa che potrebbe causare comportamenti imprevisti.

Da una sottoclasse è possibile invocare un costruttore della superclasse da cui si deriva, sia per risparmiare codice che per mantenere una logica rigorosa per quanto riguarda la creazione di un'istanza.

## 4.5 Passaggio di parametri ad un metodo o ad un costruttore

La dichiarazione di un metodo o di un costruttore specifica in modo preciso il numero e il tipo di parametri che dovranno essere forniti per garantirne il corretto funzionamento.

Si può passare anche un numero arbitrario di parametri come ultimo parametro del metodo usando la seguente scrittura:

```
1 public Polygon polygonFrom(Point... corners) {
2     int numberOfSides = corners.length;
3     double squareOfSide1, lengthOfSide1;
4     squareOfSide1 = (corners[1].x - corners[0].x)
5                     * (corners[1].x - corners[0].x)
6                     + (corners[1].y - corners[0].y)
7                     * (corners[1].y - corners[0].y);
8     lengthOfSide1 = Math.sqrt(squareOfSide1);
9 }
```

In questo caso **corners** viene trattato come un array, potendo chiamare il metodo passando da zero a  $n$  parametri che verranno messi al suo interno.

#### **4.5.1 Passaggio per valore**

I tipi primitivi vengono passati ai metodi per valore; ogni modifica al parametro avverrà solo nello scope del metodo stesso e ogni cambiamento verrà perso alla fine dell'esecuzione.

#### **4.5.2 Passaggio per riferimento**

Anche i tipi riferimento, come ad esempio gli oggetti, sono passati per valore. Questo significa che il riferimento all'oggetto continuerà a fare riferimento allo stesso oggetto, però i valori dei suoi attributi potrebbero essere cambiati se il metodo aveva permessi sufficienti per accedervi.

## 4.6 Membri statici

La keyword **static** permette di definire campi e metodi che appartengono alla **classe** invece che a una singola istanza.

### 4.6.1 Variabili statiche

Una **variabile statica**, detta anche **class variable**, è condivisa da tutte le istanze della classe.

Le variabili di istanza, invece, sono proprie di ogni singolo oggetto.

```
1 public class Bicycle {
2
3     private int id;
4     private static int numberOfBicycles = 0;
5
6     public Bicycle() {
7         id = ++numberOfBicycles;
8     }
9
10    public int getID() {
11        return id;
12    }
13 }
```

In questo esempio:

- `id` è una variabile di istanza: ogni oggetto ha il proprio valore;
- `numberOfBicycles` è una variabile statica: esiste una sola copia condivisa da tutti gli oggetti.

Una variabile statica dovrebbe essere riferita tramite il nome della classe:

```
1 Bicycle.numberOfBicycles
```

e non tramite un oggetto, perché appartiene alla classe.

### 4.6.2 Metodi statici

Un **metodo statico** appartiene alla classe e può essere invocato senza creare un'istanza.

```
1 ClassName.methodName(args);
```

Ad esempio:

```
1 public static int getNumberOfBicycles() {
2     return numberOfBicycles;
3 }
```

Il metodo può essere invocato così:

```
1 Bicycle.getNumberOfBicycles();
```

### 4.6.3 Accesso tra membri statici e membri di istanza

Non tutte le combinazioni tra membri statici e membri di istanza sono consentite.

| Contesto          | Accesso consentito                                                                                                                  |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Metodo di istanza | Può accedere direttamente sia a variabili/metodi di istanza sia a variabili/metodi statici.                                         |
| Metodo statico    | Può accedere direttamente solo a variabili/metodi statici. Per accedere a membri di istanza deve usare un riferimento a un oggetto. |

Un metodo statico non può usare direttamente variabili di istanza, metodi di istanza o la keyword `this`. Questo perché un metodo statico non è associato a nessun oggetto specifico.

### 4.6.4 Costanti

La combinazione di `static` e `final` viene utilizzata per definire costanti.

```
1 public static final double PI = 3.141592653589793;
```

`static` indica che la costante appartiene alla classe, mentre `final` indica che il valore non può essere modificato.

Per convenzione, i nomi delle costanti sono scritti in maiuscolo. Se il nome è composto da più parole, si usa l'underscore:

```
1 public static final int MAX_SPEED = 100;
```

## 4.7 Inizializzazione dei campi

I campi possono essere inizializzati direttamente nella loro dichiarazione:

```
1 public class Example {  
2     public static int capacity = 10;  
3     private boolean full = false;  
4  
5 }
```

Questa soluzione è adatta quando il valore iniziale è semplice. Se invece l'inizializzazione richiede logica più complessa (cicli, condizioni, gestione di errori, ecc.), si possono utilizzare blocchi di inizializzazione o costruttori.

### 4.7.1 Blocchi di inizializzazione statici

Un **blocco di inizializzazione statico** viene eseguito una sola volta, quando la classe viene caricata dalla JVM.

La sintassi è:

```
1 static {  
2     // codice di inizializzazione  
3 }
```

Una classe può contenere più blocchi statici, che vengono eseguiti nell'ordine in cui compaiono nel codice sorgente.

In alternativa è possibile utilizzare un metodo statico privato:

```
1 private static int initialize() {  
2     ...  
3 }  
4  
5 private static int value = initialize();
```

Questa soluzione permette di riutilizzare il codice di inizializzazione.

### 4.7.2 Inizializzazione delle istanze

Le variabili di istanza vengono normalmente inizializzate all'interno del costruttore.

In alternativa è possibile utilizzare un **blocco di inizializzazione** (senza la keyword `static`):

```
1 {  
2     // codice di inizializzazione  
3 }
```

Il compilatore copia automaticamente questo blocco all'inizio di ogni costruttore della classe. Ciò permette di condividere lo stesso codice di inizializzazione tra costruttori diversi.

### 4.7.3 Metodi **final** per l'inizializzazione

Un'altra possibilità consiste nell'inizializzare una variabile mediante un metodo:

```
1 private int value = initialize();  
2  
3 protected final int initialize() {  
4     ...  
5 }
```

Il metodo viene dichiarato **final** per impedire che venga ridefinito dalle sottoclassi, evitando problemi durante la fase di inizializzazione dell'oggetto.

## 5 Oggetti

### 5.1 Creazione di oggetti

La creazione di un oggetto in Java a partire da una classe è composto da tre punti fondamentali: dichiarazione, istanziiazione e inizializzazione.

#### 5.1.1 Dichiarazione

Quando si crea un nuovo oggetto occorre dichiarare anche una variabile che farà riferimento ad esso; per esempio posso dichiarare una variabile `Bicycle b` e non assegnarle nessun oggetto. Fare ciò non crea un oggetto, il suo valore non sarà definito finchè non verrà creato un oggetto e verrà assegnato ad essa.

#### 5.1.2 Istanziiazione

L'operatore `new` istanzia una classe allocando memoria per il nuovo oggetto e restituendo un riferimento a quella memoria. L'operatore invoca il costruttore dell'oggetto, specificato come unico argomento preso in input.

#### 5.1.3 Inizializzazione

Prendiamo questo codice di esempio:

```
1 public class Point {
2     public int x = 0;
3     public int y = 0;
4     //constructor
5     public Point(int a, int b) {
6         x = a;
7         y = b;
8     }
9 }
```

La classe `Point` contiene un solo costruttore che prende in input due variabili. Per creare un oggetto `Point` si può scrivere `Point originOne = new Point(23, 94)`.

### 5.2 Oggetti temporanei

Una scrittura in java valida è la seguente: `int height = new Rectangle().height`. Ciò indica che si vuole creare un nuovo oggetto `Rectangle` e se ne vuole prendere l'altezza. Dopo questa chiamata si perde il riferimento all'oggetto creato, poichè il programma non salva un suo riferimento da nessuna parte. Non essendo referenziato in nessun punto, viene liberata la sua memoria.

### 5.3 Garbage Collector

Il Java runtime environment elimina automaticamente gli oggetti quando determina che essi non sono più usati. Questo processo si dice "garbage collection".

Un oggetto può essere eliminato quando non vi sono più riferimenti ad esso durante l'esecuzione del programma. I riferimenti contenuti nelle variabili locali vengono eliminati quando la variabile esce dal proprio scope, cioè dal proprio ambito di visibilità. Se dopo quel momento non esistono altri riferimenti allo stesso oggetto, l'oggetto diventa candidabile per il garbage collector.

Un modo più esplicito per eliminare un riferimento è impostare il valore della variabile a `null`. In ogni caso è necessario che TUTTI i riferimenti a quell'oggetto vengano eliminati prima di procedere all'eliminazione da parte del garbage collector.

## 5.4 Keyword `this`

Il motivo più comune per usare `this` è quando un metodo prende in input un parametro con lo stesso nome di un attributo della classe.

```
1 public class Point {
2     public int x = 0;
3     public int y = 0;
4
5     // Constructor
6     public Point(int x, int y) {
7         this.x = x;
8         this.y = y;
9     }
10 }
```

All'interno di un costruttore si può chiamare anche un altro costruttore della stessa classe. Questo procedimento si dice "explicit constructor invocation".

```
1 public class Rectangle {
2     private int x, y;
3     private int width, height;
4
5     public Rectangle() {
6         this(0, 0, 1, 1);
7     }
8     public Rectangle(int width, int height) {
9         this(0, 0, width, height);
10    }
11    public Rectangle(int x, int y, int width, int height) {
12        this.x = x;
13        this.y = y;
14        this.width = width;
15        this.height = height;
16    }
17    ...
18 }
```

Sia il costruttore senza parametri che quello con due chiamano entrambi il costruttore con quattro parametri, passando i valori opportunamente. Si occuperà poi il compiler di determinare quale costruttore chiamare in base al numero e al tipo di parametri.

Se presente, la chiamata di un altro costruttore deve essere la prima riga del costruttore chiamante.

## 6 Classi annidate

Java permette di definire classi all'interno di altre. Una classe può essere dichiarata **static**, venendo definite "classi annidate statiche" (static nested classes), e classi non statiche, dette "classi interne" (inner classes).

Le classi interne hanno accesso agli attributi della classe in cui sono contenuti, anche se sono impostati come **private**. Le classi annidate statiche invece non hanno questa possibilità.

Le classi annidate possono essere dichiarate come **private**, **public**, **protected** o **package private**, mentre le classi "principali" possono essere solo **public** o **package private**.

### 6.1 Perchè usare le classi annidate

L'uso delle classi annidate porta diversi vantaggi:

- **Raggruppare classi:** se una classe è utile soltanto ad un'altra ha senso unirle in una sola, annidandole;
- **Aumenta l'incapsulamento:** se si hanno due classi, A e B, e B ha bisogno di accedere a degli attributi di A che sono dichiarati come **private**, nascondendo B dentro A possiamo mantenere gli attributi di A privati permettendo a B di accedervi in ogni caso;
- **Leggibilità e manutenibilità:** annidare classi piccole dentro delle classi top-level pone il codice più vicino a dove deve essere utilizzato.

### 6.2 Inner classes

Come per i metodi e le variabili d'istanza, una classe interna è associata ad un'istanza della classe che la contiene. Inoltre, dato che è sempre associata ad un'istanza, non può definire attributi o metodi statici al suo interno.

```
1 class OuterClass {  
2     ...  
3     class InnerClass {  
4         ...  
5     }  
6 }
```

Per istanziare una classe interna occorre prima istanziare la classe che lo contiene, per poi istanziare l'oggetto interno all'interno dell'oggetto esterno (brutto da leggere, lo so).

```
1 OuterClass outerObject = new OuterClass();  
2 OuterClass.InnerClass innerObject = outerObject.new InnerClass();
```

Esistono due tipi particolari di classi interne, classi locali (local classes) e classi anonime (anonymous classes).

### 6.3 Local classes

Una classe locale è una classe dichiarata all'interno di un "block", ovvero un gruppo di righe di codice. In parole povere, dentro il corpo di qualcosa.

In generale si dichiarano all'interno di un metodo: potrei ad esempio dichiarare una classe di utilità prima di effettuare un ciclo **for** che calcola qualcosa, e avere accesso a dei metodi aggiuntivi potrebbe risultare comodo.

Una classe locale ha accesso ai membri della classe che la contiene. Inoltre può accedere alle variabili locali dichiarati nel blocco in cui è contenuto, ma solo se queste sono state dichiarate come **final**. Se accede una variabile locale o un parametro del metodo si dice che "cattura" quel valore.

Da Java SE 8 una classe locale può accedere alle variabili e parametri del blocco se sono **final** o **effectively final**, ovvero se dopo che sono stati dichiarati non cambiano mai valore. Ci pensa il compilatore a determinare se ciò accade.

## 6.4 Anonymous classes

Una classe anonima in Java è una classe senza nome, dichiarata e istanziata nello stesso punto del codice.

Si usa quando si vuole creare rapidamente una piccola implementazione di una classe astratta o di un'interfaccia, senza dichiarare una classe separata.

```
1 Runnable r = new Runnable() {  
2     @Override  
3     public void run() {  
4         System.out.println("Esecuzione...");  
5     }  
6 };  
7  
8 r.run();
```

Come le classi locali, le classi anonime possono "catturare" delle variabili. Hanno lo stesso accesso alle variabili locali dello scope che li contiene:

- Ha accesso ai membri della classe che le contiene;
- Non possono accedere alle variabili locali dello scope se non sono dichiarate come **final** o **effectively final**;
- Come per le classi annidate, la dichiarazione di un tipo (come per una variabile) in una classe anonima fa "shadowing", ovvero nasconde, una dichiarazione precedente.

Una classe anonima in Java non può dichiarare membri statici ordinari, perché non ha un nome proprio e non è pensata come tipo autonomo. Tuttavia può dichiarare membri statici se sono constant variables, cioè costanti note a compile-time: campi **static final** di tipo primitivo o **String** inizializzati con un'espressione costante.

Spesso le classi anonime sono usate per la creazione delle GUI.

## 6.5 Lambda Expressions

Se una classe anonima è particolarmente semplice può risultare molto verbosa per compiere un'azione sola. Sono state perciò introdotte le lambda.

```
1 parametro -> espressione  
2  
3 (a, b) -> {return a + b;}
```

Se c'è un solo parametro, si possono omettere le parentesi. Se il corpo è una singola espressione, si può omettere **{}** e **return**.

Il tipo della lambda dipende dal contesto, per esempio nel seguente codice:

```
1 Predicate<Person> p = person -> person.getAge() >= 18;
```

la lambda "diventa" un `Predicate<Person>` perchè è quello il tipo atteso per la variabile `p`.

### 6.5.1 Method references

Sono una forma più compatta delle lambda quando la lambda fa solo una sola cosa: chiamare un metodo già esistente:

```
1 // Lambda expression
2 Arrays.sort(rosterAsArray, (a, b) -> Person.compareByAge(a, b));
3
4 // Method reference
5 Arrays.sort(rosterAsArray, Person::compareByAge);
```

Esistono quattro "versioni" principali dei method reference:

```
1 ContainingClass::staticMethodName | Es: Person::compareByAge
2
3 containingObject::instanceMethodName | Es: myComparisonProvider::compareByName
4
5 ContainingType::methodName | Es: String::compareToIgnoreCase
6
7 ClassName::new | Es: HashSet::new
```

## 7 Annotations

Le annotations sono una forma di metadati non facente parte del programma in sè. Non hanno un effetto diretto sull'esecuzione del codice che annotano.

Gli usi delle annotation possono essere:

- **Informazioni per il compilatore:** possono essere utilizzate dal compilatore per rilevare errori o per sopprimere determinati avvisi;
- **Elaborazione in fase di compilazione e di distribuzione:** strumenti software possono elaborare le annotazioni per generare automaticamente codice, file XML e altri artefatti;
- **Elaborazione a runtime:** alcune annotazioni sono disponibili durante l'esecuzione del programma e possono essere lette e utilizzate dal codice tramite la reflection.

### 7.1 Concetti fondamentali

Nella sua forma più semplice, una annotation è scritta come `@Entity`; le annotation possono includere elementi, con o senza nome, aggiungendo dei valori ad essi.

```
1 // Con nome
2 @Author(
3     name = "Benjamin Franklin",
4     date = "3/27/2003"
5 )
6 class MyClass { ... }
7
8 // Con un solo elemento si omette il nome
9 @SuppressWarnings("unchecked")
10 void myMethod() { ... }
```

Le annotazioni possono essere applicate alle dichiarazioni: dichiarazioni di classi, attributi, metodi ed altri elementi del programma. Di solito si inseriscono le annotazioni su righe a parte.

### 7.2 Dichiarazione di un tipo di annotazione

Molte annotazioni sostituiscono commenti informativi riguardanti il codice: è pratica comune inserire dei commenti all'inizio di una classe contenenti informazioni importanti; per fare la stessa cosa con le annotation serve definire il tipo dell'annotazione (annotation type). Un esempio di sintassi è la seguente:

```
1 @interface ClassPreamble {
2     String author();
3     String date();
4     int currentRevision() default 1;
5     String lastModified() default "N/A";
6     String lastModifiedBy() default "N/A";
7     // Note use of array
8     String[] reviewers();
9 }
```

La dichiarazione è simile a quella di un'interfaccia, usando `@` per indicare che stiamo dichiarando un tipo di annotazione. Dopo aver dichiarato il tipo, l'annotazione può essere usata ad esempio come segue:

```
1 @ClassPreamble (
2     author = "John Doe",
3     date = "3/17/2002",
4     currentRevision = 6,
5     lastModified = "4/12/2004",
6     lastModifiedBy = "Jane Doe",
7     // Note array notation
8     reviewers = {"Alice", "Bob", "Cindy"}
9 )
10 public class Generation3List extends Generation2List {
11     ...
12 }
```

Inserendo `@Documented` prima della dichiarazione di tipo si può far comparire l'annotazione all'interno della documentazione generata da Javadoc.

### 7.3 Tipi di annotazione predefiniti

Esistono molti tipi di annotazione predefinita. I più importanti sono:

- **@Deprecated:** un elemento non dovrebbe più essere usato. Il compilatore genera un warning se viene usato;
- **@Override:** un metodo sta sovrascrivendo un metodo della superclasse o di un'interfaccia. Se non lo fa davvero, il compilatore dà errore;
- **@SuppressWarnings:** chiede al compilatore di non mostrare certi warning, per esempio `unchecked` o `deprecation`;
- **@SafeVarargs:** un metodo o costruttore con `varargs` non compie operazioni potenzialmente insicure. Sopprime warning legati ai `varargs`;
- **@FunctionalInterface:** un'interfaccia deve essere funzionale, cioè avere un solo metodo astratto. Se non rispetta la regola, il compilatore dà errore.

### 7.4 Annotazioni che si applicano ad altre annotazioni

Alcune particolari annotazioni possono esser applicate ad altre annotazioni, aggiungendo informazioni utili. Queste vengono dette **meta-annotazioni**:

- **@Retention:** stabilisce fino a quando un'annotazione viene conservata: solo nel sorgente, nel `.class`, oppure anche a runtime;
- **@Documented:** fa sì che l'annotazione venga inclusa nella documentazione Javadoc;
- **@Target:** specifica dove un'annotazione può essere usata: classe, metodo, campo, parametro, costruttore, ecc;
- **@Inherited:** permette a una classe di ereditare un'annotazione dalla superclasse. Vale solo per annotazioni applicate a classi;
- **@Repeatable:** permette di usare la stessa annotazione più volte sullo stesso elemento.

## 7.5 Annotazioni di tipo e sistemi di tipi estendibili

Con Java SE 8 le annotazioni possono essere applicate non solo alle dichiarazioni (classi, metodi, campi, ecc.), ma anche a qualsiasi utilizzo di un tipo (type use), ad esempio:

```
1 // Creazione di un'istanza
2 new @Interned MyObject();
3
4 // Casting di tipo
5 myString = (@NonNull String) str;
6
7 // Clausola "implements"
8 class UnmodifiableList<T> implements
9     @ReadOnly List<@ReadOnly T> { ... }
10
11 // Dichiarazione di "Thrown exception"
12 void monitorTemperature() throws
13     @Critical TemperatureException { ... }
```

Le type annotations servono a rafforzare il controllo dei tipi tramite pluggable type systems, cioè moduli esterni che estendono il compilatore con verifiche aggiuntive. Ad esempio, un'annotazione personalizzata come `@NonNull String str`, può essere verificata da un modulo che segnala durante la compilazione eventuali assegnazioni di `null`, riducendo il rischio di `NullPointerException`.

Esistono framework già pronti, come il Checker Framework, che forniscono annotazioni e controlli aggiuntivi (ad esempio `@NonNull`) senza dover implementare un sistema di verifica personalizzato.

## 7.6 Annotazioni ripetibili

In alcune situazioni si potrebbe voler utilizzare la stessa annotazione più volte ad una dichiarazione o ad un tipo. Con Java SE 8, le **repeating annotations** permettono ciò.

Per motivi di compatibilità, è necessario dichiarare due componenti nel codice: l'interfaccia dell'annotazione che si ripete ed un "contenitore" per le annotation dichiarate.

```
1 import java.lang.annotation.Repeatable;
2
3 // Repeatable Annotation Type
4 @Repeatable(Schedules.class)
5 public @interface Schedule {
6     String dayOfMonth() default "first";
7     String dayOfWeek() default "Mon";
8     int hour() default 12;
9 }
10
11 // Containing Annotation Type
12 public @interface Schedules {
13     Schedule[] value();
14 }
```

## 7.7 Recupero delle annotazioni

Per riottenere le annotation si usa la Reflection API, che mette a disposizione diversi metodi per recuperare le annotazioni.

I metodi che restituiscono una sola annotazione, come `AnnotatedElement.getAnnotation(Class<T>)`,

continuano a comportarsi come in passato: restituiscono una sola annotazione del tipo richiesto, se presente.

Se invece sono presenti più annotazioni dello stesso tipo (annotazioni ripetibili), è possibile ottenerle recuperando prima la loro container annotation. In questo modo il codice scritto prima di Java 8 continua a funzionare senza modifiche.

Java SE 8 introduce anche nuovi metodi, come `AnnotatedElement.getAnnotationsByType(Class<T>)`, che recuperano automaticamente tutte le annotazioni dello stesso tipo, senza dover gestire manualmente la container annotation.

## 7.8 Scelte progettuali

Quando si progetta un nuovo tipo di annotazione, è necessario considerare quante volte essa potrà essere applicata. È inoltre possibile limitare dove l'annotazione può essere applicata utilizzando la meta-annotazione `@Target`. Ad esempio, si può creare un'annotazione ripetibile utilizzabile solo su metodi e campi.

È quindi importante progettare con attenzione un'annotazione affinché sia il più possibile flessibile e utile per chi la utilizza.

## 8 Interfacce

Un'interfaccia è un tipo riferimento che contiene solamente costanti, firme di metodi e tipi annidati. Il corpo dei metodi va inserito solo per metodi di default e metodi statici.

Le interfacce non possono essere istanziate, possono solo essere implementate da classi o estese da altre interfacce.

Definire un'interfaccia è molto simile a dichiarare una classe:

```
1 public interface OperateCar {
2     // constant declarations, if any
3
4     // method signatures
5
6     // An enum with values RIGHT, LEFT
7     int turn(Direction direction, double radius, double startSpeed, double
        endSpeed);
8     int changeLanes(Direction direction, double startSpeed, double endSpeed);
9     int signalTurn(Direction direction, boolean signalOn);
10    int getRadarFront(double distanceToCar, double speedOfCar);
11    int getRadarRear(double distanceToCar, double speedOfCar);
12    .....
13    // more method signatures
14 }
```

Da notare che i metodi non hanno parentesi graffe alla fine e terminano con un ;.

Per usare un'interfaccia serve dichiarare una classe che implementi l'interfaccia. Quando una classe istanziabile implementa un'interfaccia, deve anche fornire il corpo per i metodi dichiarati nell'interfaccia.

```
1 public class OperateBMW760i implements OperateCar {
2
3     // the OperateCar method signatures, with implementation --
4     // for example:
5     public int signalTurn(Direction direction, boolean signalOn) {
6         // code to turn BMW's LEFT turn indicator lights on
7         // code to turn BMW's LEFT turn indicator lights off
8         // code to turn BMW's RIGHT turn indicator lights on
9         // code to turn BMW's RIGHT turn indicator lights off
10    }
11
12    // other members, as needed -- for example, helper classes not
13    // visible to clients of the interface
14 }
```

### 8.1 Usare un'interfaccia come un tipo

Quando si definisce una nuova interfaccia si sta definendo un nuovo tipo riferimento. Un'interfaccia si può usare in ogni punto in cui si poteva utilizzare un qualunque altro tipo di dato.

Se si definisce una variabile di riferimento che ha come tipo un'interfaccia, un oggetto assegnato ad essa deve obbligatoriamente essere un'istanza di una classe che implementa l'interfaccia.

```

1 public Object findLargest(Object object1, Object object2) {
2     Relatable obj1 = (Relatable)object1;
3     Relatable obj2 = (Relatable)object2;
4     if ((obj1).isLargerThan(obj2) > 0)
5         return object1;
6     else
7         return object2;
8 }

```

Nell'esempio proposto `object1` e `object2` devono essere istanze di un tipo `Object` che implementi il metodo `isLargerThan` definito nell'interfaccia `Relatable`.

## 8.2 Evoluzione delle interfacce

Se un'interfaccia, usata per la definizione di centinaia di classi, venisse cambiata aggiungendo un metodo andrebbero riscritte tutte le classi che la implementano.

Per evitare ciò si scrive una nuova interfaccia che estende la vecchia, permettendo all'utente di scegliere se passare alla nuova interfaccia o continuare ad usare quella vecchia.

Una soluzione moderna è l'uso dei default methods.

## 8.3 Default methods

Introdotti in Java SE 8, i metodi con modificatore `default` permettono di dichiarare in una interfaccia anche l'implementazione del metodo, che verrà usata come implementazione default. In seguito, tramite `@Override`, una classe che implementa l'interfaccia potrà sovrascrivere il corpo default del metodo.

```

1 default ZonedDateTime getZonedDateTime(String zoneString) {
2     return ZonedDateTime.of(getLocalDateTime(), getZoneId(zoneString));
3 }

```

Una interfaccia che estende un'interfaccia contenente un metodo `default` può fare tre cose riguardo al metodo:

```

1 // Eredita il metodo
2 interface AnotherTimeClient extends TimeClient { }
3
4 // Rende nuovamente il metodo astratto
5 interface AbstractZoneTimeClient extends TimeClient {
6     ZonedDateTime getZonedDateTime(String zoneString);
7 }
8
9 // Ridefinisce il corpo del metodo
10 interface HandleInvalidTimeZoneClient extends TimeClient {
11     default ZonedDateTime getZonedDateTime(...) {
12         ...
13     }
14 }

```

## 9 Ereditarietà

Una classe derivata da un'altra classe si chiama *sottoclasse* (anche detta *classe derivata*, *classe estesa* o *classe figlio*). La classe da cui la sottoclasse è derivata si dice *superclasse* (anche detta *classe base* o *classe padre*).

Eccetto `Object`, che non ha una superclasse, ogni classe ha una e una sola superclasse. In assenza di una superclasse esplicita, ogni classe è implicitamente una sottoclasse di `Object`.

Una sottoclasse eredita tutti i membri della superclasse. Non eredita i costruttori (in quanto non sono membri), che però possono essere invocati all'interno dei costruttori della sottoclasse.

Una sottoclasse non eredita i membri `private` della classe padre. Nonostante ciò, se la superclasse ha metodi `public` o `protected` per accedere ai suoi campi privati, la sottoclasse può usare quelli per ottenere i membri nascosti.

### 9.1 Casting

Un oggetto può essere usato come istanza della sua classe o di una sua superclasse (upcasting), ed è automatico. Il contrario (downcasting) non è automatico e richiede un cast esplicito.

```
1 // Upcasting
2 Object obj = new MountainBike();
3
4 // Downcasting
5 MountainBike bike = (MountainBike) obj;
```

Se l'oggetto non è realmente un `MountainBike`, il cast genera una `ClassCastException` a runtime. Per evitare errori è buona pratica usare `instanceof` prima del cast.

```
1 if (obj instanceof MountainBike) {
2     MountainBike bike = (MountainBike) obj;
3 }
```

### 9.2 Ereditarietà multipla: stato, implementazione e tipo

Java non consente l'ereditarietà multipla tra classi per evitare conflitti nell'ereditare stato (campi) e implementazione (metodi). Permette invece l'ereditarietà multipla di tipo, consentendo a una classe di implementare più interfacce.

Se più interfacce forniscono lo stesso default method, il compilatore (o il programmatore) deve stabilire quale implementazione utilizzare.

### 9.3 Sovrascrivere e nascondere metodi

#### 9.3.1 Metodi d'istanza

Un metodo d'istanza in una sottoclasse con la stessa firma e valore di ritorno del metodo d'istanza della superclasse sovrascrive il metodo della superclasse.

Questo permette ad una sottoclasse di modificare di poco il comportamento di un metodo, mantenendo però la maggior parte delle funzionalità della superclasse. Un esempio è cambiare solo il valore di ritorno del metodo, restituendo un sottotipo del tipo restituito dal metodo sovrascritto. Questo sottotipo si dice *covariant return type*.

Quando si sovrascrive un metodo, si vuole usare l'annotation `@Override` che informa il compilatore che si vuole sovrascrivere un metodo della superclasse.

### 9.3.2 Metodi statici

Se una sottoclasse dichiara un metodo **static** con la stessa firma di un metodo con modificatore **static** della superclasse, allora il metodo della superclasse viene nascosto da quello della sottoclasse.

La differenza principale tra metodi d'istanza e statici è che i metodi static non vengono sovrascritti davvero, ma solo nascosti. Con i metodi d'istanza, Java guarda l'istanza dell'oggetto, mentre quelli statici appartengono alla classe, non all'istanza, quindi il metodo scelto dipende dal tipo usato nella chiamata.

```
1 Animal myAnimal = new Cat();
2 myAnimal.testInstanceMethod(); // Cat
3 myAnimal.testClassMethod();   // Animal
```

### 9.3.3 Metodi d'interfaccia

I metodi **default** e **astratti** delle interfacce vengono ereditati, ma se più supertipi forniscono metodi con la stessa firma Java deve scegliere quale usare. La regola principale è: i metodi d'istanza delle classi hanno precedenza sui metodi **default** delle interfacce. Se invece due interfacce derivano da un antenato comune, Java ignora i metodi già superati da versioni più specifiche.

Quando però due interfacce indipendenti forniscono lo stesso **default method**, Java non può scegliere da solo: la classe deve fare **override** esplicito del metodo.

```
1 class Persona {
2     public String saluta() {
3         return "Ciao, sono una persona.";
4     }
5 }
6
7 interface Studente {
8     default String saluta() {
9         return "Ciao, sono uno studente.";
10    }
11 }
12
13 interface Lavoratore {
14     default String saluta() {
15         return "Ciao, sono un lavoratore.";
16    }
17 }
18
19 class StudenteLavoratore extends Persona implements Studente, Lavoratore {
20 }
```

Se dichiarassi del codice come segue, l'output sarebbe *"Ciao, sono una persona."*:

```
1 StudenteLavoratore x = new StudenteLavoratore();
2 System.out.println(x.saluta());
```

Java preferisce il metodo concreto della classe **Persona** rispetto ai **default method** delle interfacce **Studente** e **Lavoratore**.

Se invece faccio come segue:

```
1 class StudenteLavoratore implements Studente, Lavoratore {  
2 }
```

questa classe dà errore, perché `Studente` e `Lavoratore` hanno entrambe un default `saluta()` e Java non può scegliere. Per fare ciò è necessario risolvere esplicitamente il conflitto:

```
1 class StudenteLavoratore implements Studente, Lavoratore {  
2     public String saluta() {  
3         return Studente.super.saluta();  
4     }  
5 }
```

Ora l'output sarà *"Ciao, sono uno studente."*.

### 9.3.4 Override di metodi

Quando una sottoclasse fa override di un metodo, può renderlo più accessibile, ma non meno accessibile.

Per esempio, se nella superclasse il metodo è `protected`, nella sottoclasse puoi farlo diventare `public`, ma non `private`. La gerarchia è `public > protected > package-private > private`.

Non è inoltre possibile trasformare un metodo d'istanza in metodo statico durante l'override, né il contrario: un metodo d'istanza deve rimanere d'istanza, un metodo statico deve rimanere statico.

## 9.4 Polimorfismo

Sottoclassi di una classe possono definire un loro comportamento unico, pur condividendo parte della logica usata dalla classe padre.

Immaginiamo che una classe implementi il metodo `printDescription`:

```
1 public void printDescription(){
2     System.out.println("\nBike is " + "in gear " + this.gear
3         + " with a cadence of " + this.cadence +
4         " and travelling at a speed of " + this.speed + ". ");
5 }
```

Immaginiamo poi che una sottoclasse estenda la superclasse che implementa `printDescription`:

```
1 public void printDescription() {
2     super.printDescription();
3     System.out.println("The " + "MountainBike has a" +
4         getSuspension() + " suspension.");
5 }
```

Questa versione di `printDescription`, oltre alle informazioni già fornite dal metodo della classe padre, aggiunge delle informazioni contenute solo nella classe figlio. Il seguente codice mostra diversi esempi di polimorfismo per il metodo `printDescription`:

```
1 public class TestBikes {
2     public static void main(String[] args){
3         Bicycle bike01, bike02, bike03;
4
5         bike01 = new Bicycle(20, 10, 1);
6         bike02 = new MountainBike(20, 10, 5, "Dual");
7         bike03 = new RoadBike(40, 20, 8, 23);
8
9         bike01.printDescription();
10        // Bike is in gear 1 with a cadence of 20 and travelling at a speed of 10.
11
12        bike02.printDescription();
13        // Bike is in gear 5 with a cadence of 20 and travelling at a speed of 10.
14        // The MountainBike has a Dual suspension.
15
16        bike03.printDescription();
17        // Bike is in gear 8 with a cadence of 40 and travelling at a speed of 20.
18        // The RoadBike has 23 MM tires.
19    }
20 }
```

## 9.5 Keyword super

### 9.5.1 Accedere ai membri della superclasse

Se un metodo sovrascrive un metodo della sua superclasse, si può invocare il metodo sovrascritto usando la keyword **super**. Si può anche usare per fare riferimento ad un campo nascosto (anche se non andrebbe MAI fatto).

```
1 public class Superclass {
2     public void printMethod() {
3         System.out.println("Printed in Superclass.");
4     }
5 }
6
7 public class Subclass extends Superclass {
8     // overrides printMethod in Superclass
9     public void printMethod() {
10        super.printMethod();
11        System.out.println("Printed in Subclass");
12    }
13
14    public static void main(String[] args) {
15        Subclass s = new Subclass();
16
17        s.printMethod();
18        // Output
19        // Printed in Superclass
20        // Printed in Subclass
21    }
22 }
```

### 9.5.2 Costruttori della sottoclasse

Il seguente esempio mostra come usare la keyword **super** per invocare il costruttore della superclasse:

```
1 public MountainBike(int startHeight,
2                     int startCadence,
3                     int startSpeed,
4                     int startGear) {
5     super(startCadence, startSpeed, startGear);
6     seatHeight = startHeight;
7 }
```

Da notare che se un costruttore non invoca esplicitamente un costruttore della superclasse, il compilatore di Java chiama automaticamente il costruttore senza argomenti della superclasse. Se la superclasse non ha un costruttore senza argomenti, verrà segnalato un errore a compile-time.

## 9.6 Object come Superclasse

La classe `Object`, contenuta nel pacchetto `java.lang`, siede alla cima dell'albero gerarchico. Ogni classe deriva, in modo diretto o indiretto, dalla classe `Object`, ereditandone i metodi.

### 9.6.1 Metodo clone

Il metodo `clone()` crea e restituisce una copia dell'oggetto in questione se la classe implementa l'interfaccia `Cloneable`.

L'implementazione di `Object` verifica se la classe che ha chiamato `clone()` implementi realmente l'interfaccia `Cloneable`; se così non fosse, un'eccezione `CloneNotSupportedException`.

Se si vuole scrivere un metodo `clone()` per sovrascrivere quello di default, è necessario che venga dichiarato come `protected Object clone() throws CloneNotSupportedException` oppure `public Object clone() throws CloneNotSupportedException`.

Il modo più semplice per rendere una classe clonabile è aggiungere `implements Cloneable` alla dichiarazione della classe.

### 9.6.2 Metodo equals

Il metodo `equals()` compara due oggetti e restituisce `true` se sono uguali. Il metodo fornito da `Object` usa l'operatore `==` per determinare l'uguaglianza: per i tipi di dati primitivi ciò restituisce il risultato corretto, ma non per gli oggetti.

Il metodo `equals()` verifica se due oggetti sono uguali per riferimento, non in base ai valori contenuti all'interno di essi. Per testare quest'ultimo tipo di uguaglianza è necessario sovrascrivere il metodo, ad esempio facendogli confrontare il valore di uno o più attributi delle istanze di cui è stato chiamato il metodo.

### 9.6.3 Metodo finalize

Il metodo `finalize()` può essere chiamato dal garbage collector quando un oggetto sta per essere eliminato, ma non c'è garanzia né su quando venga chiamato né sul fatto che venga chiamato davvero.

Per questo non bisogna usarlo per liberare risorse importanti, come file, socket o connessioni. Ad esempio, aspettando `finalize()` per chiudere un file, si potrebbero esaurire i file descriptor prima che Java decida di eseguire la pulizia.

La soluzione corretta è usare try-with-resources, che chiude automaticamente le risorse appena non servono più:

```
1 try (FileReader reader = new FileReader("file.txt")) {
2     // uso il file
3 }
```

### 9.6.4 Metodo getClass

Non è possibile fare override del metodo `getClass()`. Il metodo restituisce un oggetto `Class`, contenente metodi utili per ottenere alcune informazioni quali:

- `getSimpleName()`: restituisce il nome della classe;
- `getSuperclass()`: restituisce il nome della superclasse;
- `getInterfaces()`: restituisce le interfacce implementate;

- `getDeclaredMethods()`: restituisce i metodi dichiarati nella classe;
- `getDeclaredFields()`: restituisce gli attributi dichiarati nella classe.

La classe `Class` contiene oltre 50 metodi (oltre a quelli elencati sopra): altri degni di nota sono `isAnnotation()`, `isInterface()` e `isEnum()` per determinare se una classe è, rispettivamente, un'annotazione, un'interfaccia o un'enumerazione.

### 9.6.5 Metodo `hashCode`

Il valore restituito da `hashCode()` è un valore intero generato da un algoritmo di hashing. Per definizione, se due oggetti sono uguali, allora anche il loro codice hash deve essere uguale. Se in precedenza il metodo `equals()` è stato sovrascritto, sarà necessario riscrivere anche il metodo `hashCode()`.

```

1  import java.util.Objects;
2
3  class Studente {
4      String nome;
5      int matricola;
6
7      Studente(String nome, int matricola) {
8          this.nome = nome;
9          this.matricola = matricola;
10     }
11
12     @Override
13     public boolean equals(Object obj) {
14         if (this == obj) return true;
15         if (!(obj instanceof Studente)) return false;
16
17         Studente altro = (Studente) obj;
18         return this.matricola == altro.matricola;
19     }
20
21     @Override
22     public int hashCode() {
23         return Objects.hash(matricola);
24     }
25 }

```

```

1  Studente s1 = new Studente("Luca", 123);
2  Studente s2 = new Studente("Marco", 123);
3
4  System.out.println(s1.equals(s2)); // true

```

### 9.6.6 Metodo `toString`

Il metodo `toString()` restituisce una rappresentazione in `String` dell'oggetto, cosa molto utile per il debugging.

Andrebbe sempre considerato di sovrascrivere il metodo `toString()` per rappresentare al meglio i valori contenuti all'interno di un'istanza di una classe.

## 9.7 Classi e metodi `final`

Si possono dichiarare alcuni metodi di una classe come `final`. Questa keyword indica che il metodo non può essere sovrascritto dalle sottoclassi ottenute da quella classe. La classe `Object` fa questo per alcuni suoi metodi, come ad esempio `getClass()` descritto prima.

```
1 class ChessAlgorithm {
2     enum ChessPlayer { WHITE, BLACK }
3     ...
4     final ChessPlayer getFirstPlayer() {
5         return ChessPlayer.WHITE;
6     }
7     ...
8 }
```

Andrebbe usato solo per metodi la cui implementazione è critica per la consistenza dell'oggetto: citando un possibile caso d'uso, i costruttori dovrebbero chiamare sempre metodi dichiarati come `final` per evitare comportamenti imprevisti nel caso in cui una sottoclasse sovrascrivesse il metodo.

In alcuni casi potrebbe risultare utile dichiarare un'intera classe come `final`, rendendo impossibile creare sottoclassi da essa. Risulta utile quando si vuole dichiarare una classe immutabile come `String`

## 9.8 Classi e metodi astratti

Una classe astratta non può essere istanziata direttamente e serve come base per altre classi. Può contenere sia metodi concreti sia metodi astratti, cioè dichiarati senza implementazione.

Se una classe contiene almeno un metodo astratto, deve essere dichiarata `abstract`. Una sottoclasse deve implementare tutti i metodi astratti ereditati; in caso contrario, anche la sottoclasse deve essere dichiarata `abstract`.

### 9.8.1 Classi astratte vs Interfacce

Classi astratte e interfacce sono simili perché non possono essere istanziate e possono dichiarare metodi senza implementazione. La differenza principale è che una classe astratta può avere stato interno, cioè campi non `static` e non `final`, e metodi con visibilità `public`, `protected` o `private`. Un'interfaccia, invece, definisce soprattutto un comportamento comune: i suoi campi sono sempre `public static final` e i suoi metodi sono pubblici.

- **Classe astratta:** più classi sono strettamente collegate e devono condividere codice, campi o stato interno;
- **Interfaccia:** serve definire un comportamento che può essere implementato anche da classi non collegate tra loro, oppure si vuole sfruttare il fatto che una classe può implementare più interfacce.

### 9.8.2 Classe astratta che implementa un'interfaccia

Una classe che implementa un'interfaccia deve normalmente implementare tutti i suoi metodi. Però, se la classe è dichiarata **abstract**, può lasciare alcuni metodi non implementati.

In quel caso sarà una sottoclasse concreta a completare l'implementazione:

```
1 interface Animale {
2     void mangia();
3     void dorme();
4 }
5
6 abstract class Mammifero implements Animale {
7     public void mangia() {
8         System.out.println("Mangia");
9     }
10
11     // dorme() non e implementato
12 }
13
14 class Cane extends Mammifero {
15     public void dorme() {
16         System.out.println("Dorme");
17     }
18 }
```

Mammifero deve essere **abstract** perché non implementa tutto **Animale**. Cane, invece, può essere concreta perché implementa il metodo mancante.

Una classe astratta può anche avere campi e metodi **static**, usabili come segue:

```
1 abstract class Animale {
2     public static void descrizione() {
3         System.out.println("Gli animali sono esseri viventi.");
4     }
5 }
6
7 public class Main {
8     public static void main(String[] args) {
9         Animale.descrizione();
10        // Output: Gli animali sono esseri viventi.
11
12        Animale a = new Animale(); // Errore, non puo essere istanziata
13    }
14 }
```

## 10 Strutture dati e Collections

### 10.1 Introduzione alle Collections

Una *collection*, chiamata anche contenitore, è un oggetto che raggruppa più elementi in una singola unità. Le collections vengono usate per memorizzare, recuperare, manipolare e comunicare gruppi di dati.

Una collection rappresenta normalmente un insieme di elementi che hanno un legame naturale tra loro. Ad esempio, una mano di carte può essere vista come una collection di carte, una cartella di posta come una collection di messaggi, mentre una rubrica telefonica può essere vista come una struttura che associa nomi a numeri di telefono.

L'utilità principale delle collections è evitare di gestire manualmente gruppi di oggetti tramite strutture più rigide o meno espressive. In Java, infatti, le collections permettono di lavorare con insiemi di dati usando interfacce e implementazioni standard già fornite dalla piattaforma.

### 10.2 Collections Framework

Un *collections framework* è un'architettura unificata per rappresentare e manipolare collections. In Java, il *Java Collections Framework* fornisce un insieme di interfacce, implementazioni e algoritmi che permettono di gestire gruppi di oggetti in modo uniforme.

Le interfacce definiscono i tipi astratti di collection. Permettono di usare una collection senza dipendere dai dettagli della sua implementazione concreta. Questo è coerente con il principio di programmazione orientata agli oggetti secondo cui il codice dovrebbe dipendere il più possibile da astrazioni e non da implementazioni specifiche.

Le implementazioni sono le classi concrete che realizzano le interfacce del framework. In pratica, sono strutture dati riutilizzabili già pronte, come liste, insiemi e mappe. Ogni implementazione può avere caratteristiche diverse in termini di ordine degli elementi, duplicati ammessi, prestazioni e uso della memoria.

Gli algoritmi sono metodi che eseguono operazioni utili sulle collections, come ordinamento, ricerca, copia o mescolamento degli elementi. Questi algoritmi sono polimorfici, perché possono lavorare su implementazioni diverse della stessa interfaccia. In questo modo lo stesso algoritmo può essere applicato a più strutture dati senza dover essere riscritto.

### 10.3 Componenti principali del framework

Il Java Collections Framework si basa su tre componenti principali.

| Componente      | Ruolo                                                                                                                              |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------|
| Interfacce      | Definiscono i tipi astratti di collection e permettono di manipolare gruppi di oggetti senza conoscere l'implementazione concreta. |
| Implementazioni | Sono le classi concrete che implementano le interfacce. Rappresentano strutture dati riutilizzabili.                               |
| Algoritmi       | Sono metodi che eseguono operazioni comuni sulle collections, come ricerca e ordinamento.                                          |

Questa separazione è importante perché permette di scrivere codice più flessibile. Ad esempio, un metodo può ricevere come parametro un'interfaccia invece di una classe concreta. In questo modo il metodo può funzionare con implementazioni diverse.

```

1 public void stampaElementi(Collection<String> elementi) {
2     for (String elemento : elementi) {
3         System.out.println(elemento);
4     }
5 }

```

Il metodo precedente non dipende da una particolare implementazione. Può ricevere, ad esempio, una `ArrayList`, una `LinkedList` o un `HashSet`, purché l'oggetto passato implementi l'interfaccia `Collection`.

## 10.4 Vantaggi del Java Collections Framework

Il Java Collections Framework riduce lo sforzo di programmazione perché fornisce strutture dati e algoritmi già pronti. Il programmatore non deve implementare manualmente liste, insiemi, mappe, algoritmi di ricerca o algoritmi di ordinamento, ma può concentrarsi sulla logica specifica dell'applicazione.

Un altro vantaggio è l'aumento della qualità e della velocità dei programmi. Le implementazioni fornite dalla piattaforma Java sono generiche, riutilizzabili e ottimizzate. Inoltre, poiché le varie implementazioni della stessa interfaccia sono intercambiabili, è possibile modificare la struttura dati usata senza riscrivere tutta la logica del programma.

Il framework favorisce anche l'interoperabilità tra API diverse. Se due librerie indipendenti usano le stesse interfacce standard, possono scambiarsi collections senza richiedere conversioni complesse o classi adattatrici. Questo rende più semplice integrare codice scritto da sviluppatori diversi.

Le collections riducono anche lo sforzo necessario per imparare nuove API. Molte librerie ricevono collections come input o le restituiscono come output. Poiché le interfacce sono standard, non è necessario imparare ogni volta un modo diverso per manipolare gruppi di oggetti.

Infine, il framework favorisce il riuso del software. Nuove strutture dati che implementano le interfacce standard possono essere usate ovunque sia richiesta quella interfaccia. Allo stesso modo, nuovi algoritmi scritti per una certa interfaccia possono funzionare con tutte le sue implementazioni.

## 10.5 Uso delle interfacce come tipo

Uno degli aspetti più importanti del Java Collections Framework è l'uso delle interfacce come tipo delle variabili, dei parametri e dei valori di ritorno. Invece di dichiarare una variabile usando direttamente una classe concreta, spesso è preferibile usare l'interfaccia più generale che descrive le operazioni necessarie.

```

1 List<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add("Luigi");
5 nomi.add("Anna");

```

In questo esempio, la variabile è dichiarata come `List<String>`, mentre l'oggetto concreto è una `ArrayList`. Questo rende il codice più flessibile, perché in futuro l'implementazione potrebbe essere cambiata senza modificare il resto del programma.

```

1 List<String> nomi = new LinkedList<>();

```

Il codice che usa `nomi` può rimanere lo stesso, purché utilizzi operazioni definite dall'interfaccia `List`.

## 10.6 Interfaccia Collection

### 10.6.1 Concetto generale

L'interfaccia `Collection` rappresenta un gruppo di oggetti, detti elementi. È l'interfaccia più generale del Java Collections Framework per rappresentare una raccolta di elementi.

Si usa `Collection` quando si vuole scrivere codice indipendente dall'implementazione concreta. Una variabile di tipo `Collection`, infatti, può riferirsi a strutture diverse, come una lista o un insieme, purché implementino l'interfaccia `Collection`.

```
1 Collection<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add("Luigi");
5 nomi.add("Anna");
```

L'interfaccia `Collection` è utile anche per convertire una collection in un'altra. Molte implementazioni generiche, infatti, hanno un costruttore che riceve una `Collection` e inizializza la nuova struttura con gli stessi elementi.

```
1 Collection<String> c = new HashSet<>();
2
3 c.add("Java");
4 c.add("Python");
5 c.add("C++");
6
7 List<String> lista = new ArrayList<>(c);
```

In questo esempio, gli elementi contenuti nella `Collection` vengono copiati in una nuova `ArrayList`.

### 10.6.2 Metodi principali

L'interfaccia `Collection` definisce metodi per conoscere lo stato della raccolta, aggiungere o rimuovere elementi e attraversare il contenuto.

| Metodo                          | Significato                                                            |
|---------------------------------|------------------------------------------------------------------------|
| <code>size()</code>             | Restituisce il numero di elementi contenuti nella collection.          |
| <code>isEmpty()</code>          | Restituisce <code>true</code> se la collection non contiene elementi.  |
| <code>contains(Object o)</code> | Controlla se un elemento è presente nella collection.                  |
| <code>add(E e)</code>           | Aggiunge un elemento alla collection, se l'operazione è supportata.    |
| <code>remove(Object o)</code>   | Rimuove una singola occorrenza dell'elemento specificato, se presente. |
| <code>iterator()</code>         | Restituisce un iteratore per attraversare gli elementi.                |

Il metodo `add` restituisce `true` se la collection viene modificata. Questo è importante perché alcune collection non ammettono duplicati, come i `Set`. In quel caso, aggiungere un elemento già presente può non modificare la collection.

```

1 Collection<String> linguaggi = new ArrayList<>();
2
3 linguaggi.add("Java");
4 linguaggi.add("Python");
5
6 System.out.println(linguaggi.size());
7 System.out.println(linguaggi.contains("Java"));
8
9 linguaggi.remove("Python");

```

### 10.6.3 Attraversamento di una Collection

Il modo più semplice per attraversare una `Collection` è usare il costrutto `for-each`. Questo permette di leggere tutti gli elementi senza gestire manualmente gli indici.

```

1 Collection<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add("Luigi");
5 nomi.add("Anna");
6
7 for (String nome : nomi) {
8     System.out.println(nome);
9 }

```

Il `for-each` è adatto quando bisogna solo leggere gli elementi. Se invece è necessario rimuovere elementi durante l'iterazione, è meglio usare un `Iterator`.

```

1 Collection<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add("Luigi");
5 nomi.add("Anna");
6
7 Iterator<String> it = nomi.iterator();
8
9 while (it.hasNext()) {
10     String nome = it.next();
11
12     if (nome.startsWith("A")) {
13         it.remove();
14     }
15 }

```

Il metodo `remove()` dell'iteratore rimuove l'ultimo elemento restituito da `next()`. È il modo corretto per modificare una collection mentre la si sta attraversando. Modificare direttamente la collection durante un ciclo può causare comportamenti errati o eccezioni.

### 10.6.4 Operazioni bulk

Le operazioni *bulk* sono metodi che lavorano su un'intera collection invece che su un singolo elemento. Sono utili per confrontare, unire o modificare gruppi di elementi.

| Metodo                                               | Significato                                                                                              |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>containsAll(Collection&lt;?&gt; c)</code>      | Restituisce <code>true</code> se la collection contiene tutti gli elementi della collection specificata. |
| <code>addAll(Collection&lt;? extends E&gt; c)</code> | Aggiunge tutti gli elementi della collection specificata.                                                |
| <code>removeAll(Collection&lt;?&gt; c)</code>        | Rimuove tutti gli elementi presenti anche nella collection specificata.                                  |
| <code>retainAll(Collection&lt;?&gt; c)</code>        | Mantiene solo gli elementi presenti anche nella collection specificata.                                  |
| <code>clear()</code>                                 | Rimuove tutti gli elementi.                                                                              |

I metodi `addAll`, `removeAll` e `retainAll` restituiscono `true` se la collection viene modificata.

```

1 Collection<String> a = new ArrayList<>();
2 a.add("Java");
3 a.add("Python");
4 a.add("C++");
5
6 Collection<String> b = new ArrayList<>();
7 b.add("Python");
8 b.add("C++");
9
10 a.removeAll(b);
11
12 System.out.println(a);

```

In questo esempio da `a` vengono rimossi tutti gli elementi presenti anche in `b`. Alla fine rimane solo `Java`.

Un uso comune di `removeAll` è la rimozione di tutte le occorrenze di un elemento.

```

1 Collection<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add(null);
5 nomi.add("Luigi");
6 nomi.add(null);
7
8 nomi.removeAll(Collections.singleton(null));

```

Il metodo `Collections.singleton(null)` crea un insieme immutabile contenente solo `null`. Passandolo a `removeAll`, vengono rimossi tutti i valori nulli dalla collection.

### 10.6.5 Conversione in array

L'interfaccia `Collection` fornisce il metodo `toArray()`, utile quando bisogna passare gli elementi a codice che lavora con array invece che con collections.

```

1 Collection<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add("Luigi");
5
6 Object[] array = nomi.toArray();

```

La versione senza argomenti restituisce un array di `Object`. Se si vuole ottenere un array di un

tipo specifico, si usa la versione che riceve un array come parametro.

```
1 Collection<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add("Luigi");
5
6 String[] array = nomi.toArray(new String[0]);
```

Questa forma è più precisa perché restituisce un array di **String** invece di un array generico di **Object**.

### 10.6.6 Stream

Da Java 8, una **Collection** può produrre uno stream tramite il metodo **stream()**. Uno stream permette di eseguire operazioni aggregate sugli elementi, come filtri, trasformazioni e raccolta dei risultati.

```
1 Collection<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add("Luigi");
5 nomi.add("Anna");
6
7 nomi.stream()
8     .filter(nome -> nome.startsWith("A"))
9     .forEach(nome -> System.out.println(nome));
```

In questo esempio vengono stampati solo i nomi che iniziano con A. Gli stream sono utili per esprimere operazioni su gruppi di dati in modo compatto, ma non vanno confusi con le operazioni bulk come **addAll** o **removeAll**. Le operazioni bulk modificano direttamente la collection, mentre le operazioni sugli stream sono pensate per elaborare i dati senza modificare la sorgente.

## 10.7 Set

### 10.7.1 Concetto generale

Un **Set** è una **Collection** che non può contenere elementi duplicati. Rappresenta quindi il concetto matematico di insieme: un elemento può appartenere oppure non appartenere al set, ma non può comparire più volte.

L'interfaccia **Set** non aggiunge nuovi metodi fondamentali rispetto a **Collection**; eredita metodi come **add**, **remove**, **contains**, **size** e **iterator**. La differenza principale è nel vincolo sui duplicati.

```
1 Set<String> linguaggi = new HashSet<>();
2
3 linguaggi.add("Java");
4 linguaggi.add("Python");
5 linguaggi.add("Java");
6
7 System.out.println(linguaggi);
8 System.out.println(linguaggi.size());
```

In questo esempio "Java" viene aggiunto due volte, ma nel **Set** compare una sola volta. Il metodo **add** restituisce **true** solo se il set viene effettivamente modificato.

```
1 Set<String> nomi = new HashSet<>();
2
3 System.out.println(nomi.add("Mario")); // true
4 System.out.println(nomi.add("Mario")); // false
```

Due oggetti **Set** sono considerati uguali se contengono gli stessi elementi, anche se sono implementati da classi diverse.

### 10.7.2 Implementazioni principali

La piattaforma Java fornisce tre implementazioni generiche principali dell'interfaccia **Set**: **HashSet**, **TreeSet** e **LinkedHashSet**.

| Implementazione      | Caratteristiche                                                                                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>HashSet</b>       | È generalmente l'implementazione più efficiente. Non garantisce alcun ordine di iterazione degli elementi.                                                          |
| <b>TreeSet</b>       | Mantiene gli elementi ordinati secondo il loro ordinamento naturale o secondo un comparatore. È meno efficiente di <b>HashSet</b> , ma utile quando serve l'ordine. |
| <b>LinkedHashSet</b> | Mantiene l'ordine di inserimento degli elementi. Ha un costo leggermente maggiore rispetto a <b>HashSet</b> .                                                       |

Come buona pratica, la variabile dovrebbe essere dichiarata usando l'interfaccia **Set**, non la classe concreta. In questo modo è possibile cambiare implementazione senza modificare il resto del codice.

```

1 Set<String> parole = new HashSet<>();
2
3 parole.add("i");
4 parole.add("came");
5 parole.add("i");
6 parole.add("saw");
7 parole.add("i");
8 parole.add("left");
9
10 System.out.println(parole);

```

Se si vuole ottenere l'output in ordine alfabetico, basta cambiare l'implementazione concreta.

```

1 Set<String> parole = new TreeSet<>();

```

Il resto del codice può rimanere invariato, perché continua a usare i metodi definiti dall'interfaccia `Set`.

### 10.7.3 Rimozione dei duplicati

Un uso comune di `Set` è rimuovere duplicati da una collection. Poiché un set non ammette duplicati, basta creare un nuovo `Set` passando al costruttore la collection di partenza.

```

1 Collection<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add("Luigi");
5 nomi.add("Mario");
6 nomi.add("Anna");
7
8 Set<String> senzaDuplicati = new HashSet<>(nomi);
9
10 System.out.println(senzaDuplicati);

```

Se si vuole rimuovere i duplicati mantenendo l'ordine originale di inserimento, si può usare `LinkedHashSet`.

```

1 Collection<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add("Luigi");
5 nomi.add("Mario");
6 nomi.add("Anna");
7
8 Set<String> senzaDuplicati = new LinkedHashSet<>(nomi);
9
10 System.out.println(senzaDuplicati);

```

Si può anche incapsulare questo comportamento in un metodo generico.

```

1 public static <E> Set<E> removeDups(Collection<E> c) {
2     return new LinkedHashSet<>(c);
3 }

```

Il metodo riceve una `Collection` qualsiasi e restituisce un `Set` contenente gli stessi elementi, ma senza duplicati.

#### 10.7.4 Operazioni insiemistiche

Le operazioni bulk ereditate da `Collection` sono particolarmente utili con i `Set`, perché corrispondono a operazioni tipiche sugli insiemi.

| Operazione                      | Significato                                                                         |
|---------------------------------|-------------------------------------------------------------------------------------|
| <code>s1.containsAll(s2)</code> | Verifica se <code>s2</code> è sottoinsieme di <code>s1</code> .                     |
| <code>s1.addAll(s2)</code>      | Trasforma <code>s1</code> nell'unione tra <code>s1</code> e <code>s2</code> .       |
| <code>s1.retainAll(s2)</code>   | Trasforma <code>s1</code> nell'intersezione tra <code>s1</code> e <code>s2</code> . |
| <code>s1.removeAll(s2)</code>   | Rimuove da <code>s1</code> tutti gli elementi presenti in <code>s2</code> .         |

Queste operazioni modificano il set su cui vengono chiamate. Se si vuole calcolare il risultato senza modificare i set originali, bisogna prima creare una copia.

```
1 Set<String> s1 = new HashSet<>();
2 s1.add("A");
3 s1.add("B");
4 s1.add("C");
5
6 Set<String> s2 = new HashSet<>();
7 s2.add("B");
8 s2.add("C");
9 s2.add("D");
10
11 Set<String> union = new HashSet<>(s1);
12 union.addAll(s2);
13
14 Set<String> intersection = new HashSet<>(s1);
15 intersection.retainAll(s2);
16
17 Set<String> difference = new HashSet<>(s1);
18 difference.removeAll(s2);
19
20 System.out.println("Unione: " + union);
21 System.out.println("Intersezione: " + intersection);
22 System.out.println("Differenza: " + difference);
```

L'unione contiene tutti gli elementi presenti in almeno uno dei due set. L'intersezione contiene solo gli elementi comuni. La differenza contiene gli elementi presenti nel primo set ma non nel secondo.

## 10.8 List

### 10.8.1 Concetto generale

Una **List** è una **Collection** ordinata, detta anche sequenza. A differenza di un **Set**, una lista può contenere elementi duplicati e ogni elemento ha una posizione numerica, detta indice.

Gli indici partono da 0. Questo significa che il primo elemento della lista si trova in posizione 0, il secondo in posizione 1 e così via.

```
1 List<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add("Luigi");
5 nomi.add("Mario");
6
7 System.out.println(nomi);
8 System.out.println(nomi.get(0));
```

In questo esempio "Mario" può comparire più volte, perché le liste ammettono duplicati. Inoltre, l'ordine degli elementi viene mantenuto.

Due oggetti **List** sono considerati uguali se contengono gli stessi elementi nello stesso ordine, anche se sono implementati da classi diverse.

### 10.8.2 Implementazioni principali

Le due implementazioni generiche principali dell'interfaccia **List** sono **ArrayList** e **LinkedList**.

| Implementazione   | Caratteristiche                                                                                                                                                              |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ArrayList</b>  | È basata su un array ridimensionabile. È generalmente la scelta più comune ed efficiente per accessi tramite indice e letture frequenti.                                     |
| <b>LinkedList</b> | È basata su una lista collegata. Può essere utile quando servono molte inserzioni o rimozioni in punti intermedi della lista, ma l'accesso tramite indice è meno efficiente. |

Come per le altre collections, è buona pratica dichiarare la variabile usando l'interfaccia e non l'implementazione concreta.

```
1 List<String> nomi = new ArrayList<>();
```

In questo modo, se in futuro si volesse cambiare implementazione, il resto del codice potrebbe rimanere invariato.

```
1 List<String> nomi = new LinkedList<>();
```

### 10.8.3 Operazioni principali

L'interfaccia **List** eredita i metodi di **Collection**, come **add**, **remove**, **contains**, **size** e **iterator**. In più, aggiunge operazioni basate sulla posizione degli elementi.

| Metodo                                 | Significato                                               |
|----------------------------------------|-----------------------------------------------------------|
| <code>get(int index)</code>            | Restituisce l'elemento nella posizione indicata.          |
| <code>set(int index, E element)</code> | Sostituisce l'elemento nella posizione indicata.          |
| <code>add(int index, E element)</code> | Inserisce un elemento nella posizione indicata.           |
| <code>remove(int index)</code>         | Rimuove l'elemento nella posizione indicata.              |
| <code>indexOf(Object o)</code>         | Restituisce la prima posizione dell'elemento specificato. |
| <code>lastIndexOf(Object o)</code>     | Restituisce l'ultima posizione dell'elemento specificato. |

```

1 List<String> linguaggi = new ArrayList<>();
2
3 linguaggi.add("Java");
4 linguaggi.add("Python");
5 linguaggi.add("C++");
6
7 System.out.println(linguaggi.get(1));
8
9 linguaggi.set(1, "Kotlin");
10 linguaggi.add(1, "JavaScript");
11
12 System.out.println(linguaggi);
13 System.out.println(linguaggi.indexOf("Java"));

```

Il metodo `remove(Object o)`, ereditato da `Collection`, rimuove la prima occorrenza dell'elemento specificato. Il metodo `remove(int index)`, invece, rimuove l'elemento che si trova in una certa posizione.

#### 10.8.4 Concatenazione di liste

Il metodo `addAll` permette di aggiungere tutti gli elementi di una collection a una lista. Se usato senza indice, aggiunge gli elementi alla fine.

```

1 List<String> list1 = new ArrayList<>();
2 list1.add("A");
3 list1.add("B");
4
5 List<String> list2 = new ArrayList<>();
6 list2.add("C");
7 list2.add("D");
8
9 list1.addAll(list2);
10
11 System.out.println(list1);

```

Se non si vuole modificare la lista originale, si può creare prima una nuova lista tramite il costruttore di conversione.

```

1 List<String> list3 = new ArrayList<>(list1);
2 list3.addAll(list2);

```

#### 10.8.5 Arrays.asList

La classe `Arrays` fornisce il metodo statico `asList`, che permette di vedere un array come una lista.

```

1 String[] array = { "Java", "Python", "C++" };
2
3 List<String> lista = Arrays.asList(array);
4
5 System.out.println(lista);

```

La lista restituita da `Arrays.asList` è collegata all'array originale: modificare un elemento della lista modifica anche l'array, e viceversa. Tuttavia, non è una lista ridimensionabile, quindi non supporta operazioni come `add` o `remove`.

```

1 String[] array = { "Java", "Python", "C++" };
2
3 List<String> lista = Arrays.asList(array);
4
5 lista.set(0, "Kotlin"); // OK
6
7 lista.add("Go");        // Errore a runtime

```

Se serve una lista modificabile anche nella dimensione, si può creare una nuova `ArrayList`.

```

1 List<String> lista =
2     new ArrayList<>(Arrays.asList(array));

```

### 10.8.6 ListIterator

Una `List` può essere attraversata con un normale `Iterator`, ma fornisce anche un iteratore più specifico chiamato `ListIterator`.

A differenza di `Iterator`, `ListIterator` permette di muoversi sia in avanti sia all'indietro, modificare elementi durante l'iterazione e conoscere la posizione corrente.

```

1 List<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add("Luigi");
5 nomi.add("Anna");
6
7 ListIterator<String> it = nomi.listIterator();
8
9 while (it.hasNext()) {
10     String nome = it.next();
11
12     if (nome.equals("Luigi")) {
13         it.set("Luca");
14     }
15 }
16
17 System.out.println(nomi);

```

Per attraversare una lista al contrario, si può creare un `ListIterator` posizionato alla fine della lista.

```

1 ListIterator<String> it = nomi.listIterator(nomi.size());
2
3 while (it.hasPrevious()) {
4     String nome = it.previous();
5     System.out.println(nome);
6 }

```

### 10.8.7 subList

Il metodo `subList` restituisce una vista di una porzione della lista. Riceve un indice iniziale incluso e un indice finale escluso.

```

1 List<String> nomi = new ArrayList<>();
2
3 nomi.add("Mario");
4 nomi.add("Luigi");
5 nomi.add("Anna");
6 nomi.add("Giulia");
7
8 List<String> sottoLista = nomi.subList(1, 3);
9
10 System.out.println(sottoLista);

```

In questo esempio la sottolista contiene gli elementi in posizione 1 e 2. L'indice 3 non è incluso.

La lista restituita da `subList` non è una copia indipendente, ma una vista collegata alla lista originale. Le modifiche fatte sulla sottolista si riflettono sulla lista principale.

```

1 sottoLista.clear();
2
3 System.out.println(nomi);

```

Questo codice rimuove dalla lista originale gli elementi compresi nell'intervallo della sottolista.

Bisogna usare `subList` con attenzione: se si modifica direttamente la lista originale mentre si continua a usare la sottolista, il comportamento può diventare non valido. Per questo motivo, di solito `subList` si usa come oggetto temporaneo.

### 10.8.8 Algoritmi sulle liste

La classe `Collections` fornisce diversi algoritmi statici pensati soprattutto per lavorare con le liste.

| Metodo                                                                     | Significato                                                        |
|----------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>sort(List&lt;T&gt; list)</code>                                      | Ordina gli elementi della lista.                                   |
| <code>shuffle(List&lt;?&gt; list)</code>                                   | Mescola casualmente gli elementi.                                  |
| <code>reverse(List&lt;?&gt; list)</code>                                   | Inverte l'ordine degli elementi.                                   |
| <code>swap(List&lt;?&gt; list, int i, int j)</code>                        | Scambia due elementi in posizioni specifiche.                      |
| <code>fill(List&lt;? super T&gt; list, T obj)</code>                       | Sostituisce tutti gli elementi con lo stesso valore.               |
| <code>copy(List&lt;? super T&gt; dest, List&lt;? extends T&gt; src)</code> | Copia gli elementi da una lista sorgente a una lista destinazione. |
| <code>binarySearch(List&lt;? extends T&gt; list, T key)</code>             | Cerca un elemento in una lista ordinata usando ricerca binaria.    |

```
1 List<String> nomi = new ArrayList<>();
2
3 nomi.add("Luigi");
4 nomi.add("Anna");
5 nomi.add("Mario");
6
7 Collections.sort(nomi);
8 System.out.println(nomi);
9
10 Collections.reverse(nomi);
11 System.out.println(nomi);
12
13 Collections.shuffle(nomi);
14 System.out.println(nomi);
```

Questi metodi sono polimorfici: funzionano con qualunque implementazione di `List`, ad esempio `ArrayList` o `LinkedList`.

## 10.9 Queue

### 10.9.1 Concetto generale

Una *Queue* è una *Collection* pensata per contenere elementi prima che vengano processati. Il caso più comune è la coda FIFO, cioè *first-in-first-out*: il primo elemento inserito è il primo a essere rimosso.

Una coda ha quindi una testa, detta *head*, che rappresenta l'elemento che verrebbe rimosso dalla prossima operazione di estrazione. Nelle code FIFO, i nuovi elementi vengono inseriti in fondo alla coda e rimossi dalla testa.

```
1 Queue<String> coda = new LinkedList<>();
2
3 coda.add("Mario");
4 coda.add("Luigi");
5 coda.add("Anna");
6
7 System.out.println(coda.remove()); // Mario
8 System.out.println(coda.remove()); // Luigi
```

L'interfaccia *Queue* estende *Collection*, quindi eredita metodi come *add*, *remove*, *size*, *isEmpty* e *iterator*. In più, aggiunge metodi specifici per inserire, rimuovere e consultare l'elemento in testa alla coda.

### 10.9.2 Metodi principali

I metodi principali di *Queue* esistono in due forme: una forma lancia un'eccezione se l'operazione fallisce, l'altra restituisce un valore speciale, come *null* o *false*.

| Operazione    | Lancia eccezione | Restituisce valore speciale |
|---------------|------------------|-----------------------------|
| Inserimento   | <i>add(e)</i>    | <i>offer(e)</i>             |
| Rimozione     | <i>remove()</i>  | <i>poll()</i>               |
| Consultazione | <i>element()</i> | <i>peek()</i>               |

Il metodo *add(e)* inserisce un elemento nella coda. Se l'inserimento non è possibile, ad esempio in una coda con capacità limitata già piena, può lanciare un'eccezione. Il metodo *offer(e)* ha lo stesso scopo, ma in caso di fallimento restituisce *false*.

Il metodo *remove()* rimuove e restituisce l'elemento in testa alla coda. Se la coda è vuota, lancia una *NoSuchElementException*. Il metodo *poll()* rimuove e restituisce la testa della coda, ma se la coda è vuota restituisce *null*.

Il metodo *element()* restituisce l'elemento in testa senza rimuoverlo. Se la coda è vuota, lancia una *NoSuchElementException*. Il metodo *peek()* fa la stessa cosa, ma restituisce *null* se la coda è vuota.

```
1 Queue<String> coda = new LinkedList<>();
2
3 coda.offer("Java");
4 coda.offer("Python");
5
6 System.out.println(coda.peek()); // Java
7 System.out.println(coda.poll()); // Java
8 System.out.println(coda.poll()); // Python
9 System.out.println(coda.poll()); // null
```

In generale, quando si lavora con code che potrebbero essere vuote o piene, i metodi `offer`, `poll` e `peek` sono spesso più comodi perché evitano la gestione esplicita delle eccezioni.

### 10.9.3 Implementazioni principali

Una `Queue` non specifica necessariamente un ordinamento FIFO. Ogni implementazione deve indicare il criterio con cui ordina gli elementi.

| Implementazione            | Caratteristiche                                                                                                                                                  |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>LinkedList</code>    | Può essere usata come implementazione di <code>Queue</code> . Gli elementi vengono gestiti normalmente in ordine FIFO.                                           |
| <code>PriorityQueue</code> | Non segue l'ordine di inserimento, ma ordina gli elementi in base alla loro priorità, cioè secondo l'ordinamento naturale o secondo un <code>Comparator</code> . |
| <code>ArrayDeque</code>    | Implementazione efficiente per code e deque. È spesso preferibile a <code>LinkedList</code> quando serve una normale coda FIFO.                                  |

Una `PriorityQueue` rimuove per primo l'elemento con priorità più alta secondo il criterio di ordinamento, non necessariamente quello inserito per primo.

```
1 Queue<Integer> numeri = new PriorityQueue<>();
2
3 numeri.offer(30);
4 numeri.offer(10);
5 numeri.offer(20);
6
7 System.out.println(numeri.poll()); // 10
8 System.out.println(numeri.poll()); // 20
9 System.out.println(numeri.poll()); // 30
```

In questo esempio gli elementi vengono estratti in ordine crescente, perché gli interi hanno un ordinamento naturale.

### 10.9.4 Code limitate e valori null

Alcune implementazioni di `Queue` possono avere una capacità massima. Queste code sono dette *bounded queues*. Le implementazioni in `java.util.concurrent`, usate nella programmazione concorrente, possono essere limitate e possono attendere che si liberi spazio o che compaiano nuovi elementi.

Le implementazioni principali in `java.util`, invece, normalmente non hanno una capacità fissa imposta dalla struttura.

In generale, è meglio non inserire `null` in una `Queue`. Il motivo è che metodi come `poll()` e `peek()` usano `null` come valore speciale per indicare che la coda è vuota. Inserire `null` renderebbe quindi ambiguo il risultato.

## 10.10 Deque

### 10.10.1 Concetto generale

Una **Deque**, pronunciata spesso *deck*, è una *double-ended queue*, cioè una coda a doppia estremità. A differenza di una normale **Queue**, una **Deque** permette di inserire, rimuovere e consultare elementi sia all'inizio sia alla fine della struttura.

Questo rende **Deque** più generale sia di una coda sia di uno stack. Può infatti essere usata come coda FIFO, cioè *first-in-first-out*, oppure come stack LIFO, cioè *last-in-first-out*.

```
1 Deque<String> deque = new ArrayDeque<>();
2
3 deque.addFirst("B");
4 deque.addFirst("A");
5 deque.addLast("C");
6
7 System.out.println(deque);
```

In questo esempio "A" e "B" vengono inseriti all'inizio, mentre "C" viene inserito alla fine.

Le implementazioni più comuni di **Deque** sono **ArrayDeque** e **LinkedList**. In generale, **ArrayDeque** è spesso preferibile quando serve una struttura efficiente da usare come coda o stack.

### 10.10.2 Metodi principali

I metodi di **Deque** sono divisi in operazioni di inserimento, rimozione e consultazione. Per ogni operazione esistono metodi che lavorano sulla prima estremità e metodi che lavorano sull'ultima estremità.

| Operazione    | Inizio della Deque                                     | Fine della Deque                                     |
|---------------|--------------------------------------------------------|------------------------------------------------------|
| Inserimento   | <code>addFirst(e)</code><br><code>offerFirst(e)</code> | <code>addLast(e)</code><br><code>offerLast(e)</code> |
| Rimozione     | <code>removeFirst()</code><br><code>pollFirst()</code> | <code>removeLast()</code><br><code>pollLast()</code> |
| Consultazione | <code>getFirst()</code><br><code>peekFirst()</code>    | <code>getLast()</code><br><code>peekLast()</code>    |

I metodi che iniziano con `add`, `remove` e `get` lanciano un'eccezione se l'operazione non può essere completata. I metodi che iniziano con `offer`, `poll` e `peek`, invece, restituiscono un valore speciale, come `false` o `null`.

```
1 Deque<String> deque = new ArrayDeque<>();
2
3 deque.offerFirst("Java");
4 deque.offerLast("Python");
5
6 System.out.println(deque.peekFirst()); // Java
7 System.out.println(deque.peekLast()); // Python
8
9 System.out.println(deque.pollFirst()); // Java
10 System.out.println(deque.pollLast()); // Python
11 System.out.println(deque.pollFirst()); // null
```

Quando la struttura potrebbe essere vuota o avere capacità limitata, i metodi `offer`, `poll` e `peek` sono spesso più comodi perché evitano la gestione esplicita delle eccezioni.

### 10.10.3 Uso come coda

Una `Deque` può essere usata come una normale coda FIFO. In questo caso gli elementi vengono inseriti alla fine e rimossi dall'inizio.

```
1 Deque<String> queue = new ArrayDeque<>();
2
3 queue.offerLast("Primo");
4 queue.offerLast("Secondo");
5 queue.offerLast("Terzo");
6
7 System.out.println(queue.pollFirst()); // Primo
8 System.out.println(queue.pollFirst()); // Secondo
9 System.out.println(queue.pollFirst()); // Terzo
```

Questo comportamento è equivalente a quello di una coda: il primo elemento inserito è il primo a essere rimosso.

### 10.10.4 Uso come stack

Una `Deque` può essere usata anche come stack LIFO. In questo caso gli elementi vengono inseriti e rimossi dalla stessa estremità.

```
1 Deque<String> stack = new ArrayDeque<>();
2
3 stack.push("Primo");
4 stack.push("Secondo");
5 stack.push("Terzo");
6
7 System.out.println(stack.pop()); // Terzo
8 System.out.println(stack.pop()); // Secondo
9 System.out.println(stack.pop()); // Primo
```

I metodi `push` e `pop` rendono l'uso della `Deque` simile a quello di uno stack. In Java moderno, quando serve uno stack, è preferibile usare `ArrayDeque` invece della vecchia classe `Stack`.

### 10.10.5 Rimozione di occorrenze

Oltre ai metodi principali, `Deque` fornisce metodi per rimuovere occorrenze specifiche di un elemento.

```
1 Deque<String> deque = new ArrayDeque<>();
2
3 deque.addLast("A");
4 deque.addLast("B");
5 deque.addLast("A");
6 deque.addLast("C");
7
8 deque.removeFirstOccurrence("A");
9 System.out.println(deque);
10
11 deque.removeLastOccurrence("A");
12 System.out.println(deque);
```

Il metodo `removeFirstOccurrence` rimuove la prima occorrenza dell'elemento specificato. Il metodo `removeLastOccurrence` rimuove invece l'ultima occorrenza. Entrambi restituiscono `true` se la struttura è stata modificata.

## 10.11 Map

### 10.11.1 Concetto generale

Una **Map** è una struttura che associa chiavi a valori. Ogni chiave può essere associata al massimo a un valore, quindi una **Map** non può contenere chiavi duplicate.

A differenza di **List**, **Set**, **Queue** e **Deque**, l'interfaccia **Map** non estende **Collection**. Questo perché una mappa non rappresenta semplicemente un gruppo di elementi, ma un insieme di associazioni chiave-valore.

```
1 Map<String, Integer> eta = new HashMap<>();
2
3 eta.put("Mario", 20);
4 eta.put("Luigi", 22);
5 eta.put("Anna", 19);
6
7 System.out.println(eta.get("Mario"));
```

In questo esempio le chiavi sono stringhe, mentre i valori sono interi. La chiave "Mario" è associata al valore 20.

Se si inserisce un nuovo valore usando una chiave già presente, il valore precedente viene sostituito.

```
1 Map<String, Integer> eta = new HashMap<>();
2
3 eta.put("Mario", 20);
4 eta.put("Mario", 21);
5
6 System.out.println(eta.get("Mario")); // 21
```

### 10.11.2 Implementazioni principali

Le implementazioni generiche principali di **Map** sono **HashMap**, **TreeMap** e **LinkedHashMap**. Il loro comportamento è analogo a quello di **HashSet**, **TreeSet** e **LinkedHashSet**.

| Implementazione      | Caratteristiche                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------|
| <b>HashMap</b>       | È generalmente l'implementazione più usata ed efficiente. Non garantisce alcun ordine sulle chiavi.         |
| <b>TreeMap</b>       | Mantiene le chiavi ordinate secondo il loro ordinamento naturale o secondo un <b>Comparator</b> .           |
| <b>LinkedHashMap</b> | Mantiene l'ordine di inserimento delle chiavi. Ha un costo leggermente maggiore rispetto a <b>HashMap</b> . |

Come per le altre strutture del framework, è buona pratica dichiarare la variabile usando l'interfaccia.

```
1 Map<String, Integer> frequenze = new HashMap<>();
```

In questo modo si può cambiare implementazione senza modificare il resto del codice.

```
1 Map<String, Integer> frequenze = new TreeMap<>();
```

Con **TreeMap**, ad esempio, le chiavi vengono mantenute ordinate.

### 10.11.3 Operazioni principali

L'interfaccia `Map` fornisce metodi per inserire, leggere, rimuovere e controllare associazioni chiave-valore.

| Metodo                                   | Significato                                                                                        |
|------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>put(K key, V value)</code>         | Inserisce o aggiorna l'associazione tra una chiave e un valore.                                    |
| <code>get(Object key)</code>             | Restituisce il valore associato alla chiave, oppure <code>null</code> se la chiave non è presente. |
| <code>remove(Object key)</code>          | Rimuove l'associazione relativa alla chiave specificata.                                           |
| <code>containsKey(Object key)</code>     | Verifica se la mappa contiene una certa chiave.                                                    |
| <code>containsValue(Object value)</code> | Verifica se la mappa contiene un certo valore.                                                     |
| <code>size()</code>                      | Restituisce il numero di associazioni presenti.                                                    |
| <code>isEmpty()</code>                   | Restituisce <code>true</code> se la mappa è vuota.                                                 |

```
1 Map<String, Integer> voti = new HashMap<>();
2
3 voti.put("Mario", 28);
4 voti.put("Luigi", 25);
5 voti.put("Anna", 30);
6
7 System.out.println(voti.containsKey("Anna"));
8 System.out.println(voti.containsValue(18));
9
10 voti.remove("Luigi");
11
12 System.out.println(voti.size());
```

Due oggetti `Map` sono considerati uguali se contengono le stesse associazioni chiave-valore, indipendentemente dalla loro implementazione concreta.

### 10.11.4 Esempio: tabella delle frequenze

Un uso tipico di `Map` è contare quante volte compare un elemento. In questo caso, la chiave rappresenta l'elemento osservato e il valore rappresenta la sua frequenza.

```

1 import java.util.*;
2
3 public class Freq {
4     public static void main(String[] args) {
5         Map<String, Integer> frequenze = new HashMap<>();
6
7         for (String parola : args) {
8             Integer freq = frequenze.get(parola);
9
10            if (freq == null) {
11                frequenze.put(parola, 1);
12            } else {
13                frequenze.put(parola, freq + 1);
14            }
15        }
16
17        System.out.println(frequenze.size()
18            + " parole distinte:");
19        System.out.println(frequenze);
20    }
21 }

```

Se una parola non è ancora presente nella mappa, viene inserita con frequenza 1. Se invece è già presente, il valore associato viene incrementato.

Lo stesso codice può essere reso più compatto usando `getOrDefault`.

```

1 Map<String, Integer> frequenze = new HashMap<>();
2
3 for (String parola : args) {
4     int freq = frequenze.getOrDefault(parola, 0);
5     frequenze.put(parola, freq + 1);
6 }

```

### 10.11.5 Operazioni bulk

Le operazioni bulk principali di `Map` sono `putAll` e `clear`. Il metodo `putAll` copia tutte le associazioni da una mappa a un'altra, mentre `clear` rimuove tutte le associazioni.

```

1 Map<String, Integer> m1 = new HashMap<>();
2
3 m1.put("A", 1);
4 m1.put("B", 2);
5
6 Map<String, Integer> m2 = new HashMap<>();
7
8 m2.put("C", 3);
9 m2.put("D", 4);
10
11 m1.putAll(m2);
12
13 System.out.println(m1);

```

Come per le collection, le implementazioni generiche di `Map` forniscono un costruttore di conversione che permette di creare una nuova mappa contenente le stesse associazioni di un'altra.

```
1 Map<String, Integer> copia = new HashMap<>(m1);
```

Questo è utile quando si vuole modificare una copia senza alterare la mappa originale.

### 10.11.6 Collection views

Una `Map` può essere vista come un insieme di chiavi, una collezione di valori oppure un insieme di coppie chiave-valore. Queste viste permettono di attraversare il contenuto della mappa.

| Metodo                  | Significato                                                                                    |
|-------------------------|------------------------------------------------------------------------------------------------|
| <code>keySet()</code>   | Restituisce un <code>Set</code> contenente tutte le chiavi della mappa.                        |
| <code>values()</code>   | Restituisce una <code>Collection</code> contenente tutti i valori della mappa.                 |
| <code>entrySet()</code> | Restituisce un <code>Set</code> di oggetti <code>Map.Entry</code> , cioè coppie chiave-valore. |

Per iterare sulle sole chiavi si usa `keySet()`.

```
1 Map<String, Integer> voti = new HashMap<>();
2
3 voti.put("Mario", 28);
4 voti.put("Luigi", 25);
5 voti.put("Anna", 30);
6
7 for (String nome : voti.keySet()) {
8     System.out.println(nome);
9 }
```

Per iterare sui valori si usa `values()`.

```
1 for (Integer voto : voti.values()) {
2     System.out.println(voto);
3 }
```

Per iterare sulle coppie chiave-valore si usa `entrySet()`.

```
1 for (Map.Entry<String, Integer> entry : voti.entrySet()) {
2     System.out.println(entry.getKey()
3         + ": " + entry.getValue());
4 }
```

`entrySet()` è spesso il modo migliore per attraversare una mappa quando servono sia la chiave sia il valore.

### 10.11.7 Modifica durante l'iterazione

Le viste restituite da `keySet`, `values` ed `entrySet` sono collegate alla mappa originale. Questo significa che rimuovere un elemento tramite una vista rimuove anche la relativa associazione dalla mappa.

Il modo sicuro per rimuovere elementi durante l'iterazione è usare l'`Iterator` della vista.

```

1 Map<String, Integer> voti = new HashMap<>();
2
3 voti.put("Mario", 28);
4 voti.put("Luigi", 17);
5 voti.put("Anna", 30);
6
7 Iterator<Map.Entry<String, Integer>> it = voti.entrySet().iterator();
8
9 while (it.hasNext()) {
10     Map.Entry<String, Integer> entry = it.next();
11
12     if (entry.getValue() < 18) {
13         it.remove();
14     }
15 }
16
17 System.out.println(voti);

```

In questo esempio vengono rimossi dalla mappa gli studenti con voto insufficiente.

Se si attraversa `entrySet`, è anche possibile modificare il valore associato a una chiave tramite `setValue`.

```

1 for (Map.Entry<String, Integer> entry : voti.entrySet()) {
2     if (entry.getKey().equals("Mario")) {
3         entry.setValue(30);
4     }
5 }

```

Durante l'iterazione non bisogna modificare direttamente la mappa con `put` o `remove`, perché il comportamento può diventare non valido.

### 10.11.8 Map come multimap

Il Java Collections Framework non fornisce un'interfaccia specifica per le *multimap*. Una *multimap* è una struttura simile a una mappa, ma in cui una chiave può essere associata a più valori.

Questo comportamento può essere ottenuto usando una `Map` in cui i valori sono liste.

```

1 Map<String, List<String>> studentiPerCorso = new HashMap<>();
2
3 studentiPerCorso.put("Programmazione", new ArrayList<>());
4
5 studentiPerCorso.get("Programmazione").add("Mario");
6 studentiPerCorso.get("Programmazione").add("Luigi");

```

Un modo più sicuro per aggiungere valori è controllare se la lista associata alla chiave esiste già.

```
1 Map<String, List<String>> studentiPerCorso = new HashMap<>();
2
3 String corso = "Programmazione";
4 String studente = "Mario";
5
6 List<String> studenti = studentiPerCorso.get(corso);
7
8 if (studenti == null) {
9     studenti = new ArrayList<>();
10    studentiPerCorso.put(corso, studenti);
11 }
12
13 studenti.add(studente);
```

Così una singola chiave può essere collegata a più valori.

## 10.12 Ordinamento, SortedSet e SortedMap

### 10.12.1 Ordinamento naturale e Comparable

Alcune collections hanno bisogno di ordinare automaticamente i propri elementi. Ad esempio, una `List` può essere ordinata con il metodo `Collections.sort`, mentre strutture come `TreeSet` e `TreeMap` mantengono gli elementi ordinati durante l'inserimento.

```
1 List<String> parole = new ArrayList<>();
2
3 parole.add("banana");
4 parole.add("apple");
5 parole.add("orange");
6
7 Collections.sort(parole);
8
9 System.out.println(parole);
```

In questo esempio le stringhe vengono ordinate alfabeticamente. Questo è possibile perché `String` implementa l'interfaccia `Comparable`. Una classe che implementa `Comparable` definisce un ordinamento naturale per i propri oggetti.

```
1 public interface Comparable<T> {
2     int compareTo(T o);
3 }
```

Il metodo `compareTo` confronta l'oggetto corrente con un altro oggetto dello stesso tipo. Restituisce un valore negativo se l'oggetto corrente viene prima, 0 se i due oggetti sono considerati equivalenti nell'ordinamento, e un valore positivo se l'oggetto corrente viene dopo.

Molte classi della piattaforma Java implementano già `Comparable`. Ad esempio, `String` usa un ordinamento lessicografico, i tipi numerici wrapper come `Integer` e `Double` usano l'ordinamento numerico, mentre `Date` usa l'ordinamento cronologico.

```

1 public class Persona implements Comparable<Persona> {
2     private String cognome;
3     private String nome;
4
5     public Persona(String cognome, String nome) {
6         this.cognome = cognome;
7         this.nome = nome;
8     }
9
10    @Override
11    public int compareTo(Persona altra) {
12        int confrontoCognome =
13            this.cognome.compareTo(altra.cognome);
14
15        if (confrontoCognome != 0) {
16            return confrontoCognome;
17        }
18
19        return this.nome.compareTo(altra.nome);
20    }
21
22    @Override
23    public String toString() {
24        return cognome + " " + nome;
25    }
26 }

```

In questo esempio, gli oggetti **Persona** vengono ordinati prima per cognome e poi, in caso di cognome uguale, per nome.

### 10.12.2 Comparator

A volte l'ordinamento naturale non esiste oppure non è quello desiderato. In questi casi si usa un **Comparator**, cioè un oggetto che definisce un criterio di ordinamento esterno alla classe degli oggetti da ordinare.

```

1 public interface Comparator<T> {
2     int compare(T o1, T o2);
3 }

```

Il metodo **compare** funziona in modo analogo a **compareTo**, ma riceve entrambi gli oggetti da confrontare come parametri.

```

1 Comparator<Persona> ordinePerNome =
2     new Comparator<Persona>() {
3         @Override
4         public int compare(Persona p1, Persona p2) {
5             return p1.getNome().compareTo(p2.getNome());
6         }
7     };

```

Con Java moderno lo stesso ordinamento può essere espresso in modo più compatto usando una lambda expression.

```

1 Comparator<Persona> ordinePerNome = (p1, p2) -> p1.getNome().compareTo(p2.
    getNome());

```

Un `Comparator` può essere passato a metodi come `Collections.sort` oppure a strutture ordinate come `TreeSet` e `TreeMap`.

```
1 List<Persona> persone = new ArrayList<>();
2
3 persone.add(new Persona("Rossi", "Mario"));
4 persone.add(new Persona("Bianchi", "Anna"));
5
6 persone.sort(ordinePerNome);
```

Quando un `Comparator` viene usato in strutture come `TreeSet`, è importante che sia coerente con il concetto di uguaglianza. Se due oggetti diversi vengono considerati uguali dal comparatore, un `TreeSet` potrebbe conservarne solo uno, perché per il set sarebbero duplicati.

### 10.12.3 SortedSet

Un `SortedSet` è un `Set` che mantiene i propri elementi in ordine crescente. L'ordine può essere quello naturale degli elementi, definito tramite `Comparable`, oppure un ordine stabilito da un `Comparator` fornito al momento della creazione.

L'implementazione più comune di `SortedSet` è `TreeSet`.

```
1 SortedSet<String> parole = new TreeSet<>();
2
3 parole.add("banana");
4 parole.add("apple");
5 parole.add("orange");
6
7 System.out.println(parole);
```

L'output sarà ordinato alfabeticamente [apple, banana, orange].

Poiché `SortedSet` estende `Set`, non ammette duplicati. In più, garantisce che l'iterazione sugli elementi avvenga secondo l'ordine del set.

### 10.12.4 Operazioni principali di SortedSet

L'interfaccia `SortedSet` aggiunge metodi per ottenere viste su intervalli ordinati, accedere agli estremi e conoscere il criterio di ordinamento.

| Metodo                        | Significato                                                                                                                       |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>subSet(from, to)</code> | Restituisce una vista degli elementi compresi tra <code>from</code> , incluso, e <code>to</code> , escluso.                       |
| <code>headSet(to)</code>      | Restituisce gli elementi minori di <code>to</code> .                                                                              |
| <code>tailSet(from)</code>    | Restituisce gli elementi maggiori o uguali a <code>from</code> .                                                                  |
| <code>first()</code>          | Restituisce il primo elemento del set ordinato.                                                                                   |
| <code>last()</code>           | Restituisce l'ultimo elemento del set ordinato.                                                                                   |
| <code>comparator()</code>     | Restituisce il <code>Comparator</code> usato per ordinare il set, oppure <code>null</code> se viene usato l'ordinamento naturale. |

```

1 SortedSet<String> parole = new TreeSet<>();
2
3 parole.add("apple");
4 parole.add("banana");
5 parole.add("doorbell");
6 parole.add("orange");
7 parole.add("pickle");
8
9 System.out.println(parole.first());
10 System.out.println(parole.last());
11
12 SortedSet<String> intervallo = parole.subSet("banana", "pickle");
13
14 System.out.println(intervallo);

```

Le viste restituite da `subSet`, `headSet` e `tailSet` sono collegate al set originale. Modificare la vista modifica anche il set principale.

```

1 parole.subSet("banana", "pickle").clear();
2
3 System.out.println(parole);

```

Questa operazione rimuove dal set principale tutti gli elementi compresi nell'intervallo indicato.

### 10.12.5 SortedMap

Una `SortedMap` è una `Map` che mantiene le proprie associazioni ordinate in base alle chiavi. Anche in questo caso l'ordine può essere quello naturale delle chiavi oppure quello stabilito da un `Comparator`.

L'implementazione più comune di `SortedMap` è `TreeMap`.

```

1 SortedMap<String, Integer> frequenze = new TreeMap<>();
2
3 frequenze.put("banana", 2);
4 frequenze.put("apple", 5);
5 frequenze.put("orange", 1);
6
7 System.out.println(frequenze);

```

L'output mostra le associazioni ordinate secondo le chiavi: `apple=5`, `banana=2`, `orange=1`.

Le operazioni ereditate da `Map`, come `put`, `get`, `remove`, `containsKey` e `entrySet`, funzionano come nelle mappe normali. La differenza è che le viste della mappa, come `keySet`, `values` ed `entrySet`, vengono attraversate secondo l'ordine delle chiavi.

### 10.12.6 Operazioni principali di SortedMap

L'interfaccia `SortedMap` è l'equivalente di `SortedSet` per le mappe. Fornisce metodi per lavorare su intervalli di chiavi, accedere alla prima e all'ultima chiave e conoscere il criterio di ordinamento.

| Metodo                              | Significato                                                                                                                          |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>subMap(fromKey, toKey)</code> | Restituisce una vista della parte di mappa compresa tra <b>fromKey</b> , inclusa, e <b>toKey</b> , esclusa.                          |
| <code>headMap(toKey)</code>         | Restituisce la parte di mappa con chiavi minori di <b>toKey</b> .                                                                    |
| <code>tailMap(fromKey)</code>       | Restituisce la parte di mappa con chiavi maggiori o uguali a <b>fromKey</b> .                                                        |
| <code>firstKey()</code>             | Restituisce la prima chiave della mappa ordinata.                                                                                    |
| <code>lastKey()</code>              | Restituisce l'ultima chiave della mappa ordinata.                                                                                    |
| <code>comparator()</code>           | Restituisce il <code>Comparator</code> usato per ordinare le chiavi, oppure <code>null</code> se viene usato l'ordinamento naturale. |

```

1 SortedMap<String, Integer> voti = new TreeMap<>();
2
3 voti.put("Anna", 30);
4 voti.put("Luigi", 25);
5 voti.put("Mario", 28);
6 voti.put("Paolo", 21);
7
8 System.out.println(voti.firstKey());
9 System.out.println(voti.lastKey());
10
11 SortedMap<String, Integer> intervallo = voti.subMap("Luigi", "Paolo");
12
13 System.out.println(intervallo);

```

Anche le viste restituite da `subMap`, `headMap` e `tailMap` sono collegate alla mappa originale. Modificare la vista significa modificare anche la `SortedMap` principale.

```

1 voti.headMap("Mario").clear();
2
3 System.out.println(voti);

```

Questo codice rimuove dalla mappa tutte le associazioni con chiave precedente a "Mario".

### 10.12.7 TreeSet e TreeMap con Comparator

Sia `TreeSet` sia `TreeMap` possono ricevere un `Comparator` nel costruttore. Questo permette di usare un criterio di ordinamento diverso da quello naturale.

```

1 Comparator<String> ordineDecrescente = (s1, s2) -> s2.compareTo(s1);
2
3 SortedSet<String> parole = new TreeSet<>(ordineDecrescente);
4
5 parole.add("apple");
6 parole.add("banana");
7 parole.add("orange");
8
9 System.out.println(parole);

```

L'output sarà ordinato in senso decrescente.

```

1 [orange, banana, apple]

```

Lo stesso principio vale per `TreeMap`: il `Comparator` ordina le chiavi, non i valori.

```
1 SortedMap<String, Integer> mappa = new TreeMap<>(ordineDecrescente);
2
3 mappa.put("apple", 5);
4 mappa.put("banana", 2);
5 mappa.put("orange", 1);
6
7 System.out.println(mappa);
```

La cosa fondamentale da ricordare è che l'ordinamento influenza non solo l'ordine di attraversamento, ma anche il modo in cui strutture come **TreeSet** stabiliscono se due elementi sono duplicati.

## 11 Exceptions

Un'eccezione è un evento, avvenuto durante l'esecuzione di un programma, che interrompe il normale flusso delle istruzioni del programma.

Quando un errore avviene in un metodo, quest'ultimo crea un oggetto e lo fornisce al runtime system. L'oggetto, detto **exception object**, contiene informazioni riguardanti l'errore, incluso il tipo e lo stato del programma quando è avvenuto l'errore. Creare un oggetto e fornirlo al runtime system si dice **throwing an exception**.

Dopo che viene fornito l'errore al runtime system, questo cerca di trovare qualcosa per gestirlo. In ordine la ricerca avviene così, risalendo se non si trova un **handler**:

1. Metodo dove si è verificato l'errore;
2. Metodo chiamante del metodo che ha generato l'errore;
3. Viene risalita al contrario la catena dei chiamanti;
4. Si raggiunge eventualmente il metodo **main**;
5. Il programma termina.

### 11.1 Catch or Specify

I codici Java validi devono rispettare il principio *Catch or Specify*: qualsiasi codice che potrebbe lanciare un'eccezione deve essere racchiuso da uno dei seguenti:

- Uno statement **try** che cattura l'eccezione. Il **try** deve contenere un **handler** per l'eccezione;
- Un metodo che specifica che potrebbe lanciare l'eccezione. Il metodo deve provvedere una clausola **throws**.

### 11.2 Tipi di eccezioni

| Tipo                | Significato                                                                | Obbligo di gestione                                                   | Esempio                        |
|---------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------|--------------------------------|
| Checked exception   | Situazione prevedibile da cui un programma ben scritto dovrebbe recuperare | Sì, va catturata con <b>catch</b> oppure dichiarata con <b>throws</b> | <b>FileNotFoundException</b>   |
| Error               | Problema esterno o grave, spesso non recuperabile dall'applicazione        | No                                                                    | <b>IOException</b>             |
| Runtime exception   | Errore interno al programma, solitamente causato da bug di programmazione  | No                                                                    | <b>NullPointerException</b>    |
| Unchecked exception | Categoria che comprende <b>Error</b> e <b>RuntimeException</b>             | No                                                                    | <b>Error, RuntimeException</b> |

## 11.3 Catturare e gestire le eccezioni

### 11.3.1 Costrutto try-catch-finally

Il blocco **try** contiene il codice che potrebbe generare un'eccezione. Se durante l'esecuzione del codice dentro il **try** viene lanciata un'eccezione, il runtime cerca un gestore associato a quel blocco.

```
1 private List<Integer> list;
2 private static final int SIZE = 10;
3
4 public void writeList() {
5     PrintWriter out = null;
6
7     try {
8         System.out.println("Entered try statement");
9
10        FileWriter f = new FileWriter("OutFile.txt");
11        out = new PrintWriter(f);
12
13        for (int i = 0; i < SIZE; i++) {
14            out.println("Value at: " + i + " = " + list.get(i));
15        }
16    }
17    catch and finally blocks ...
18 }
```

Se il codice è corto o può generare una sola eccezione di solito si usa un solo **try**, altrimenti si suddivide in blocchi più brevi per gestire le singole eccezioni.

Un blocco **catch** associa un gestore di eccezioni ad un blocco **try**. Ogni **catch** gestisce un tipo specifico di eccezione. Il tipo indicato, **ExceptionType**, deve essere una classe che eredita da **Throwable**:

```
1 try {
2     // codice che può lanciare eccezioni
3 } catch (IndexOutOfBoundsException e) {
4     System.err.println("IndexOutOfBoundsException: " + e.getMessage());
5 } catch (IOException e) {
6     System.err.println("Caught IOException: " + e.getMessage());
7 }
```

Da Java SE 7, un singolo blocco **catch** può gestire più tipi di eccezione usando il simbolo **|**:

```
1 catch (IOException | SQLException ex) {
2     logger.log(ex);
3     throw ex;
4 }
```

Il blocco **finally** contiene codice che viene eseguito sempre quando il blocco **try** termina, a prescindere di come termina. Potrebbe terminare normalmente, a causa di un'eccezione non gestita o non prevista oppure a causa di istruzioni come **return**, **break** o **continue**.

In alcuni casi particolari, ad esempio se la JVM termina mentre il codice del **try** o del **catch** è ancora in esecuzione, il blocco **finally** potrebbe non venire mai eseguito.

### 11.3.2 Statement try-with-resources

Il `try-with-resources` è una forma particolare di `try` usata per gestire automaticamente la chiusura delle risorse, oggetti che devono essere chiusi dopo l'uso. Esempi di risorse sono `FileReader`, `BufferedReader`, `BufferedWriter`, `ZipFile` o `Statement`.

Una risorsa può essere usata in un `try-with-resources` se implementa l'interfaccia `java.lang.AutoCloseable` oppure `java.lang.Closeable` (che estende `AutoCloseable`).

La sintassi generale è la seguente:

```
1 static String readFirstLineFromFile(String path) throws IOException {
2     try (FileReader fr = new FileReader(path);
3         BufferedReader br = new BufferedReader(fr)) {
4         return br.readLine();
5     }
6 }
```

La risorsa viene dichiarata tra parentesi subito dopo la parola chiave `try`; alla fine del blocco in cui viene dichiarata, la risorsa viene chiusa automaticamente, a prescindere dal modo in cui il blocco `try` è terminato.

### 11.4 Specificare le eccezioni lanciate da un metodo

Se un metodo non cattura e gestisce le eccezioni *checked* al suo interno, allora dovrà specificare che quel metodo potrebbe lanciare una o più eccezioni. Non è necessario invece esplicitare le eccezioni *unchecked*.

```
1 // Errore di compilazione
2 public void writeList() {
3     PrintWriter out = new PrintWriter(new FileWriter("OutFile.txt"));
4     for (int i = 0; i < SIZE; i++) {
5         out.println("Value at: " + i + " = " + list.get(i));
6     }
7     out.close();
8 }
9
10 // Corretto, non è necessario specificare IndexOutOfBoundsException
11 public void writeList() throws IOException {
12     PrintWriter out = new PrintWriter(new FileWriter("OutFile.txt"));
13     for (int i = 0; i < SIZE; i++) {
14         out.println("Value at: " + i + " = " + list.get(i));
15     }
16     out.close();
17 }
```

## 11.5 Lanciare eccezioni

Oltre alle eccezioni lanciabili durante l'esecuzione del programma, si potrebbe decidere di scrivere le nostre classi `Throwable` per lanciare eccezioni ad-hoc, oppure è possibile lanciare eccezioni in un qualsiasi momento tramite `throw`.

```
1 public Object pop() {
2     Object obj;
3
4     if (size == 0) {
5         throw new EmptyStackException();
6     }
7
8     obj = objectAt(size - 1);
9     setObjectAt(size - 1, null);
10    size--;
11    return obj;
12 }
```

## 11.6 Eccezioni concatenate

Di solito ad un'eccezione si risponde con un'altra eccezione: dato che è possibile risalire la pila di chiamate, può essere molto utile come strumento di debug.

```
1 // Metodi e costruttori di Throwable
2 Throwable getCause()
3 Throwable initCause(Throwable)
4 Throwable(String, Throwable)
5 Throwable(Throwable)
6
7 // Esempio di utilizzo
8 try {
9 } catch (IOException e) {
10     throw new SampleException("Other IOException", e);
11 }
```

## 11.7 Stack trace

Supponiamo ora che un gestore di eccezioni di livello superiore voglia visualizzare lo stack trace usando un formato personalizzato.

Uno stack trace descrive la sequenza di chiamate effettuate dal thread corrente fino al punto in cui si è verificata l'eccezione. In particolare, mostra le classi e i metodi coinvolti nella catena di chiamate.

Il codice seguente mostra come invocare `getStackTrace` sull'oggetto eccezione.

```
1 catch (Exception cause) {
2     StackTraceElement elements[] = cause.getStackTrace();
3     for (int i = 0, n = elements.length; i < n; i++) {
4         System.err.println(elements[i].getFileName()
5             + ":" + elements[i].getLineNumber() + ">> "
6             + elements[i].getMethodName() + "()");
7     }
8 }
```

## 11.8 Quando e come creare una eccezione custom

Un'eccezione personalizzata può essere comoda in diversi scenari:

- Le eccezioni standard non descrivono bene il problema;
- Chi usa il codice può capire se l'errore proviene dal proprio package;
- Conviene raggruppare eccezioni sotto una gerarchia comune;
- Gli utenti non devono dipendere da eccezioni definite altrove.

Esempio nel caso di eccezioni custom per una linked list: si potrebbe generare una gerarchia di eccezioni a partire da `LinkedListException`, per poi creare sottoclassi più specifiche, ad esempio `InvalidIndexException`, `EmptyListException` e `ObjectNotFoundException`. In questo modo è possibile gestirle tutte tramite un `catch` che controlla `LinkedListException`.

Solitamente le eccezioni custom estendono direttamente la classe `Exception`, ma nel caso di una gerarchia si possono estendere eccezioni personalizzate già dichiarate in precedenza.

Non è una buona pratica estendere `Error`, in quanto è dedicato a problemi gravi del sistema, spesso non recuperabili, riguardanti il funzionamento della JVM.

## 11.9 Unchecked exceptions

Esiste una categoria di eccezioni, le *unchecked exception*, per le quali il compilatore non ci obbliga a catturarle con `catch` o a dichiararle con `throws`.

Questo evita errori di compilazione e rende il codice più veloce da scrivere, ma può creare problemi a chi usa quelle classi, perché nasconde informazioni importanti sul comportamento dei metodi.

Chi chiama un metodo deve sapere quali eccezioni possono verificarsi, in modo da decidere come gestirle. Per questo Java obbliga i metodi a dichiarare le checked exceptions non gestite internamente.

Le runtime exceptions rappresentano di solito errori di programmazione, cioè problemi interni al codice, perciò non vanno sempre dichiarate dato che possono avvenire praticamente ovunque in un programma.

Ha senso lanciare una `RuntimeException` nel caso di metodi chiamati in modo scorretto:

```
1 public void setName(String name) {  
2     if (name == null) {  
3         throw new IllegalArgumentException("name cannot be null");  
4     }  
5  
6     this.name = name;  
7 }
```

Importante: non bisogna creare sottoclassi di `RuntimeException` solo per evitare di scrivere `throws` o gestire le eccezioni.

### 11.10 Vantaggi delle eccezioni

Le eccezioni permettono di gestire gli errori in modo più ordinato rispetto alle tecniche tradizionali basate su codici di errore. I principali vantaggi sono:

- **Separazione del codice di gestione degli errori dal codice ordinario:** le eccezioni permettono di mantenere il flusso principale del programma pulito e leggibile, spostando la gestione degli errori nei blocchi `catch`.
- **Propagazione degli errori lungo la call stack:** un'eccezione può risalire automaticamente la catena delle chiamate fino a raggiungere un metodo capace di gestirla, evitando che ogni metodo intermedio debba controllare e restituire codici di errore.
- **Raggruppamento e distinzione dei tipi di errore tramite la gerarchia delle classi:** poiché le eccezioni sono classi, possono essere organizzate in gerarchie: si possono gestire errori specifici oppure intere famiglie di errori usando una superclasse comune.

## 12 I/O

Nel corso ci si è concentrati su I/O **Stream**: uno stream rappresenta una fonte di input o una destinazione di output. Uno stream può supportare molti tipi di dati, alcuni semplicemente trasferendoli mentre alcuni possono anche manipolarli e trasformarli se necessario.

Non importa come funzionano internamente, tutti gli stream sono considerabili una sequenza di dati provenienti da una generica fonte di dati.

### 12.1 Byte Streams

I programmi usano gli stream di byte per l'input e l'output di byte da 8 bit. Tutte le classi di byte stream derivano da `InputStream` e `OutputStream`.

Come esempio prenderemo `FileInputStream` e `FileOutputStream`, esaminando un programma che copia un file un byte alla volta:

```
1 import java.io.FileInputStream;
2 import java.io.FileOutputStream;
3 import java.io.IOException;
4
5 public class CopyBytes {
6     public static void main(String[] args) throws IOException {
7
8         FileInputStream in = null;
9         FileOutputStream out = null;
10
11         try {
12             in = new FileInputStream("xanadu.txt");
13             out = new FileOutputStream("outagain.txt");
14             int c;
15
16             while ((c = in.read()) != -1) {
17                 out.write(c);
18             }
19         } finally {
20             if (in != null) {
21                 in.close();
22             }
23             if (out != null) {
24                 out.close();
25             }
26         }
27     }
28 }
```

Il programma `CopyBytes` spende la maggior parte del tempo in un loop che legge lo stream di input e scrive lo stream di output, un byte alla volta. Una cosa da ricordare sempre è **chiudere lo stream quando non è più necessario**.

In realtà l'approccio precedente andrebbe evitato, poichè il file che stiamo analizzando contiene caratteri e non byte, però l'esempio serviva a mostrare un possibile caso d'uso. Lo stream di byte grezzi andrebbe usato solo per I/O estremamente primitivo. Occorre parlarne per primi poichè tutti gli stream si basano su questi.

## 12.2 Character Streams

Java rappresenta internamente i caratteri utilizzando **Unicode**. I **character streams** si occupano automaticamente di convertire i caratteri tra la rappresentazione Unicode interna e la codifica di caratteri utilizzata dal sistema operativo (ad esempio UTF-8, Windows-1252, ecc.).

Questa conversione è trasparente al programmatore: leggendo o scrivendo testo tramite classi come **Reader** e **Writer**, non è necessario gestire manualmente la codifica dei caratteri.

I principali vantaggi sono:

- gestione automatica della conversione tra Unicode e la codifica locale;
- codice indipendente dalla piattaforma;
- supporto all'internazionalizzazione senza modifiche sostanziali al programma.

Per lavorare con dati testuali è quindi consigliato utilizzare i **character streams**, mentre i **byte streams** sono più adatti alla gestione di dati binari (immagini, file audio, PDF, ecc.).

```
1 import java.io.FileReader;
2 import java.io.FileWriter;
3 import java.io.IOException;
4
5 public class CopyCharacters {
6     public static void main(String[] args) throws IOException {
7
8         FileReader inputStream = null;
9         FileWriter outputStream = null;
10
11         try {
12             inputStream = new FileReader("xanadu.txt");
13             outputStream = new FileWriter("characteroutput.txt");
14
15             int c;
16             while ((c = inputStream.read()) != -1) {
17                 outputStream.write(c);
18             }
19         } finally {
20             if (inputStream != null) {
21                 inputStream.close();
22             }
23             if (outputStream != null) {
24                 outputStream.close();
25             }
26         }
27     }
28 }
```

La classe **FileReader** svolge un ruolo analogo a **FileInputStream**, mentre **FileWriter** è l'equivalente di **FileOutputStream**. La differenza principale è che queste classi lavorano con **caratteri** anziché con **byte**.

Come per i byte streams, i metodi **read()** e **write()** utilizzano un valore di tipo **int**. Tuttavia:

- nei **byte streams**, l'**int** contiene un byte negli ultimi 8 bit;
- nei **character streams**, l'**int** contiene un carattere Unicode negli ultimi 16 bit.

L'utilizzo di `int` permette inoltre al metodo `read()` di restituire il valore speciale `-1` per indicare la fine del flusso (EOF, *End Of File*).

### 12.2.1 Line-Oriented I/O

L'I/O basato sui caratteri spesso non lavora un carattere alla volta, ma su unità più grandi. Una delle unità più comuni è la **riga**, cioè una sequenza di caratteri terminata da un terminatore di riga.

I principali terminatori di riga sono:

- `\r\n`, tipico di Windows;
- `\r`, usato nei vecchi sistemi Mac;
- `\n`, tipico di Unix, Linux e macOS.

Il supporto a terminatori diversi permette a un programma Java di leggere file di testo creati su sistemi operativi differenti.

Per gestire l'I/O una riga alla volta si usano principalmente:

- `BufferedReader`, che fornisce il metodo `readLine()`;
- `PrintWriter`, che fornisce il metodo `println()`.

```
1 import java.io.FileReader;
2 import java.io.FileWriter;
3 import java.io.BufferedReader;
4 import java.io.PrintWriter;
5 import java.io.IOException;
6
7 public class CopyLines {
8     public static void main(String[] args) throws IOException {
9
10         BufferedReader inputStream = null;
11         PrintWriter outputStream = null;
12
13         try {
14             inputStream = new BufferedReader(new FileReader("xanadu.txt"));
15             outputStream = new PrintWriter(new FileWriter("characteroutput.txt"))
16         );
17
18         String l;
19         while ((l = inputStream.readLine()) != null) {
20             outputStream.println(l);
21         }
22     } finally {
23         if (inputStream != null) {
24             inputStream.close();
25         }
26         if (outputStream != null) {
27             outputStream.close();
28         }
29     }
30 }
```

Il metodo `readLine()` legge una riga alla volta e restituisce `null` quando raggiunge la fine del file. Il metodo `println()` scrive una riga aggiungendo automaticamente il terminatore di riga.

## 12.3 Buffered Streams

Negli esempi precedenti sono stati utilizzati **stream non bufferizzati** (*unbuffered streams*), nei quali ogni operazione di lettura o scrittura viene inoltrata direttamente al sistema operativo. Questo approccio è poco efficiente, poiché ogni operazione può comportare un accesso al disco, alla rete o ad altre risorse costose.

Per migliorare le prestazioni, Java mette a disposizione i **buffered streams**. Questi utilizzano un'area di memoria temporanea (*buffer*) per accumulare i dati:

- uno stream di input legge dal sistema operativo solo quando il buffer è vuoto;
- uno stream di output scrive sul sistema operativo solo quando il buffer è pieno.

In questo modo si riduce notevolmente il numero di accessi al sistema operativo, migliorando l'efficienza delle operazioni di I/O.

Uno stream bufferizzato viene ottenuto **wrappando** uno stream non bufferizzato:

```
1 BufferedReader input =  
2     new BufferedReader(new FileReader("input.txt"));  
3  
4 BufferedWriter output =  
5     new BufferedWriter(new FileWriter("output.txt"));
```

Le principali classi bufferizzate sono:

- `BufferedInputStream` e `BufferedOutputStream` per gli stream di byte;
- `BufferedReader` e `BufferedWriter` per gli stream di caratteri.

Negli stream di output bufferizzati i dati vengono inizialmente scritti nel buffer e non direttamente nella destinazione finale. Per forzarne immediatamente la scrittura si utilizza il metodo `flush()`.

Alcune classi, come `PrintWriter`, possono eseguire automaticamente il *flush* (*autoflush*) dopo operazioni come `println()` o `format()`.

Il metodo `flush()` è disponibile per tutti gli stream di output, ma produce effetti solo se lo stream è bufferizzato.

## 12.4 Scansionare i dati

Gli oggetti del tipo `Scanner` sono utili per suddividere dell'input formattato in singoli token, in modo tale da poterli tradurre e gestire in base al loro tipo di dati.

Di default viene usato il white space (in genere blank, tab e terminatori di linea) per separare i token.

```

1 import java.io.*;
2 import java.util.Scanner;
3
4 public class ScanXan {
5     public static void main(String[] args) throws IOException {
6
7         Scanner s = null;
8
9         try {
10             s = new Scanner(new BufferedReader(new FileReader("xanadu.txt")));
11
12             while (s.hasNext()) {
13                 System.out.println(s.next());
14             }
15         } finally {
16             if (s != null) {
17                 s.close();
18             }
19         }
20     }
21 }

```

In questo modo verranno presi i dati parola per parola dal file aperto. Per usare un altro terminatore si usa il metodo `useDelimiter()`, specificando una regex. Ad esempio, supponendo che il separatore debba essere una virgola seguito opzionalmente da un carattere whitespace, si userebbe `s.useDelimiter(", s*)`.

Negli esempi precedenti lo `Scanner` è stato utilizzato per leggere token come semplici stringhe. In realtà, questa classe è in grado di leggere direttamente valori di quasi tutti i tipi primitivi di Java (ad eccezione di `char`), oltre a `BigInteger` e `BigDecimal`.

Ad esempio, è possibile utilizzare metodi come:

- `nextInt()`;
- `nextDouble()`;
- `nextBoolean()`;
- `nextLong()`;
- `nextFloat()`;
- `nextByte()`.

L'interpretazione dei valori numerici dipende dalla **locale** utilizzata dallo `Scanner`. Ad esempio, il separatore delle migliaia e il separatore decimale possono variare da un Paese all'altro. Se necessario, è possibile specificare esplicitamente la locale tramite il metodo `useLocale()`.

## 12.5 Formattare i dati

Gli oggetti stream che implementano la formattazione sono istanze di `PrintWriter`, per uno stream di caratteri, e `PrintStream`, per uno stream di byte. In generale gli unici oggetti che si utilizzeranno saranno `System.out` e `System.err`.

I metodi `print()` e `println()` stampano un singolo valore convertendolo automaticamente in una stringa (tramite `toString()` o conversioni automatiche del compilatore).

- `print()` stampa il valore senza andare a capo.
- `println()` stampa il valore e aggiunge un terminatore di riga.

```
1 public class Root {
2     public static void main(String[] args) {
3         int i = 2;
4         double r = Math.sqrt(i);
5
6         System.out.print("The square root of ");
7         System.out.print(i);
8         System.out.print(" is ");
9         System.out.print(r);
10        System.out.println(".");
11
12        i = 5;
13        r = Math.sqrt(i);
14        System.out.println("The square root of " + i + " is " + r + ".");
15
16        // Output
17        // The square root of 2 is 1.4142135623730951.
18        // The square root of 5 is 2.23606797749979.
19    }
20 }
```

Il metodo `format()` permette di controllare con precisione l'aspetto dell'output tramite una *format string*.

Una format string è composta da:

- testo normale, stampato così com'è;
- specificatori di formato, che iniziano con `%` e indicano come stampare ciascun argomento.

Sintassi generale:

```
1 public class Root2 {
2     public static void main(String[] args) {
3         int i = 2;
4         double r = Math.sqrt(i);
5
6         System.out.format("The square root of %d is %f.%n", i, r);
7
8         // Output
9         // The square root of 2 is 1.414214.
10    }
11 }
```

## 12.6 I/O da Riga Comando

Un programma Java può interagire con l'utente tramite la riga di comando utilizzando gli **Standard Streams** oppure la classe **Console**.

Gli Standard Streams sono tre flussi predefiniti che non devono essere aperti esplicitamente:

- **System.in**: input standard (tastiera);
- **System.out**: output standard (schermo);
- **System.error**: output degli errori.

**System.out** e **System.err** permettono di separare l'output normale dai messaggi di errore, così da poter reindirizzare il primo verso un file mantenendo visibili gli errori.

Per motivi storici, gli Standard Streams sono **byte stream**. In particolare:

- **System.out** e **System.err** sono oggetti di tipo **PrintStream**;
- **System.in** è un oggetto di tipo **InputStream**.

Se si vuole leggere caratteri anziché byte, è necessario convertire **System.in** tramite **InputStreamReader**:

```
1 InputStreamReader reader = new InputStreamReader(System.in);
```

La classe **Console** fornisce un'interfaccia più completa per l'I/O da terminale e utilizza veri **character stream**.

Per ottenere l'oggetto **Console** si usa: **Console console = System.console()**.

Se il metodo restituisce **null**, la console non è disponibile (ad esempio perché il programma è stato eseguito da un IDE o in un ambiente non interattivo).

Oltre ai metodi di input e output, la **Console** offre il metodo:

```
1 char[] password = console.readPassword();
```

che legge una password senza mostrarla sullo schermo (*echo disabilitato*) e restituisce un array di caratteri invece di una **String**. Questa scelta permette di eliminare il contenuto dalla memoria una volta terminato l'utilizzo:

```
1 Arrays.fill(password, ' ');
```

Un tipico programma per il cambio della password esegue i seguenti passaggi:

1. recupera l'oggetto **Console**;
2. legge username e password corrente;
3. verifica le credenziali;
4. legge due volte la nuova password e controlla che coincidano (**Arrays.equals()**);
5. aggiorna la password;
6. sovrascrive tutti gli array contenenti le password mediante **Arrays.fill()** per rimuoverle dalla memoria.

L'uso di **char[]** al posto di **String** rappresenta una misura di sicurezza, poiché il contenuto dell'array può essere cancellato immediatamente dopo l'utilizzo.

## 12.7 Data Streams

I **data streams** permettono di eseguire I/O binario su dati primitivi (`boolean`, `char`, `byte`, `short`, `int`, `long`, `float`, `double`) e su valori di tipo `String`.

Le classi principali sono:

- **`DataInputStream`**: legge dati primitivi e stringhe da uno stream binario.
- **`DataOutputStream`**: scrive dati primitivi e stringhe in uno stream binario.

Entrambe sono dei *wrapper*: devono essere costruite sopra uno stream di byte esistente.

### 12.7.1 Scrittura dei dati

Un `DataOutputStream` viene generalmente creato sopra un `FileOutputStream`, spesso utilizzando anche un `BufferedOutputStream`.

```
1 DataOutputStream out =  
2     new DataOutputStream(  
3         new BufferedOutputStream(  
4             new FileOutputStream(dataFile)));
```

Ogni tipo di dato possiede un metodo di scrittura dedicato:

- **`writeDouble()`**: scrive un valore `double`.
- **`writeInt()`**: scrive un valore `int`.
- **`writeUTF()`**: scrive una `String` utilizzando una versione modificata della codifica UTF-8.

Esempio:

```
1 for (int i = 0; i < prices.length; i++) {  
2     out.writeDouble(prices[i]);  
3     out.writeInt(units[i]);  
4     out.writeUTF(descs[i]);  
5 }
```

### 12.7.2 Lettura dei dati

Anche `DataInputStream` deve essere costruito sopra uno stream di byte.

```
1 DataInputStream in =  
2     new DataInputStream(  
3         new BufferedInputStream(  
4             new FileInputStream(dataFile)));
```

I dati devono essere letti nello stesso ordine con cui sono stati scritti.

```
1 price = in.readDouble();  
2 unit = in.readInt();  
3 desc = in.readUTF();
```

Ogni metodo di lettura deve corrispondere esattamente al metodo di scrittura utilizzato, poiché il file contiene solo dati binari e non informazioni sul loro tipo.

### 12.7.3 Fine del file

La fine del file viene rilevata intercettando l'eccezione `EOFException`, non tramite un valore di ritorno.

```
1 try {
2     while (true) {
3         price = in.readDouble();
4         unit = in.readInt();
5         desc = in.readUTF();
6     }
7 } catch (EOFException e) {
8     // Fine del file
9 }
```

I Data Streams sono efficienti per memorizzare dati primitivi in formato binario, ma richiedono che il programmatore mantenga la corrispondenza tra operazioni di scrittura e lettura.

Ad esempio, se i dati vengono scritti come:

```
1 out.writeDouble(price);
2 out.writeInt(unit);
3 out.writeUTF(desc);
```

devono essere letti nello stesso ordine:

```
1 price = in.readDouble();
2 unit = in.readInt();
3 desc = in.readUTF();
```

### 12.7.4 Nota sui valori monetari

È sconsigliato utilizzare `double` per rappresentare importi monetari, poiché i numeri floating point non rappresentano esattamente molte frazioni decimali (ad esempio 0.1).

Per applicazioni finanziarie è preferibile utilizzare:

```
1 java.math.BigDecimal
```

Essendo un tipo oggetto, `BigDecimal` non può essere gestito direttamente dai Data Streams; per serializzare oggetti si utilizzano gli **Object Streams**.

## 12.8 Object Streams

Gli **object streams** permettono di eseguire I/O su oggetti. Sono l'equivalente dei *data streams*, ma per tipi oggetto invece che solo per tipi primitivi.

Affinché un oggetto possa essere scritto o letto tramite object stream, la sua classe deve implementare l'interfaccia marker **Serializable**.

Le classi principali sono:

- **ObjectInputStream**: legge oggetti e dati primitivi da uno stream.
- **ObjectOutputStream**: scrive oggetti e dati primitivi su uno stream.

Queste classi implementano anche i metodi di I/O dei tipi primitivi, perché derivano dalle interfacce **DataInput** e **DataOutput**. Di conseguenza, uno stesso object stream può contenere sia oggetti sia valori primitivi.

### 12.8.1 Serializzazione

La **serializzazione** è il processo con cui un oggetto viene trasformato in una sequenza di byte, in modo da poter essere salvato su file o inviato attraverso uno stream.

Per scrivere un oggetto si usa `out.writeObject(obj)`, per leggerlo si usa `in.readObject()`.

Poiché `readObject()` restituisce un valore di tipo **Object**, spesso è necessario effettuare un cast, ad esempio con `Calendar date = (Calendar) in.readObject()`.

Se il tipo letto non è quello atteso, oppure la classe dell'oggetto non è disponibile, può essere generata una **ClassNotFoundException**.

### 12.8.2 Oggetti complessi

I metodi `writeObject()` e `readObject()` gestiscono automaticamente anche gli oggetti referenziati. Se un oggetto contiene riferimenti ad altri oggetti, `writeObject()` non salva solo l'oggetto principale, ma attraversa l'intero grafo dei riferimenti e scrive anche tutti gli oggetti necessari per ricostruirlo.

Ad esempio, se l'oggetto **a** contiene riferimenti agli oggetti **b** e **c**, e **b** contiene riferimenti a **d** ed **e**, allora `out.writeObject(a)` scrive nello stream anche **b**, **c**, **d** ed **e**. Quando **a** viene letto con `readObject()`, vengono ricostruiti anche tutti gli oggetti collegati e vengono preservati i riferimenti originali.

### 12.8.3 Riferimenti condivisi

Uno stream mantiene una sola copia di ogni oggetto scritto. Se lo stesso oggetto viene scritto più volte nello stesso stream, non viene duplicato: vengono salvati più riferimenti allo stesso oggetto.

```
1 Object ob = new Object();
2
3 out.writeObject(ob);
4 out.writeObject(ob);
```

La lettura deve comunque avere lo stesso numero di chiamate a `readObject()`:

```
1 Object ob1 = in.readObject();
2 Object ob2 = in.readObject();
```

In questo caso, `ob1` e `ob2` faranno riferimento allo stesso oggetto ricostruito. Se invece lo stesso oggetto viene scritto su due stream diversi, alla lettura verranno creati due oggetti distinti.

#### 12.8.4 Uso con tipi primitivi

Gli object streams possono gestire anche dati primitivi, usando gli stessi metodi dei data streams.

Esempio:

```
1 out.writeInt(10);
2 out.writeObject("Testo");
3 out.writeObject(new BigDecimal("19.99"));
```

Durante la lettura bisogna rispettare lo stesso ordine:

```
1 int value = in.readInt();
2 String text = (String) in.readObject();
3 BigDecimal price = (BigDecimal) in.readObject();
```

Gli object streams sono utili quando bisogna salvare strutture dati più complesse rispetto ai soli tipi primitivi. A differenza dei data streams, permettono di salvare oggetti come `BigDecimal`, `Calendar` o oggetti definiti dal programmatore, purché siano serializzabili.

## 13 Reflection e oggetti Class

Per ogni tipo esistente, la JVM crea un'istanza della classe `java.lang.Class`.

L'oggetto `Class` rappresenta il tipo a runtime e contiene informazioni sulla sua struttura. È inoltre il punto di accesso principale alle API di **Reflection**.

Un oggetto `Class` è immutabile e permette di ottenere informazioni quali:

- nome della classe;
- modificatori (`public`, `abstract`, `final`, ...);
- campi (*fields*);
- metodi;
- costruttori;
- classi annidate (*nested classes*);
- relazioni di ereditarietà e implementazione.

Inoltre, tramite Reflection è possibile creare dinamicamente oggetti e invocare metodi durante l'esecuzione del programma.

### 13.1 Operazioni principali della Reflection

Le operazioni più comuni eseguibili tramite la classe `Class` sono:

- ottenere l'oggetto `Class` associato a un tipo;
- esaminare modificatori e informazioni sul tipo;
- scoprire costruttori, campi, metodi e classi annidate;
- utilizzare la Reflection per creare oggetti o invocare metodi a runtime.

### 13.2 Ottenere un oggetto Class

La classe `Class` è il punto di accesso alle API di Reflection. Esistono diversi modi per ottenere un oggetto `Class`, a seconda delle informazioni disponibili.

Se si possiede un'istanza di un oggetto, è possibile ottenere il relativo oggetto `Class` tramite il metodo `getClass()`.

```
1 String s = "Hello";
2 Class<?> c = s.getClass();
```

Questo metodo può essere utilizzato solo con i tipi riferimento, poiché eredita da `Object`. Anche gli array e gli enum sono oggetti, quindi supportano `getClass()`.

```
1 byte[] bytes = new byte[100];
2 Class<?> c = bytes.getClass();
```

Se non si dispone di un'istanza, ma si conosce il tipo a compile-time, è possibile utilizzare la sintassi `Class<?> c = String.class`. Questo è anche il metodo consigliato per ottenere il `Class` dei tipi primitivi:

```

1 Class<?> c1 = int.class;
2 Class<?> c2 = boolean.class;
3 Class<?> c3 = double.class;
4
5 // Array
6 Class<?> c = int[] [].class;

```

Se si conosce soltanto il nome completo della classe (*fully-qualified name*), è possibile ottenere il relativo oggetto `Class` mediante il metodo statico `forName()`:

```

1 Class<?> c = Class.forName("java.lang.String");

```

Questo metodo richiede il nome completo della classe e non può essere utilizzato direttamente con i tipi primitivi.

Ogni classe wrapper dei tipi primitivi possiede un campo statico `TYPE`, equivalente all'utilizzo della sintassi `.class`.

```

1 Class<?> c1 = Double.TYPE;
2 Class<?> c2 = Void.TYPE;

```

Ad esempio `Double.TYPE == double.class` è una scrittura valida. Anche se disponibile, la sintassi `.class` è generalmente preferita.

### 13.3 Metodi che restituiscono oggetti `Class`

| Metodo                            | Descrizione                                                                                                                                                          |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>getSuperclass()</code>      | Restituisce la superclasse della classe corrente.                                                                                                                    |
| <code>getClasses()</code>         | Restituisce tutte le classi, interfacce ed enum <b>pubblici</b> dichiarati o ereditati.                                                                              |
| <code>getDeclaredClasses()</code> | Restituisce tutte le classi, interfacce ed enum dichiarati direttamente nella classe, compresi quelli <b>privati</b> .                                               |
| <code>getDeclaringClass()</code>  | Restituisce la classe nella quale è stato dichiarato un membro (campo, metodo, costruttore o classe annidata). Per le classi anonime restituisce <code>null</code> . |
| <code>getEnclosingClass()</code>  | Restituisce la classe che racchiude immediatamente la classe corrente. È utile soprattutto per classi interne e classi anonime.                                      |

## 14 Generics

### 14.1 Introduzione

Le generics permettono di passare tipi (classi ed interfacce) come parametri durante la definizione di classi, interfacce e metodi. Questo permette di riutilizzare lo stesso codice usando diversi tipi come input, mantenendo pur sempre lo stesso comportamento.

L'utilizzo dei **Generics** offre diversi vantaggi:

- **Maggiore sicurezza dei tipi (*type safety*)**: il compilatore verifica la correttezza dei tipi durante la compilazione, riducendo la possibilità di errori a runtime.
- **Eliminazione dei cast espliciti**: conoscendo il tipo degli elementi contenuti in una collezione, il compilatore evita la necessità di effettuare conversioni esplicite (*cast*).
- **Possibilità di implementare algoritmi generici**: è possibile scrivere classi e metodi che funzionano con tipi diversi, mantenendo il controllo statico dei tipi, migliorando il riutilizzo del codice e la leggibilità.

### 14.2 Generic Types

Un **generic type** è una classe o un'interfaccia parametrizzata rispetto a uno o più tipi. In questo modo è possibile definire classi indipendenti dal tipo di dato che gestiscono, mantenendo il controllo statico dei tipi.

La sintassi generale è:

```
1 class NomeClasse<T1, T2, ..., Tn> {  
2     ...  
3 }
```

I parametri di tipo possono essere utilizzati ovunque all'interno della classe, come se fossero normali tipi.

Ad esempio:

```
1 public class Box<T> {  
2  
3     private T value;  
4  
5     public void set(T value) {  
6         this.value = value;  
7     }  
8  
9     public T get() {  
10        return value;  
11    }  
12 }
```

Il parametro di tipo può rappresentare qualsiasi tipo non primitivo: classi, interfacce, array oppure un altro parametro di tipo.

### 14.2.1 Convenzioni sui nomi

Per convenzione i parametri di tipo sono identificati mediante singole lettere maiuscole:

- T: Type;
- E: Element;
- K: Key;
- V: Value;
- N: Number;
- S, U, V: ulteriori parametri di tipo.

### 14.2.2 Istanziare un tipo generico

Per utilizzare una classe generica è necessario specificare un **type argument**, ottenendo un **parameterized type**.

```
1 Box<Integer> integerBox;
```

L'oggetto viene poi creato normalmente mediante l'operatore **new**:

```
1 Box<Integer> integerBox = new Box<Integer>();
```

A partire da Java 7 è possibile utilizzare la **diamond notation** (<>), lasciando al compilatore l'inferenza dei parametri di tipo:

```
1 Box<Integer> integerBox = new Box<>();
```

### 14.2.3 Parametri di tipo multipli

Una classe o un'interfaccia generica può dichiarare più parametri di tipo.

```
1 public class OrderedPair<K, V> {  
2  
3     private K key;  
4     private V value;  
5  
6     public OrderedPair(K key, V value) {  
7         this.key = key;  
8         this.value = value;  
9     }  
10 }
```

Utilizzo:

```
1 OrderedPair<String, Integer> p = new OrderedPair<>("Even", 8);
```

I parametri di tipo possono essere a loro volta tipi parametrizzati:

```
1 OrderedPair<String, Box<Integer>> p = new OrderedPair<>("primes", new Box<>());
```

## 14.3 Raw Types

Un **raw type** è una classe o un'interfaccia generica utilizzata senza specificare i parametri di tipo.

Ad esempio, data la classe:

```
1 public class Box<T> { ... }
```

il seguente è un tipo parametrizzato:

```
1 Box<Integer> intBox = new Box<>();
```

mentre questo è un **raw type**:

```
1 Box rawBox = new Box();
```

I raw types esistono principalmente per garantire la **retrocompatibilità** con il codice scritto prima dell'introduzione dei Generics (JDK 5).

Utilizzando un raw type, il compilatore perde le informazioni sul tipo degli oggetti contenuti e tratta i valori come semplici `Object`. Di conseguenza vengono disabilitati molti controlli statici sui tipi.

È consentito assegnare un tipo parametrizzato a un raw type:

```
1 Box<String> stringBox = new Box<>();
2 Box rawBox = stringBox;           // OK
```

L'operazione inversa è anch'essa consentita, ma produce un **unchecked warning**:

```
1 Box rawBox = new Box();
2 Box<Integer> intBox = rawBox;    // Warning
```

Anche l'invocazione di metodi generici tramite un raw type genera un warning:

```
1 Box<String> stringBox = new Box<>();
2 Box rawBox = stringBox;
3
4 rawBox.set(8);    // Warning
```

In generale, l'utilizzo dei raw types è sconsigliato, poiché aggira i controlli dei Generics e può causare errori rilevati solo a runtime.

## 14.4 Generic Methods

Un **generic method** è un metodo che dichiara uno o più parametri di tipo propri, indipendentemente dal fatto che appartenga a una classe generica o meno.

La sintassi generale è:

```
1 public <T> T metodo(T valore) {  
2     ...  
3 }
```

I parametri di tipo sono dichiarati tra parentesi angolari prima del tipo di ritorno del metodo.

Anche i metodi **static** possono essere generici:

```
1 public static <T> void stampa(T obj) {  
2     System.out.println(obj);  
3 }
```

Un metodo può dichiarare più parametri di tipo:

```
1 public static <K, V> boolean compare(  
2     Pair<K, V> p1,  
3     Pair<K, V> p2) {  
4  
5     return p1.getKey().equals(p2.getKey()) &&  
6         p1.getValue().equals(p2.getValue());  
7 }
```

L'invocazione può specificare esplicitamente i parametri di tipo:

```
1 boolean same = Util.<Integer, String>compare(p1, p2);
```

Nella maggior parte dei casi non è necessario specificarli, poiché il compilatore è in grado di dedurli automaticamente (**type inference**):

```
1 boolean same = Util.compare(p1, p2);
```

## 14.5 Bounded Type Parameters

In alcuni casi è utile limitare i tipi che possono essere utilizzati come argomenti di un tipo generico. A questo scopo si utilizzano i **bounded type parameters**.

La sintassi generale è:

```
1 <T extends Tipo>
```

Ad esempio, il seguente metodo accetta solo oggetti di tipo `Number` o di una sua sottoclasse:

```
1 public <T extends Number> void print(T value) {  
2     System.out.println(value);  
3 }
```

In questo contesto la parola chiave `extends` indica che il parametro di tipo deve essere una sottoclasse della classe indicata oppure implementare l'interfaccia specificata.

L'utilizzo di un bound offre due vantaggi:

- limita i tipi che possono essere utilizzati come argomenti;
- permette di invocare i metodi definiti nel bound.

Ad esempio:

```
1 public class NaturalNumber<T extends Integer> {  
2  
3     private T n;  
4  
5     public boolean isEven() {  
6         return n.intValue() % 2 == 0;  
7     }  
8 }
```

Poiché `T` estende `Integer`, è possibile invocare il metodo `intValue()`.

### 14.5.1 Bound multipli

Un parametro di tipo può avere più bound:

```
1 <T extends A & B & C>
```

Il parametro di tipo dovrà quindi estendere o implementare tutti i tipi specificati.

Se uno dei bound è una classe, questa deve comparire per prima:

```
1 class D<T extends A & B & C> { } // Corretto  
2 class D<T extends B & A & C> { } // Errore
```

## 14.6 Metodi generici e bounded type parameters

I **bounded type parameters** sono particolarmente utili nella definizione di algoritmi generici, cioè algoritmi che devono funzionare su tipi diversi ma che richiedono alcune operazioni comuni.

Ad esempio, si potrebbe voler scrivere un metodo generico che conta quanti elementi di un array sono maggiori di un certo elemento:

```
1 public static <T> int countGreaterThan(T[] array, T elem) {
2     int count = 0;
3
4     for (T e : array) {
5         if (e > elem) {      // Errore di compilazione
6             count++;
7         }
8     }
9
10    return count;
11 }
```

Questo codice non compila, perché l'operatore `>` può essere usato solo con tipi primitivi, non con oggetti.

Per confrontare oggetti è necessario imporre che il tipo `T` implementi l'interfaccia `Comparable<T>`. In questo modo il compilatore sa che gli oggetti di tipo `T` possiedono il metodo `compareTo()`.

```
1 public static <T extends Comparable<T>> int countGreaterThan(
2     T[] array,
3     T elem) {
4
5     int count = 0;
6
7     for (T e : array) {
8         if (e.compareTo(elem) > 0) {
9             count++;
10        }
11    }
12
13    return count;
14 }
```

Il bound:

```
1 <T extends Comparable<T>>
```

significa che `T` deve essere un tipo confrontabile con altri oggetti dello stesso tipo.

Il metodo `compareTo()` restituisce:

- un valore maggiore di 0, se l'oggetto corrente è maggiore dell'oggetto passato come argomento;
- 0, se i due oggetti sono equivalenti;
- un valore minore di 0, se l'oggetto corrente è minore dell'oggetto passato come argomento.

## 14.7 Generics, ereditarietà e sottotipi

In Java è possibile assegnare un oggetto di una sottoclasse a una variabile di una sua superclasse:

```
1 Object obj = new Integer(10);    // OK
2 Number n = new Integer(10);      // OK
```

Questo è valido perché `Integer` è un sottotipo sia di `Number` sia di `Object`.

Con i Generics, però, questa relazione non si trasferisce automaticamente ai tipi parametrizzati. Anche se `Integer` è un sottotipo di `Number`, `Box<Integer>` **non** è un sottotipo di `Box<Number>`.

Ad esempio:

```
1 public void boxTest(Box<Number> box) { ... }
2
3 Box<Integer> intBox = new Box<>();
4
5 boxTest(intBox);    // Errore di compilazione
```

Il metodo accetta solo oggetti di tipo `Box<Number>`, non `Box<Integer>` o `Box<Double>`.

In generale, dati due tipi concreti A e B, anche se A è sottotipo di B, non è detto che `MyClass<A>` sia sottotipo di `MyClass<B>`.

```
1 Integer extends Number
2
3 Box<Integer> non extends Box<Number>
```

Il supertipo comune di `Box<Integer>` e `Box<Number>` è semplicemente `Object`.

### 14.7.1 Classi generiche e sottotipi

Una relazione di sottotipo tra classi o interfacce generiche esiste solo se viene definita tramite `extends` o `implements`, mantenendo lo stesso type argument.

Ad esempio:

```
1 ArrayList<String> list = new ArrayList<>();
2
3 List<String> l = list;           // OK
4 Collection<String> c = list;    // OK
```

Questo è valido perché:

```
1 ArrayList<E> implements List<E>
2 List<E> extends Collection<E>
```

Quindi:

```
1 ArrayList<String> extends List<String>
2 List<String> extends Collection<String>
```

La relazione di ereditarietà viene preservata solo se il parametro di tipo rimane lo stesso.

## 14.8 Type Inference

La **type inference** è la capacità del compilatore Java di dedurre automaticamente i parametri di tipo di classi e metodi generici a partire dal contesto, evitando al programmatore di specificarli esplicitamente.

Il compilatore utilizza principalmente:

- i tipi degli argomenti passati ai metodi;
- il tipo della variabile a cui viene assegnato il risultato;
- il tipo di ritorno atteso.

### 14.8.1 Metodi generici

Nella maggior parte dei casi non è necessario specificare esplicitamente i parametri di tipo di un metodo generico.

Ad esempio, sono equivalenti `Util.<Integer>print(10)` e `Util.print(10)`.

Nel secondo caso il compilatore deduce automaticamente che il parametro di tipo è `Integer`.

### 14.8.2 Diamond operator

La type inference viene utilizzata anche durante l'istanziatura di classi generiche mediante il **diamond operator** (`<>`).

Invece di scrivere:

```
1 Map<String, List<String>> map = new HashMap<String, List<String>>();
```

è possibile scrivere:

```
1 Map<String, List<String>> map = new HashMap<>();
```

Il compilatore deduce automaticamente i parametri di tipo dal contesto.

È preferibile utilizzare il diamond operator rispetto al raw type:

```
1 Map<String, List<String>> map = new HashMap();    // Unchecked warning
```

poiché quest'ultimo disabilita i controlli dei Generics.

### 14.8.3 Target Type

La type inference sfrutta anche il **target type**, cioè il tipo atteso dal contesto.

Ad esempio:

```
1 List<String> list = Collections.emptyList();
```

Poiché la variabile è di tipo `List<String>`, il compilatore deduce automaticamente che il parametro di tipo di `emptyList()` è `String`.

## 14.9 Wildcards

Il carattere `?`, detto **wildcard**, rappresenta un tipo sconosciuto.

Le wildcard vengono utilizzate quando non è importante conoscere il tipo esatto di un parametro generico, ma solo alcune sue proprietà.

Esistono tre tipologie principali:

- wildcard superiore (`? extends T`);
- wildcard non limitata (`?`);
- wildcard inferiore (`? super T`).

### 14.9.1 Upper bounded wildcard

Una wildcard superiore limita il tipo sconosciuto a una classe (o interfaccia) e a tutte le sue sottoclassi.

La sintassi è `<? extends T>`. Ad esempio:

```
1 public static double sum(List<? extends Number> list) {
2     double s = 0;
3
4     for (Number n : list)
5         s += n.doubleValue();
6
7     return s;
8 }
```

Il metodo può essere invocato con:

```
1 List<Integer>
2 List<Double>
3 List<Float>
4 List<Number>
```

Una wildcard superiore è indicata quando un metodo deve solamente **leggere** elementi dalla collezione.

### 14.9.2 Unbounded wildcard

La wildcard non limitata è indicata con `<?>`. Essa rappresenta una collezione di tipo sconosciuto.

È utile quando il metodo non dipende dal tipo degli elementi, ad esempio per stampare una collezione o invocarne metodi come `size()` o `clear()`.

```
1 public static void printList(List<?> list) {
2     for (Object obj : list)
3         System.out.println(obj);
4 }
```

Il metodo può ricevere qualsiasi lista:

```
1 List<Integer>
2 List<String>
3 List<Double>
4 ...
```

In una `List<?>` non è possibile inserire nuovi elementi (ad eccezione di `null`), poiché il tipo effettivo della lista è sconosciuto.

### 14.9.3 Lower bounded wildcard

Una wildcard inferiore limita il tipo sconosciuto a una classe e a tutti i suoi supertipi.

La sintassi è `<? super T>`. Ad esempio:

```
1 public static void addNumbers(List<? super Integer> list) {  
2     for (int i = 1; i <= 10; i++)  
3         list.add(i);  
4 }
```

Il metodo può essere invocato con:

```
1 List<Integer>  
2 List<Number>  
3 List<Object>
```

Una wildcard inferiore è indicata quando un metodo deve principalmente **scrivere** elementi nella collezione.

## 14.10 Wildcards e sottotipi

Come visto in precedenza, anche se `Integer` è un sottotipo di `Number`, non vale la relazione:

```
1 List<Integer> <: List<Number>    // Falso
```

Il supertipo comune di tutte le liste parametrizzate è:

```
1 List<?>
```

L'utilizzo delle wildcard permette però di creare relazioni di sottotipo tra tipi generici.

Ad esempio:

```
1 List<? extends Integer> intList = new ArrayList<>();
2
3 List<? extends Number> numList = intList;    // OK
```

Poiché `Integer` è un sottotipo di `Number`, anche `List<? extends Integer>` è un sottotipo di `List<? extends Number>`.

| Wildcard                         | Quando usarla                                              | Operazioni consentite                                                                                                     |
|----------------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>&lt;? extends T&gt;</code> | Quando la collezione produce elementi ( <i>Producer</i> ). | Si possono leggere elementi di tipo <code>T</code> ; non è possibile inserirne (eccetto <code>null</code> ).              |
| <code>&lt;? super T&gt;</code>   | Quando la collezione consuma elementi ( <i>Consumer</i> ). | È possibile inserire oggetti di tipo <code>T</code> ; leggendo si ottengono oggetti di tipo <code>Object</code> .         |
| <code>&lt;?&gt;</code>           | Quando il tipo degli elementi non è rilevante.             | È possibile usare solo operazioni indipendenti dal tipo ( <code>size()</code> , <code>clear()</code> , iterazione, ecc.). |

**Regola PECS** Per scegliere correttamente una wildcard si utilizza la regola **PECS** (*Producer Extends, Consumer Super*):

- **Producer Extends:** usare `<? extends T>` quando una collezione fornisce dati da leggere;
- **Consumer Super:** usare `<? super T>` quando una collezione riceve dati da inserire.

Se una collezione deve essere sia letta sia modificata, è preferibile non utilizzare wildcard.

## 14.11 Type Erasure

I Generics in Java esistono principalmente a tempo di compilazione. Il compilatore utilizza i Generics per effettuare controlli più forti sui tipi, ma poi applica un processo chiamato **type erasure**.

Con la **type erasure**, il compilatore sostituisce i type parameter con il loro bound, oppure con `Object` se non hanno bound, inserisce automaticamente i cast necessari e genera, se necessario, dei **bridge methods** per preservare il polimorfismo.

Di conseguenza, a runtime non vengono create classi diverse per ogni tipo parametrizzato. Ad esempio, `List<String>` e `List<Integer>` condividono la stessa classe runtime `List`. Questo significa che i Generics non introducono overhead a runtime.

### 14.11.1 Erasure dei tipi generici

Se un type parameter non ha bound, viene sostituito con `Object`:

```
1 // Prima
2 public class Node<T> {
3     private T data;
4
5     public T getData() {
6         return data;
7     }
8 }
9
10 // Dopo
11 public class Node {
12     private Object data;
13
14     public Object getData() {
15         return data;
16     }
17 }
```

Se invece il type parameter ha un bound, viene sostituito con il primo bound:

```
1 // Prima
2 public class Node<T extends Comparable<T>> {
3     private T data;
4
5     public T getData() {
6         return data;
7     }
8 }
9
10 // Dopo
11 public class Node {
12     private Comparable data;
13
14     public Comparable getData() {
15         return data;
16     }
17 }
```

### 14.11.2 Erasure dei metodi generici

La type erasure si applica anche ai metodi generici.

Prima:

```
1 public static <T> int count(T[] array, T elem) {
2     int count = 0;
3
4     for (T e : array) {
5         if (e.equals(elem))
6             count++;
7     }
8
9     return count;
10 }
```

Dopo:

```
1 public static int count(Object[] array, Object elem) {
2     int count = 0;
3
4     for (Object e : array) {
5         if (e.equals(elem))
6             count++;
7     }
8
9     return count;
10 }
```

Se il parametro ha un bound:

```
1 // Prima
2 public static <T extends Shape> void draw(T shape) {
3     ...
4 }
5
6 // Dopo
7 public static void draw(Shape shape) {
8     ...
9 }
```

### 14.11.3 Bridge methods

In alcuni casi, dopo la type erasure, le firme dei metodi non coincidono più con quelle attese per l'override. Per preservare il polimorfismo, il compilatore può generare automaticamente dei **bridge methods**.

Ad esempio, se una classe specializza un metodo generico:

```
1 class Node<T> {
2     public void setData(T data) { ... }
3 }
4
5 class MyNode extends Node<Integer> {
6     public void setData(Integer data) { ... }
7 }
```

dopo la type erasure i metodi diventano:

```
1 // Metodo superclasse
2 public void setData(Object data)
3
4 // Metodo sottoclasse
5 public void setData(Integer data)
```

Per mantenere valido l'override, il compilatore genera un metodo ponte:

```
1 public void setData(Object data) {
2     setData((Integer) data);
3 }
```

Normalmente i bridge methods non devono essere gestiti dal programmatore.

## 14.12 Non-Reifiable Types

Un tipo è **reifiable** se le sue informazioni di tipo sono completamente disponibili a runtime.

Sono reifiable:

- i tipi primitivi;
- i tipi non generici;
- i raw types;
- i tipi con wildcard non limitata, come `List<?>`.

Sono invece **non-reifiable** i tipi generici parametrizzati, perché le informazioni sui type argument vengono eliminate dalla type erasure.

Esempi:

```
1 List<String>
2 List<Integer>
3 List<Number>
```

A runtime, la JVM non distingue tra questi tipi: vede semplicemente delle `List`.

### 14.13 Restrizioni dei Generics

A causa della type erasure, i Generics in Java hanno alcune limitazioni importanti.

- Non si possono usare tipi primitivi come type argument.

```
1 Pair<int, char> p;           // Errore
2 Pair<Integer, Character> p; // Corretto
```

- Non si possono creare istanze di un type parameter.

```
1 public static <T> void metodo() {
2     T obj = new T(); // Errore
3 }
```

- Non si possono dichiarare campi statici del tipo del type parameter.

```
1 class Box<T> {
2     private static T value; // Errore
3 }
```

- Non si può usare instanceof con tipi parametrizzati.

```
1 if (list instanceof ArrayList<Integer>) { } // Errore
2 if (list instanceof ArrayList<?>) { }      // Corretto
```

- Non si possono creare array di tipi parametrizzati.

```
1 List<String>[] lists =
2     new List<String>[10]; // Errore
```

- Una classe generica non può estendere Throwable.

```
1 class MyException<T> extends Exception { } // Errore
```

- Non si possono sovraccaricare metodi che, dopo la type erasure, hanno la stessa firma.

```
1 void print(Set<String> set) { }
2 void print(Set<Integer> set) { } // Errore
```

## 15 ClassLoader

### 15.1 Concetto generale

Un **ClassLoader** è l'oggetto responsabile del caricamento delle classi nella JVM. Data una classe identificata tramite il suo nome binario, il **ClassLoader** prova a trovare o generare i dati necessari per definirla. Nella situazione più comune questi dati si trovano in un file `.class`, ma non è obbligatorio: una classe può anche provenire dalla rete, da un database, da un archivio o essere generata dinamicamente dal programma.

Il risultato del caricamento è un oggetto **Class**, cioè la rappresentazione a runtime della classe caricata. Ogni oggetto **Class** contiene un riferimento al **ClassLoader** che ha definito quella classe. Per questo motivo, nella JVM l'identità effettiva di una classe non dipende solo dal suo nome, ma anche dal **ClassLoader** che l'ha caricata.

Questo significa che due classi con lo stesso nome binario possono essere considerate diverse se sono state caricate da due **ClassLoader** differenti. Questo comportamento è importante in ambienti come application server, sistemi a plugin e framework modulari.

### 15.2 Classi array

Le classi array sono un caso particolare. Non vengono create direttamente dai **ClassLoader**, ma automaticamente dalla JVM quando servono. Il **ClassLoader** associato a una classe array è lo stesso del tipo dei suoi elementi. Ad esempio, se **String** è caricata dal class loader della piattaforma, anche **String[]** userà lo stesso class loader.

Se invece l'array contiene un tipo primitivo, come `int[]` o `double[]`, allora la classe array non ha un **ClassLoader** associato, perché i tipi primitivi non vengono caricati tramite class loader.

### 15.3 ClassLoader e risorse

Un **ClassLoader** non serve solo a caricare classi. Può anche cercare risorse, cioè dati associati all'applicazione o a una libreria. Una risorsa può essere un file `.class`, un file di configurazione, un'immagine o qualunque altro file incluso nel classpath o nel module path.

Le risorse sono identificate tramite un percorso astratto separato da `/`. Questo permette di accedere a file distribuiti insieme all'applicazione senza usare direttamente percorsi assoluti del file system.

```
1 ClassLoader loader = Main.class.getClassLoader();
2
3 InputStream input = loader.getResourceAsStream("config/app.properties");
4
5 if (input != null) {
6     // Lettura della risorsa
7 }
```

I metodi più usati per lavorare con le risorse sono `getResource`, che restituisce un URL, e `getResourceAsStream`, che restituisce uno stream di input.

### 15.4 Modello di delega

I **ClassLoader** usano normalmente un modello di delega. Ogni class loader, tranne il bootstrap class loader, ha un parent. Quando riceve una richiesta per caricare una classe o una risorsa, prima delega la ricerca al parent; solo se il parent non riesce a trovare l'elemento richiesto, prova a cercarlo autonomamente.

Questo modello serve a evitare che le classi fondamentali della piattaforma vengano ridefinite accidentalmente dall'applicazione. In pratica, le classi più vicine alla JVM hanno precedenza rispetto alle classi definite dall'utente.

Il caricamento tramite `loadClass` segue in generale questa logica:

```
1 protected Class<?> loadClass(String name, boolean resolve)
2     throws ClassNotFoundException {
3
4     // 1. Controlla se la classe e' gia' stata caricata
5     Class<?> c = findLoadedClass(name);
6
7     if (c == null) {
8         try {
9             // 2. Delega al parent class loader
10            c = getParent().loadClass(name);
11        } catch (ClassNotFoundException e) {
12            // 3. Se il parent non la trova, prova a cercarla direttamente
13            c = findClass(name);
14        }
15    }
16
17    if (resolve) {
18        resolveClass(c);
19    }
20
21    return c;
22 }
```

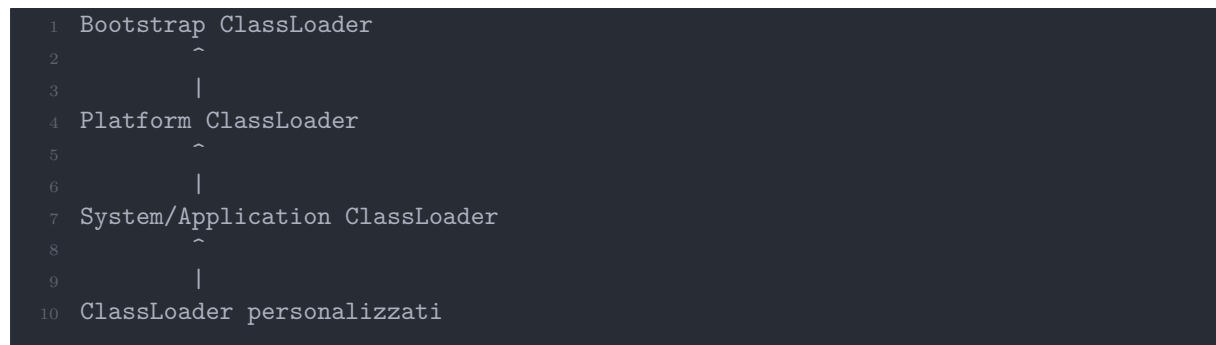
Questo codice è una semplificazione del comportamento reale, ma mostra l'idea principale: prima si controlla se la classe è già stata caricata, poi si delega al parent, infine si cerca direttamente la classe.

## 15.5 ClassLoader predefiniti

A runtime, Java usa diversi class loader predefiniti. I più importanti sono il bootstrap class loader, il platform class loader e il system class loader.

| ClassLoader            | Ruolo                                                                                                                                                                                                                                        |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bootstrap class loader | È il class loader interno della JVM. Carica le classi fondamentali della piattaforma Java. Non ha un parent ed è normalmente rappresentato con <code>null</code> , perché non è un normale oggetto Java.                                     |
| Platform class loader  | Carica le classi della piattaforma Java, cioè le API Java SE, le loro implementazioni e alcune classi specifiche del JDK. Può essere usato come parent di altri class loader.                                                                |
| System class loader    | È detto anche application class loader. Carica normalmente le classi dell'applicazione presenti nel classpath, nel module path e negli strumenti specifici del JDK. Ha come parent, direttamente o indirettamente, il platform class loader. |

Il modello può essere rappresentato così:



Questa gerarchia spiega perché, quando un'applicazione richiede una classe, la JVM tende prima a cercarla tra le classi fondamentali e poi tra quelle dell'applicazione.

## 15.6 Metodi principali

La classe `ClassLoader` contiene molti metodi, ma non è necessario memorizzare tutta l'API. Gli elementi più importanti da conoscere sono quelli legati al caricamento, alla ricerca delle classi e alla gestione delle risorse.

| Metodo                                        | Significato                                                                                            |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>loadClass(String name)</code>           | Carica una classe dato il suo nome binario. Usa il modello di delega.                                  |
| <code>findClass(String name)</code>           | Cerca direttamente una classe. Di solito è il metodo da sovrascrivere nei class loader personalizzati. |
| <code>findLoadedClass(String name)</code>     | Controlla se una classe è già stata caricata da quel class loader.                                     |
| <code>defineClass(...)</code>                 | Trasforma un array di byte o un <code>ByteBuffer</code> in un oggetto <code>Class</code> .             |
| <code>resolveClass(Class&lt;?&gt; c)</code>   | Collega la classe caricata, rendendola pronta all'uso.                                                 |
| <code>getParent()</code>                      | Restituisce il parent class loader usato per la delega.                                                |
| <code>getResource(String name)</code>         | Cerca una risorsa e restituisce un URL.                                                                |
| <code>getResourceAsStream(String name)</code> | Cerca una risorsa e restituisce uno stream di input.                                                   |
| <code>getSystemClassLoader()</code>           | Restituisce il system class loader.                                                                    |
| <code>getPlatformClassLoader()</code>         | Restituisce il platform class loader.                                                                  |

Il punto più importante è che, quando si implementa un class loader personalizzato, normalmente non si sovrascrive `loadClass`, ma `findClass`. In questo modo si mantiene il modello di delega standard.

## 15.7 ClassLoader personalizzati

Un'applicazione può definire un proprio `ClassLoader` estendendo la classe `ClassLoader`. Questo è utile quando le classi non devono essere caricate dal classpath ordinario, ma da una sorgente diversa, come rete, file system esterno, database o archivio compresso.

Un esempio tipico è un class loader che scarica classi da un server.

```

1  class NetworkClassLoader extends ClassLoader {
2
3      private String host;
4      private int port;
5
6      public NetworkClassLoader(String host, int port) {
7          this.host = host;
8          this.port = port;
9      }
10
11      @Override
12      protected Class<?> findClass(String name)
13          throws ClassNotFoundException {
14
15          byte[] bytes = loadClassData(name);
16
17          return defineClass(name, bytes, 0, bytes.length);
18      }
19
20      private byte[] loadClassData(String name)
21          throws ClassNotFoundException {
22
23          // Qui andrebbe implementata la logica per recuperare
24          // i byte della classe, ad esempio tramite rete.
25          throw new ClassNotFoundException(name);
26      }
27 }

```

Il metodo `defineClass` è quello che trasforma i byte letti in memoria in un oggetto `Class`. Prima di chiamarlo, il class loader deve quindi recuperare i byte della classe da qualche sorgente.

```

1  ClassLoader loader = new NetworkClassLoader("localhost", 8080);
2
3  Class<?> clazz = loader.loadClass("Main");
4  Object obj = clazz.getDeclaredConstructor().newInstance();

```

In questo esempio, la classe `Main` non viene caricata necessariamente dal classpath ordinario, ma dal `NetworkClassLoader`. Se la classe caricata fa riferimento ad altre classi, la JVM usa lo stesso class loader per risolvere quei riferimenti, mantenendo coerente l'ambiente di esecuzione.

## 15.8 Nome binario delle classi

I metodi di `ClassLoader` ricevono normalmente il nome binario della classe. Il nome binario è il nome usato dalla JVM per identificare una classe.

Per una classe ordinaria il nome binario coincide spesso con il nome completamente qualificato:

```

1  java.lang.String
2  java.util.ArrayList
3  mypackage.MyClass

```

Per le classi annidate, invece, il nome binario usa il simbolo `$`:

```

1  javax.swing.JSpinner$DefaultEditor
2  java.security.KeyStore$Builder$FileBuilder$1

```

Questa distinzione è importante perché passare un nome scritto in modo errato a `loadClass` o

`findClass` causa una `ClassNotFoundException`.

## 15.9 `defineClass` e sicurezza

Il metodo `defineClass` non si limita a creare un oggetto `Class`: controlla anche che i byte ricevuti rappresentino una classe valida. Se i byte non rispettano il formato previsto dalla JVM, viene generato un errore come `ClassFormatError`.

Alcune versioni di `defineClass` ricevono anche un `ProtectionDomain`. Questo oggetto rappresenta il dominio di protezione della classe, cioè le informazioni usate dalla JVM e dal sistema di sicurezza per stabilire quali permessi assegnare al codice caricato.

Un class loader non può definire liberamente qualunque classe. Per esempio, non può definire classi in package riservati come `java.*`, a meno che non sia un class loader autorizzato dalla piattaforma. Questo evita che un'applicazione sostituisca classi fondamentali della libreria standard.

## 15.10 Package e moduli

Un `ClassLoader` può anche definire package tramite `definePackage`. Quando una classe viene definita, il class loader può associare quella classe a un package definito nel proprio spazio di caricamento.

Con Java 9 e il sistema dei moduli, alcuni metodi di `ClassLoader` supportano anche la ricerca di classi e risorse all'interno di uno specifico modulo. Per esempio, esistono versioni di `findClass` e `findResource` che ricevono anche il nome del modulo.

Per un uso base di Java è sufficiente ricordare che il class loader non lavora solo con singole classi, ma anche con package, risorse e moduli.

## 15.11 Caricamento concorrente

In ambienti complessi, più thread possono richiedere il caricamento di classi nello stesso momento. Alcuni class loader sono detti *parallel capable*, cioè capaci di gestire il caricamento parallelo di più classi.

Questo è importante perché il caricamento delle classi usa dei lock. Se il modello di delega tra class loader non è strettamente gerarchico, un caricamento non gestito correttamente può causare deadlock. Per questo motivo, i class loader che vogliono supportare il caricamento parallelo devono registrarsi tramite `registerAsParallelCapable`.

Nell'uso ordinario di Java non è necessario implementare direttamente questa funzionalità, ma è importante sapere che il caricamento delle classi può avere implicazioni anche sulla concorrenza.

## 15.12 ClassLoader e Reflection

Il collegamento con la reflection è diretto: un **ClassLoader** produce oggetti **Class**, e gli oggetti **Class** sono la base della reflection. Una volta ottenuto un oggetto **Class**, è possibile ispezionare metodi, costruttori, campi, annotazioni e creare istanze dinamicamente.

```
1 Class<?> clazz = Class.forName("mypackage.Person");
2
3 System.out.println(clazz.getName());
4
5 Object obj = clazz.getDeclaredConstructor().newInstance();
```

**Class.forName** usa un class loader per caricare la classe indicata. Per questo motivo, class loading e reflection sono spesso collegati: il primo rende disponibile la classe a runtime, la seconda permette di ispezionarla e usarla dinamicamente.

## 16 Maven e gestione delle dipendenze

### 16.1 Cos'è Maven

Maven è uno strumento usato per automatizzare la gestione, la compilazione e la distribuzione di un progetto Java. Il suo scopo principale è rendere prevedibile la struttura di un progetto e ridurre la quantità di configurazione manuale necessaria per compilarlo, testarlo e impacchettarlo.

Senza uno strumento come Maven, il programmatore dovrebbe occuparsi manualmente di scaricare le librerie esterne, aggiungerle al classpath, compilare i file sorgente nell'ordine corretto, eseguire i test e creare il file finale dell'applicazione, ad esempio un archivio `.jar`. Maven standardizza questo processo attraverso una configurazione dichiarativa.

Il file centrale di un progetto Maven è il `pom.xml`. In questo file vengono descritte le informazioni principali del progetto, le dipendenze esterne, i plugin da usare e le fasi della build.

Maven segue il principio della *convention over configuration*: se il progetto rispetta una struttura standard, molte informazioni non devono essere specificate esplicitamente. Ad esempio, Maven si aspetta che il codice sorgente si trovi in `src/main/java` e che i test si trovino in `src/test/java`.

### 16.2 Project Object Model

Il *Project Object Model*, abbreviato in **POM**, è il modello con cui Maven rappresenta un progetto. Il POM è scritto in XML e si trova nel file `pom.xml`. Questo file contiene la configurazione necessaria a Maven per sapere come gestire il progetto.

Ogni progetto Maven è identificato da tre coordinate principali: `groupId`, `artifactId` e `version`. Il `groupId` identifica solitamente l'organizzazione, il gruppo o il dominio del progetto. L'`artifactId` identifica il nome del progetto o del modulo. La `version` indica la versione dell'artefatto prodotto.

```
1 <project>
2   <modelVersion>4.0.0</modelVersion>
3
4   <groupId>it.universita</groupId>
5   <artifactId>progetto-java</artifactId>
6   <version>1.0.0</version>
7
8   <packaging>jar</packaging>
9 </project>
```

In questo esempio, Maven identifica il progetto tramite le coordinate `it.universita:progetto-java:1.0.0`. Il campo `packaging` indica il tipo di artefatto che Maven deve produrre. Il valore più comune è `jar`, ma possono esistere anche altri tipi, come `war` per applicazioni web.

Il POM può contenere anche informazioni aggiuntive, come il nome del progetto, la descrizione, gli sviluppatori, le dipendenze, i repository, i plugin e le proprietà di configurazione.

```

1 <project>
2   <modelVersion>4.0.0</modelVersion>
3
4   <groupId>it.universita</groupId>
5   <artifactId>progetto-java</artifactId>
6   <version>1.0.0</version>
7
8   <properties>
9     <maven.compiler.source>17</maven.compiler.source>
10    <maven.compiler.target>17</maven.compiler.target>
11  </properties>
12 </project>

```

La sezione **properties** permette di definire valori riutilizzabili nel resto del file. In questo caso viene indicata la versione di Java da usare per compilare il progetto.

### 16.3 Struttura standard di un progetto Maven

Maven assume che il progetto segua una struttura standard di directory. Questa struttura permette allo strumento di sapere dove trovare il codice sorgente, i test e le risorse senza doverlo specificare ogni volta nel `pom.xml`.

```

1 progetto-java/
2 |-- pom.xml
3 |-- src/
4     |-- main/
5         |-- java/
6         |-- resources/
7     |
8     |-- test/
9         |-- java/
10        |-- resources/

```

La directory `src/main/java` contiene il codice sorgente principale dell'applicazione. La directory `src/main/resources` contiene risorse usate dal programma, come file di configurazione, immagini o file testuali.

La directory `src/test/java` contiene il codice dei test. La directory `src/test/resources` contiene invece le risorse usate durante l'esecuzione dei test.

Quando Maven compila il progetto, produce una directory chiamata **target**. Questa directory contiene i file generati durante la build, come classi compilate, report di test e archivi finali.

```

1 target/
2 |-- classes/
3 |-- test-classes/
4 |-- progetto-java-1.0.0.jar

```

La directory `target/classes` contiene i file `.class` generati a partire da `src/main/java`. La directory `target/test-classes` contiene invece i file compilati dei test. Il file `.jar` è l'artefatto finale prodotto dalla build.

### 16.4 Dipendenze

Una dipendenza è una libreria esterna necessaria al progetto per compilare, eseguire o testare il codice. In Maven le dipendenze vengono dichiarate nel file `pom.xml`, all'interno della sezione

dependencies.

```
1 <dependencies>
2   <dependency>
3     <groupId>org.junit.jupiter</groupId>
4     <artifactId>junit-jupiter</artifactId>
5     <version>5.10.0</version>
6     <scope>test</scope>
7   </dependency>
8 </dependencies>
```

In questo esempio viene aggiunta una dipendenza verso JUnit, una libreria usata per scrivere test. Maven scarica automaticamente la libreria e la rende disponibile al progetto.

Ogni dipendenza è identificata tramite **groupId**, **artifactId** e **version**, cioè le stesse coordinate usate per identificare un progetto Maven. Questo significa che anche una libreria esterna è trattata come un artefatto Maven.

Il campo **scope** indica in quale fase del progetto la dipendenza deve essere disponibile. Una dipendenza con scope **test**, ad esempio, viene usata solo durante la compilazione e l'esecuzione dei test, ma non viene considerata parte del codice principale dell'applicazione.

| Scope           | Significato                                                                                   |
|-----------------|-----------------------------------------------------------------------------------------------|
| <b>compile</b>  | È lo scope predefinito. La dipendenza è disponibile durante compilazione, test ed esecuzione. |
| <b>test</b>     | La dipendenza è disponibile solo durante la compilazione e l'esecuzione dei test.             |
| <b>runtime</b>  | La dipendenza non è necessaria per compilare, ma serve durante l'esecuzione.                  |
| <b>provided</b> | La dipendenza serve per compilare, ma sarà fornita dall'ambiente di esecuzione.               |

Maven gestisce anche le dipendenze transitive. Se una libreria dichiarata nel **pom.xml** dipende a sua volta da altre librerie, Maven scarica automaticamente anche quelle, evitando al programmatore di doverle indicare manualmente una per una.

## 16.5 Repository

Un repository Maven è un archivio da cui Maven può scaricare dipendenze e plugin, oppure in cui può pubblicare gli artefatti prodotti da un progetto.

Maven usa principalmente tre tipi di repository: il repository locale, i repository remoti e Maven Central. Il repository locale si trova sulla macchina del programmatore e contiene le dipendenze già scaricate. Quando Maven ha bisogno di una libreria, controlla prima il repository locale. Se la libreria non è presente, prova a scaricarla da un repository remoto.

Il repository locale si trova normalmente nella directory **.m2/repository** dell'utente.

Maven Central è il repository remoto pubblico più usato. Contiene moltissime librerie Java già pubblicate e pronte per essere usate come dipendenze.

Il funzionamento generale è il seguente: quando una dipendenza viene dichiarata nel **pom.xml**, Maven cerca prima nel repository locale; se non la trova, la scarica da un repository remoto e la salva localmente. Le build successive potranno quindi usare la copia locale senza scaricarla di nuovo.

Un progetto può anche dichiarare repository remoti aggiuntivi, se una dipendenza non è disponibile su Maven Central.

```
1 <repositories>
2   <repository>
3     <id>repo-azienda</id>
4     <url>https://repo.example.com/maven2</url>
5   </repository>
6 </repositories>
```

Questa configurazione indica a Maven un repository remoto aggiuntivo da cui può scaricare dipendenze.

## 16.6 Build lifecycle

Maven organizza la costruzione del progetto tramite un ciclo di vita, detto *build lifecycle*. Un lifecycle è composto da una sequenza ordinata di fasi. Quando si esegue una fase, Maven esegue anche tutte le fasi precedenti necessarie.

Il lifecycle più usato è il lifecycle **default**, che gestisce la compilazione, i test e la creazione dell'artefatto finale.

| Fase            | Significato                                               |
|-----------------|-----------------------------------------------------------|
| <b>validate</b> | Controlla che il progetto sia configurato correttamente.  |
| <b>compile</b>  | Compila il codice sorgente principale.                    |
| <b>test</b>     | Esegue i test del progetto.                               |
| <b>package</b>  | Crea l'artefatto finale, ad esempio un file <b>.jar</b> . |
| <b>verify</b>   | Esegue controlli aggiuntivi sul risultato della build.    |
| <b>install</b>  | Installa l'artefatto nel repository locale.               |
| <b>deploy</b>   | Pubblica l'artefatto in un repository remoto.             |

Se si esegue il comando **mvn package**, Maven non esegue solo la fase **package**, ma anche le fasi precedenti del lifecycle, come **validate**, **compile** e **test**. Questo rende la build prevedibile e ripetibile.

Un altro comando comune è **mvn clean package**; la fase **clean** elimina i file generati da build precedenti, in particolare la directory **target**. Subito dopo, Maven ricostruisce il progetto ed esegue la fase **package**.

## 16.7 Plugin

I plugin sono componenti che eseguono concretamente le operazioni della build. Maven definisce il ciclo di vita e le sue fasi, ma il lavoro effettivo viene svolto dai plugin. Ad esempio, la compilazione del codice è gestita dal Maven Compiler Plugin, mentre l'esecuzione dei test è gestita da plugin dedicati come Maven Surefire Plugin.

Un plugin può essere configurato nel `pom.xml` dentro la sezione `build`.

```
1 <build>
2   <plugins>
3     <plugin>
4       <groupId>org.apache.maven.plugins</groupId>
5       <artifactId>maven-compiler-plugin</artifactId>
6       <version>3.11.0</version>
7       <configuration>
8         <source>17</source>
9         <target>17</target>
10      </configuration>
11    </plugin>
12  </plugins>
13 </build>
```

In questo esempio viene configurato il plugin che compila il codice Java, specificando che il progetto deve essere compilato usando Java 17.

I plugin possono essere collegati alle fasi del lifecycle. Quando Maven raggiunge una certa fase, esegue i goal dei plugin associati a quella fase. Un *goal* è una specifica operazione eseguibile da un plugin.

Un goal può anche essere eseguito direttamente da riga di comando, come `mvn compiler:compile`. In questo caso Maven esegue il goal `compile` del plugin `compiler`. Nella pratica quotidiana, però, si usano più spesso i comandi legati al lifecycle, come `mvn compile`, `mvn test` o `mvn package`.

I plugin permettono di estendere Maven senza modificare Maven stesso. Grazie ai plugin è possibile compilare codice, eseguire test, generare documentazione, creare archivi, controllare la qualità del codice e pubblicare artefatti.